Highlights

The Multiproduct Two-Stage Capacitated Facility Location Problem: A Parameterized Complexity Analysis

Pedro Henrique González, Rafael Schneider

- First parameterized complexity analysis of the two-stage multiproduct model
- The four size parameters obey a dichotomy around the two facility counts
- No polynomial kernel in the facility counts unless NP lies in coNP/poly
- W[2]-hardness in the opening count, with the XP enumeration optimal under ETH
- The covering bound tracks root relaxation tightness on the public benchmark

The Multiproduct Two-Stage Capacitated Facility Location Problem: A Parameterized Complexity Analysis

Pedro Henrique González[a,*], Rafael Schneider[a]

[a]*Federal University of Rio de Janeiro, Rio de Janeiro, Brazil*

ARTICLE INFO

Keywords:
Facility Location
Supply Chain Design
Parameterized Complexity
Fixed-Parameter Tractability
Kernelization Lower Bounds
Branch-and-Bound
2020 MSC: 90B80
68Q27
90C27

ABSTRACT

In the multiproduct two-stage capacitated facility location problem (MP-TSCFLP), factories and depots open at fixed cost and route products to customers under per-product capacities and demands. On the hundred-instance benchmark of Mauri and coauthors a commercial solver proved no instance optimal within the original limits. Parameterized analyses of facility location cover uncapacitated single-stage variants, and we give, to our knowledge, the first for this model. Unless P equals NP, the four size parameters obey a dichotomy in which the problem is fixed-parameter tractable exactly when the parameter set contains both facility counts. The facility-count parameter admits no polynomial kernel unless NP lies in coNP/poly. One product and one factory already force strong NP-hardness and, unless P equals NP, logarithmic set-cover inapproximability. In the number of opened facilities the problem is W[2]-hard, and under the Exponential Time Hypothesis its XP algorithm is optimal up to the constant in the exponent. With one customer and one product, separable first-stage costs delimit a pseudo-polynomial, only weakly hard slice, while strong hardness persists outside the separable class. A verification battery on the order of nine hundred thousand computational checks, most of them exhaustive design enumerations, validated every hardness gadget and the exact codes, with no failure. Integrality of the data turns any certified absolute gap below one into optimality under the solver's tolerances, yielding three solver-certified optimal values that improve the best values of Mauri and coauthors.

1. Introduction

A manufacturer that supplies many markets rarely ships from the factory gate to the customer in a single leg. Goods leave a small number of production sites, pass through intermediate depots that consolidate and break bulk, and only then reach the retailers. Deciding which factories and which depots to operate, and how to route several products through the two stages so that every demand is met at least total cost, is among the oldest planning questions in distribution logistics [1, 2], and it remains a daily one wherever a supply chain is redesigned or a distribution network is sized. The two layers interact from the start, because a depot is worth opening only if some open factory can feed it and some customer needs what it holds, so the fixed decisions and the transport decisions cannot be taken apart, and it is this coupling that makes the problem more than the sum of its two stages.

This interaction is captured by the multiproduct two-stage capacitated facility location problem, which we abbreviate MP-TSCFLP. Several products flow from candidate factories to candidate depots and on to customers. Opening a factory or a depot costs a fixed charge, each unit shipped on a stage carries a per-product transport cost, and every factory and depot has a per-product capacity, while each customer has a per-product demand that must be satisfied. Both stages are complete bipartite, so any open factory may serve any depot and any open depot may serve any customer. The uncapacitated single-product ancestor of the model is the classical facility location problem, and the two-stage capacitated multiproduct form was already observed to be NP-hard by Pirkul and Jayaraman [3]. The benchmark and the solution methods we build on are those of Mauri et al. [4], who introduced the hundred-instance benchmark and attacked it with two hybrid metaheuristics, and of Morais et al. [5], who later added local branching to a genetic-algorithm hybrid for the same model.

*Corresponding author.
pegonzalez@cos.ufrj.br (P.H. González); rschneider@cos.ufrj.br (R. Schneider)
ORCID(s):

Two facts recur in this literature, and both concern difficulty rather than modelling. First, the instances resist exact solution. On the hundred instances assembled by Mauri et al. [4] a commercial mixed-integer solver solved none to proven optimality within a one-hour budget per instance, which is why the authors turned to heuristics in the first place. Second, this difficulty has no theoretical counterpart. Beyond the inherited statement that the problem is NP-hard [3], no finer-grained account exists of *where* the hardness sits, which structural features drive it, or which of them could be exploited to solve the problem faster.

This paper takes the empirical hardness as the phenomenon to be explained. Facility location as such has a parameterized literature, whose closest antecedent here is the parameterized view of Fellows and Fernau [6], a map of uncapacitated single-stage variants, but no parameterized analysis covers the two-stage capacitated multiproduct model, and we give what is, to our knowledge, the first. We work throughout with the cardinality-constrained decision version, in which a solution may open at most k facilities, counting factories and depots together, and k is the parameter that measures how lean a viable design is allowed to be. Alongside k we study the four size parameters of an instance, the numbers $|I|$ of factories, $|J|$ of depots, $|K|$ of customers and $|L|$ of products, both singly and in every combination. The guiding question is which of these quantities, when held small, tames the exponential behaviour of the problem, and which leave it intractable even when fixed. An answer matters beyond taxonomy, because each tractable combination names a regime in which exact solution scales, and each hard combination names a feature whose cost no exact method avoids unless $\mathsf{P} = \mathsf{NP}$. The analysis is a classification, so the algorithms it produces are devices for locating the difficulty rather than candidates to beat a commercial solver, and its practical residue is an a priori bound that annotates an instance before any optimization is run.

Our findings fall into three groups, one locating the hardness along the four dimensions, one drawing the tractability border together with its preprocessing limits, and one certifying the map computationally and confronting it with a solver. The hardness comes first because its slices are the material from which the border is proved.

First, we locate the hardness. The problem collapses when all fixed charges vanish and when a single factory faces a single depot (Propositions 4 and 5), and hardness begins one step above. With one product and one factory the problem is already strongly NP-hard, the depot-to-customer stage alone encoding a covering problem, and no approximation beats the logarithmic Set Cover threshold unless $\mathsf{P} = \mathsf{NP}$ (Theorem 1, Corollary 1). Fixing any single size parameter to a constant leaves the problem NP-hard, which we prove through four hard groups, each fixing two of the four dimensions to one. Behind the groups sit four discrimination pairs, pairs in which one facility side tells the members of another dimension apart through its cost matrix, a notion Section 4 fixes precisely, so hardness survives every single restriction.

Second, we draw the tractability border. Parameterizing by any subset P of the four size parameters, the problem is fixed-parameter tractable whenever P contains both $|I|$ and $|J|$, through a single $2^{|I|+|J|} \cdot \mathrm{poly}$ algorithm, and para-NP-hard for every other P, so under $\mathsf{P} \neq \mathsf{NP}$ the tractable subsets are exactly those containing both (Theorem 10). MP-TSCFLP is W[2]-hard in the cardinality parameter k, already with one product and one factory and already when only the depots are counted (Theorem 4), and under the Exponential Time Hypothesis the naive n^k enumeration is essentially optimal (Theorem 5). A separate numeric layer, our label for hardness that persists after the combinatorial dimensions are trivialized, governs the smallest instances. With a single customer and a single product the problem is NP-hard, separability of the first-stage cost delimits a pseudo-polynomially solvable part, and strong hardness persists outside the separable class, the general boundary remaining open (Theorem 6 and Theorem 7). On the preprocessing side, the problem admits no polynomial kernel in $|I| + |J|$ unless $\mathsf{NP} \subseteq \mathrm{coNP/poly}$ (Theorem 11), although a polynomial kernel does appear once costs and total demands are polynomially bounded in the larger parameter $|I| + |J| + |L| + \tau$, with τ the number of distinct second-stage cost columns (Proposition 9). The obstruction over the incidence representation we analyse is arithmetic rather than structural, for that incidence graph always has clique-width at most two (Observation 5). Several further positive results, the pseudo-polynomial regimes and the customer aggregation among them, are stated and classified but kept outside the algorithmic core, since our aim is the map and not the tuning of any one route through it.

Third, every construction behind the map is checked computationally. The gadgets were validated by a shared battery of roughly 9×10^5 individual checks, none of which failed, and the enumeration algorithm agrees with an independent brute-force solver across all 1,381 comparisons. The computational study asks in what regime the enumeration is practical and how it stands against Gurobi, the mixed-integer baseline of

Section 6. It then asks how the pruning threshold k^* that our branch-and-bound derives, available before any optimization is run, relates to the gaps a mixed-integer solver reaches under a fixed limit, and at which stage of the solving process the association is anchored. The protocol, instances and seeds are fixed in advance so that every figure can be regenerated.

The remainder of this paper is organized as follows. Section 2 places the model in the exact, heuristic and parameterized literatures and closes with a positioning table. Section 3 fixes the model, proves the routing oracle, that is the subroutine that, given a design, computes its optimal routing cost, together with the feasibility test and the monotonicity on which every later argument rests, and maps the polynomial cases. Section 4 develops the four covering reductions, the numeric layer and the lower bounds for the parameter k. Section 5 gives the enumeration algorithm with its branch-and-bound, the dichotomy theorem and the two sides of the kernelization question, and Section 6 reports the computational study built on them. Section 7 states the two-layer synthesis and five open problems.

2. Related work

Two-stage capacitated facility location entered the operations research literature as a distribution-planning model. General surveys of facility location and of multi-level location are given by Klose and Drexl [1] and Ortiz-Astorquiza et al. [2]. Pirkul and Jayaraman [3] formulated the multiproduct two-stage capacitated version, observed its NP-hardness and solved it by Lagrangian relaxation, and this is the formulation that the later benchmark work inherits. Exact and heuristic effort has continued on both the single- and multi-stage variants. Litvinchev and Ozuna Espinosa [7] derived Lagrangian bounds for two-stage capacitated location, together with an accompanying heuristic, and Fernandes et al. [8] proposed a simple and effective genetic algorithm for the single-product two-stage capacitated problem along with the large benchmark instance sets that the multiproduct studies later reused. On the exact side, Benders decompositions tailored to facility location are a mature computational line [9], and the disaggregated cuts of Section 5.4, derived from the routing oracle of Proposition 1, sit in that tradition. The benchmark this paper builds on is due to Mauri et al. [4], who assembled a hundred multiproduct instances and, finding that a commercial mixed-integer solver closed none of them within the imposed limits, proposed two hybrid metaheuristics, a clustering search matheuristic that calls CPLEX on network-flow subproblems and a biased random-key genetic algorithm with local search. Morais et al. [5] then addressed the same multiproduct problem with a hybrid biased random-key genetic algorithm that adds local branching, extending a single-product hybrid from the same group. Across this line the model is treated as empirically hard, and the hardness is managed rather than explained.

The complexity of facility location itself is well understood in the uncapacitated single-product case, and it is where our covering arguments originate. Without the metric assumption, uncapacitated facility location contains Set Cover, so Hochbaum [10] obtained a greedy $O(\log n)$ approximation, and the lower bounds recorded next show that no better guarantee is possible in general unless $\mathsf{P} = \mathsf{NP}$. The matching lower bounds are those of Feige [11] and, under the weaker assumption $\mathsf{P} \neq \mathsf{NP}$, of Dinur and Steurer [12], which together pin the Set Cover threshold at $(1-\varepsilon)\ln n$. These bounds transfer to the depot-to-customer stage of MP-TSCFLP and account for its inapproximability. The classical hardness of the underlying decision problems traces back to Karp [13], and the distinction between weak and strong NP-hardness that organizes our numeric layer is that of Garey and Johnson [14].

Parameterized complexity has been brought to bear on optimization and covering problems for two decades, and the vocabulary and lower-bound machinery we use are standard [15, 16]. The W[2]-hardness of Set Cover parameterized by the solution size is classical [15], the ETH-based lower bounds we compose with are due to Chen et al. [17], and the compression arguments in the kernelization section rely on the number-reduction technique of Frank and Tardos [18]. The parameterized view of location and network problems has grown quickly, though along axes different from ours. The closest antecedent is the parameterized view of facility location by Fellows and Fernau [6], published in this journal, which establishes W-hardness and FPT results for uncapacitated facility location variants under related parameters. Capacities, a second stage and multiple products all lie outside its scope, and that is the gap the present analysis fills. Facility-location-flavoured clustering admits tight fixed-parameter approximations in the number of centres [19]. Under uniform capacities tight FPT approximations in the number of centres remain available, and constant-factor FPT approximations

with larger factors extend to the general capacitated setting [20]. The survivable network design problem is fixed-parameter tractable in its edge- and element-connectivity forms while turning W[1]-hard for vertex connectivity [21]. Covering objectives on structured graphs, tractable by treewidth alone in their nonnegative form, in general require treewidth combined with maximum degree [22]. Each of the clustering, connectivity and covering results turns on a metric, a connectivity requirement or a graph width, whereas the hardness we isolate is arithmetic, carried by capacities and demands, so their positive machinery does not transfer to the representations analysed here, while richer representations remain unanalysed rather than excluded. The contrast is sharpest on the approximation side. What separates our covering core from the capacitated clustering results is the absence of a metric. There the parameterized lower bounds have recently caught up with the classical ones, since approximating Set Cover parameterized by the solution size within any constant factor is itself W[2]-hard [23]. This suggests the same barrier for MP-TSCFLP, a transfer we state as a conjecture among the open problems of Section 7. Closest in spirit to the present analysis is the parameterized study of minimum-weight solution subgraphs of and-or graphs by Dantas da Silva et al. [24], where a cheapest subnetwork must respect local dependency semantics and zero-weight arcs are exactly what turns a fixed-parameter tractable problem W[2]-hard. Zero fixed charges play a comparable role in our constructions, which require either large numbers, many products or positive charges to produce hardness.

Table 1 closes the section by placing this paper against the works discussed above, one line per selected work and one column per feature, with a mark recording only what the corresponding paragraph of this section states. The last line of the table records a combination of features that no previous line fills, a parameterized analysis of the two-stage capacitated multiproduct model, and building that analysis requires first pinning the model down, which the next section does.

Table 1 Positioning against the related work of this section. A check mark means the feature is treated in the cited work as described above. The circle marks bounds or partial guarantees claimed without an exactness or completeness proof. Blank means not treated.

	two-stage location	multi-product	capa-cities	exact or lower bounds	heuris-tics	parameterized analysis
Pirkul and Jayaraman [3]	✓	✓	✓	○	✓	
Litvinchev and Ozuna Espinosa [7]	✓		✓	○	✓	
Fernandes et al. [8]	✓		✓		✓	
Mauri et al. [4]	✓	✓	✓		✓	
Morais et al. [5]	✓	✓	✓		✓	
Fellows and Fernau [6]				✓		✓
Dantas da Silva et al. [24]				✓		✓
Cohen-Addad and Li [20]			✓	○		✓
Feldmann et al. [21]				✓		✓
This paper	✓	✓	✓	✓		✓

3. Problem definition and structural properties

This section fixes the model, records its decision versions and the parameterized objects we study, and collects the structural facts that the later sections use as tools. The facts are elementary but used throughout the subsequent proofs, so we prove them in full, with the three longest proofs collected in A. Each is additionally checked computationally, over the bounded instance families recorded in the accompanying software archive.

An instance of MP-TSCFLP is given by four finite sets, the factories I, the depots J, the customers K and the products L, together with nonnegative integer data. Opening factory i costs f_i and opening depot j costs g_j. Shipping one unit of product l from factory i to depot j costs c_{ijl}, and shipping it from depot j to customer k costs d_{jkl}. Factory i can supply b_{il} units of product l and depot j can hold p_{jl}, while customer k demands q_{kl}. Both stages are complete bipartite, so no arc is forbidden a priori. A solution opens a set of factories and depots, encoded by $y \in \{0,1\}^{|I|}$ and $z \in \{0,1\}^{|J|}$, and routes the products through nonnegative flows x_{ijl} on the first stage and w_{jkl} on the second. We write $D_l = \sum_{k \in K} q_{kl}$ for the total demand of product l and $n = |I| + |J|$ for the number of candidate facilities. Following Pirkul and Jayaraman [3], Mauri et al.

[4], the problem minimizes total cost,

$$\min \sum_{i\in I} f_i y_i + \sum_{j\in J} g_j z_j + \sum_{i,j,l} c_{ijl}\, x_{ijl} + \sum_{j,k,l} d_{jkl}\, w_{jkl} \tag{1}$$

$$\text{s.t. } \sum_{j\in J} w_{jkl} \geq q_{kl} \qquad \forall k \in K,\ l \in L, \tag{2}$$

$$\sum_{i\in I} x_{ijl} \geq \sum_{k\in K} w_{jkl} \qquad \forall j \in J,\ l \in L, \tag{3}$$

$$\sum_{j\in J} x_{ijl} \leq b_{il}\, y_i \qquad \forall i \in I,\ l \in L, \tag{4}$$

$$\sum_{k\in K} w_{jkl} \leq p_{jl}\, z_j \qquad \forall j \in J,\ l \in L, \tag{5}$$

$$y_i, z_j \in \{0,1\}, \quad x_{ijl}, w_{jkl} \geq 0. \tag{6}$$

Constraint (2) serves each demand, (3) balances flow through a depot, and (4) and (5) cap the throughput of a factory and a depot by its capacity while forcing the facility open before any flow may use it. At $y_i = 0$ the right-hand side of (4) vanishes and nonnegativity forces $x_{ijl} = 0$ for every j and l, and (5) acts on the w_{jkl} in the same way at $z_j = 0$. Capacities and demands are per product and never couple two products, a feature the analysis exploits throughout. Figure 1 shows the two-stage network and the decisions a solution fixes.

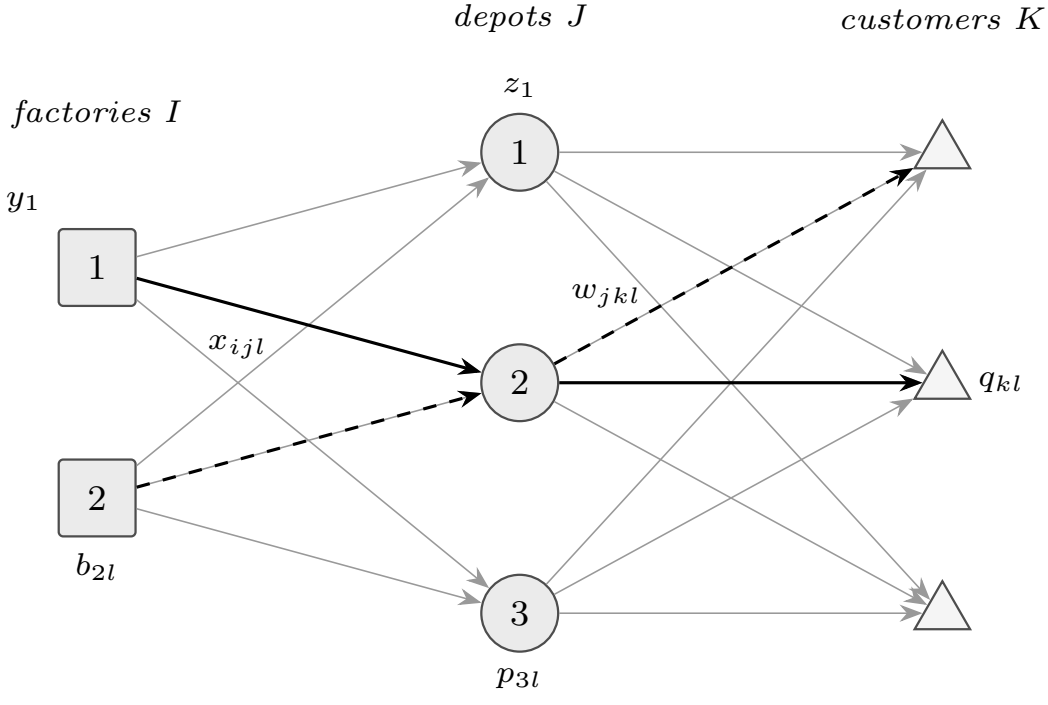


Figure 1 The MP-TSCFLP network of model (1)–(6). A designer opens factories (y_i) and depots (z_j), after which each product flows across two capacitated stages, from factories through depots to customers, along stage-1 arcs carrying x_{ijl} and stage-2 arcs carrying w_{jkl}. Factory capacities b_{il} and depot capacities p_{jl} bound the flow, which must meet every customer demand q_{kl}. The heavy solid arcs trace product $l = 1$ and the heavy dashed arcs product $l = 2$, so the two line styles convey the multiproduct structure, while the light arcs show that both stages are complete. The coupling of the two stages through the depots is the interaction that the reductions of Section 4 turn into hardness.

A tiny instance already shows the interaction that drives the hardness. Take one product and one factory of ample capacity, two depots and three customers, with unit second-stage cost d_{jk} equal to 0 when depot j can reach customer k cheaply and to 1 otherwise, the product index suppressed as there is a single product. Choosing which depots to open so that every customer is served through a zero-cost arc is exactly a set-cover decision, and no amount of first-stage freedom removes it, because the single factory feeds whichever depots

are open. A core of the model's combinatorial difficulty is thus already present in the second stage, and the reductions of Section 4 grow from this observation.

The optimization problem has two decision versions. In the first only the budget is bounded, and in the second the number of opened facilities is bounded as well, which is the version we parameterize.

Definition 1 (MP-TSCFLP(B))**.** Given an MP-TSCFLP instance and an integer $B \geq 0$, decide whether some feasible solution has total cost at most B.

Definition 2 (MP-TSCFLP(B, k))**.** Given an MP-TSCFLP instance and integers $B \geq 0$ and $k \geq 0$, decide whether some feasible solution has total cost at most B and opens at most k facilities, that is $\sum_i y_i + \sum_j z_j \leq k$. The parameter is k.

A tempting alternative would parameterize by the number of facilities open in an optimal solution. Solution-dependent parameters are studied in other settings, but this one is not well defined here, because the optimum is not unique. When a closed facility carries zero fixed charge, opening it yields another optimum, since by Proposition 3 below the total cost cannot rise and optimality of the original solution means it cannot fall. On such instances the count can therefore vary by up to n depending on which optimum one names, so we do not adopt it. Turning the count into an explicit budget repairs the defect, and setting $k = n$ recovers Definition 1, so nothing is lost.

We phrase the complexity statements in the standard parameterized vocabulary, which we recall briefly following Downey and Fellows [15] and Cygan et al. [16]. A parameterized problem is fixed-parameter tractable, in the class FPT, when it is solvable in time $h(p) \cdot \mathrm{poly}(N)$ for a computable function h of the parameter p, with N the instance size, and it lies in XP when the polynomial degree may depend on p. A problem that is NP-hard already at a fixed value of p is para-NP-hard and, unless P = NP, admits neither an FPT nor an XP algorithm. We call a parameterization strongly para-NP-hard when some fixed slice is strongly NP-hard, so that the exclusion survives a unary encoding.[1] Hardness within the W hierarchy is transferred by FPT reductions. An FPT reduction maps an instance of one parameterized problem to an equivalent instance of another in FPT time, with the parameter of the image bounded by a computable function of the source parameter [15, 16]. A parameterized problem is W[2]-hard when some W[2]-complete problem, Set Cover parameterized by the cover size for instance, admits an FPT reduction to it, and a W[2]-hard problem is fixed-parameter tractable only if every problem in W[2] is, which is considered implausible. A kernel is a polynomial-time self-reduction that maps an instance to an equivalent instance of the same problem whose size is bounded by a computable function of p alone, and a polynomial kernel is one whose size bound is polynomial. More generally, a polynomial compression is a polynomial-time reduction that maps an instance of a parameterized problem to an equivalent instance of some fixed problem, not necessarily the same one, whose size is bounded by a polynomial of the parameter alone. A polynomial kernel is thus the special case of a polynomial compression in which the target problem is the problem itself.

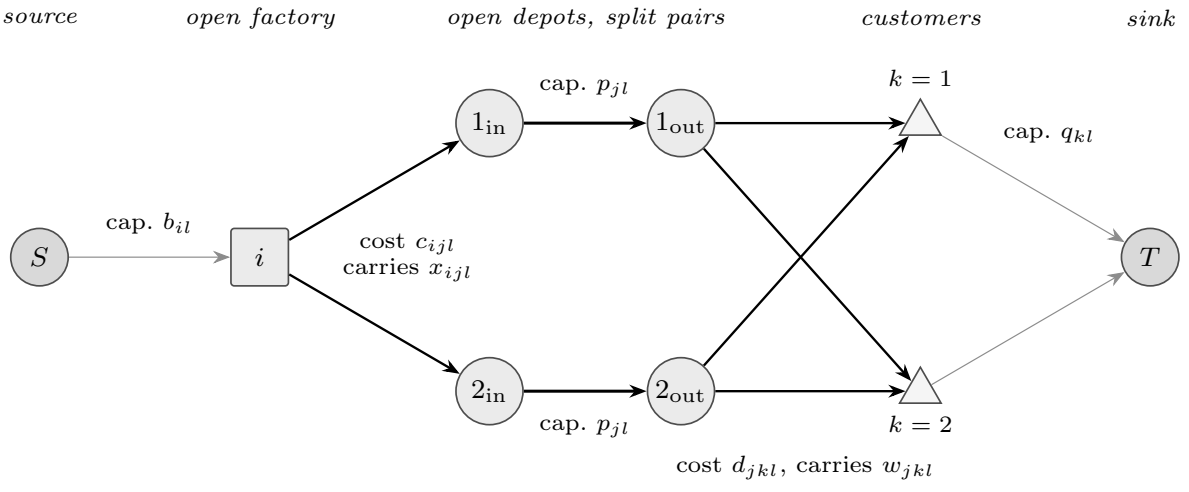
The first structural fact isolates the routing subproblem. Once the open facilities are fixed the remaining decisions are pure transport, and because capacities and costs never couple products the transport splits into one independent flow per product. Each such flow is an ordinary minimum-cost flow on a layered network, which both makes the residual value computable in strongly polynomial time and, by total unimodularity, forces it to be an integer. A routing for the fixed design (y, z) is a pair of nonnegative flow vectors (x, w) satisfying (2)–(5) with (y, z) held fixed, and its block l is the slice of (x, w) formed by the variables of product l. We write $v(y, z)$ for the least total cost of a routing for the design, a minimum that Proposition 1 shows to be attained whenever a routing exists, with the convention $v(y, z) = +\infty$ when no routing exists, that is when the design is infeasible.

The layered network For a fixed design (y, z) and a product l, build the network $N_l(y, z)$ with a source S, a sink T, one node per open factory ($y_i = 1$), one node per customer, and, for each open depot ($z_j = 1$), a split pair $j_{\mathrm{in}} \to j_{\mathrm{out}}$. The arcs are as follows. From the source, each $S \to i$ has capacity b_{il} and cost 0. Each $i \to j_{\mathrm{in}}$ has infinite capacity and cost c_{ijl}, the split arc $j_{\mathrm{in}} \to j_{\mathrm{out}}$ has capacity p_{jl} and cost 0, each $j_{\mathrm{out}} \to k$ has infinite capacity and cost d_{jkl}, and each $k \to T$ has capacity q_{kl} and cost 0. A closed facility contributes no node, so

[1] The term is our coinage for the combination, para-NP-hardness witnessed by a strongly NP-hard slice, and we are not aware of an established name.

its flow is forced to 0, in accordance with the vanishing right-hand sides of (4) and (5) noted after the model. An open depot caps its throughput at p_{jl} through the split arc, which is (5) at $z_j = 1$, and the source-arc capacity b_{il} of an open factory is (4) at $y_i = 1$. Every S–T path has the form $S \to i \to j_{\text{in}} \to j_{\text{out}} \to k \to T$, because every arc leads from one layer of nodes to the next and no arc skips a layer or returns to an earlier one. Figure 2 draws $N_l(y, z)$ for a small design and labels each arc family with its capacity, its cost and the model variable it carries.

Figure 2 The layered network $N_l(y, z)$ of Proposition 1 for one product l, drawn for a design with one open factory, two open depots and two customers. Source arcs $S \to i$ have capacity b_{il} and cost 0, each arc $i \to j_{\text{in}}$ costs c_{ijl} and carries the model variable x_{ijl}, each split arc $j_{\text{in}} \to j_{\text{out}}$ has capacity p_{jl} and cost 0, each arc $j_{\text{out}} \to k$ costs d_{jkl} and carries w_{jkl}, and each demand arc $k \to T$ has capacity q_{kl} and cost 0. This correspondence between the arc families and the model variables is the content of the proposition's proof.



Proposition 1 (routing oracle)**.** *Fix $(y, z) \in \{0,1\}^{|I|} \times \{0,1\}^{|J|}$. The residual routing problem separates over products, and for each product l its optimal cost equals the minimum cost μ_l of an S–T flow of value D_l in the layered network $N_l(y, z)$ defined above, with $\mu_l = +\infty$ when no such flow exists. The finite μ_l are attained by integral flows and are nonnegative integers, and $v(y, z) = \sum_l \mu_l$. Hence $v(y, z)$ is computable in strongly polynomial time by $|L|$ minimum-cost flow computations [25].*

The proof identifies each per-product block with a minimum-cost flow of value D_l in the layered network $N_l(y, z)$, matching routings and flows in both directions at equal cost after a trimming step that tightens (2) and (3). Total unimodularity of the node–arc incidence matrix makes every finite block optimum an integer attained by an integral flow. Summing the blocks delivers the identity $v(y, z) = \sum_l \mu_l$ used throughout the paper. The full proof is in A.

Whether a fixed design can serve the demand at all is settled without solving any flow, by an aggregate test on each product. The completeness of the two stages is what makes the test exact, since any open factory reaches any open depot and any open depot reaches any customer, so only the totals matter.

Proposition 2 (feasibility)**.** *A design (y, z) is feasible if and only if, for every product l,*

$$\sum_{i \in I} b_{il}\, y_i \;\geq\; D_l \quad \textit{(F1)} \qquad \textit{and} \qquad \sum_{j \in J} p_{jl}\, z_j \;\geq\; D_l \quad \textit{(F2)}. \tag{7}$$

The test runs in $O((n + |K|)\,|L|)$ time, of which $O(|K|\,|L|)$ computes the totals D_l.

Necessity follows by aggregating the four constraint groups per product, which bounds the served demand by the total open capacity of each of the two layers, a two-cut argument. Sufficiency exhibits an explicit flow of value D_l in $N_l(y, z)$ through northwest-corner transport constructions on the two complete stages, so the aggregate test (F1) and (F2) is exact. The full proof is in A.

The characterization rests on the completeness of the two stages in an essential way, and the following counterexample shows that the dependence is real rather than an artefact of the proof.

Remark 1 (completeness is essential). The aggregate test fails once an arc may be forbidden, a forbidden arc being modelled by deleting its flow variable. Take one product, two factories, two depots and one customer, and open all four facilities. Since there is a single product and a single customer we drop those indices and write b_i, p_j and D for the capacities and the demand, with $b_1 = b_2 = 1$, $p_1 = p_2 = 1$ and $D = 2$, so (F1) and (F2) both read $2 \geq 2$ and hold. Suppose the first stage is sparse and both factories may ship only to depot 1. Depot 2 then receives nothing, so (3) forces it to forward nothing, while depot 1 forwards at most one unit by (5). The customer therefore receives at most one unit, which violates (2) against the demand of two, so no routing exists although both tests pass. The per-product totals no longer determine feasibility, which is why the characterization is stated only for the complete model.

Opening more facilities preserves feasibility and never raises the routing cost, only the fixed charge. This monotonicity underlies both the polynomial cases below and the admissible bound used by the enumeration algorithm.

Proposition 3 (monotonicity)**.** *If $(y, z) \leq (y', z')$ componentwise then feasibility of (y, z) implies feasibility of (y', z'), and $v(y', z') \leq v(y, z)$ always, with the convention $v = +\infty$ on infeasible designs. In particular the all-open design attains the least routing value among all feasible designs.*

The feasibility part holds because the left-hand sides of (F1) and (F2) are nondecreasing in the design. For the value part, $N_l(y, z)$ is a subnetwork of $N_l(y', z')$, so every value-D_l flow of the smaller network is one of the larger network at the same cost and no block optimum increases. Enlarging a design therefore preserves feasibility and never raises the routing value, which is the form in which later sections use the proposition. The full proof is in A.

Two regimes collapse to polynomial-time problems. When all fixed charges vanish the fixed cost drops out and monotonicity makes opening everything optimal, and when a single factory faces a single depot the routing is forced and the optimum has a closed form.

Proposition 4 (polynomial cases)**.** *If $f \equiv 0$ and $g \equiv 0$, then MP-TSCFLP is solvable in strongly polynomial time, its instance is feasible exactly when $\sum_i b_{il} \geq D_l$ and $\sum_j p_{jl} \geq D_l$ for every l, and its optimum is the all-open routing value $v(\mathbf{1}, \mathbf{1})$.*

Proof. With no fixed charge the cost of a design is its routing value alone. By Proposition 3 the all-open design attains the least routing value among feasible designs, and by Proposition 2 it is feasible precisely under the displayed conditions, which are (F1) and (F2) at $(y, z) = (\mathbf{1}, \mathbf{1})$. If they fail, no design at all is feasible, for every design lies componentwise below $(\mathbf{1}, \mathbf{1})$, so its feasibility would imply that of the all-open design by the feasibility part of Proposition 3. In the feasible case the all-open design is itself a solution of cost $v(\mathbf{1}, \mathbf{1})$, and no feasible design costs less, so the optimum equals $v(\mathbf{1}, \mathbf{1})$, computed in strongly polynomial time by Proposition 1. □

The second polynomial regime fixes one facility on each side. There the design is forced and the optimum admits a closed form.

Proposition 5 (single factory and depot)**.** *If $|I| = |J| = 1$, then MP-TSCFLP is solvable in $O(|K|\,|L|)$ time after reading the input, and feasibility reduces to $b_{1l} \geq D_l$ and $p_{1l} \geq D_l$ for every l. When the instance is feasible and some demand is positive the optimum equals $f_1 + g_1 + \sum_l \big(c_{11l} D_l + \sum_k d_{1kl} q_{kl}\big)$, and when all demand is zero the optimum is 0, attained by opening nothing.*

Proof. Feasibility forces the design, and nonnegativity of the costs then makes the cheapest routing explicit. By Proposition 2, a design (y_1, z_1) is feasible if and only if $b_{1l} y_1 \geq D_l$ and $p_{1l} z_1 \geq D_l$ for every l, which is what (F1) and (F2) read with one facility per side. If all demand is zero, these conditions hold for every design and the stated condition holds because every D_l equals 0. In that case the empty design is feasible, since (F1) and (F2) read $0 \geq 0$ and the all-zero routing satisfies (2)–(5), and its cost 0 is optimal by nonnegativity of the costs. Otherwise some $D_l > 0$, and any feasible design has $y_1 = z_1 = 1$, since $y_1 = 0$ or $z_1 = 0$ would make (F1) or (F2) read $0 \geq D_l > 0$. For such a design the conditions become $b_{1l} \geq D_l$ and $p_{1l} \geq D_l$, so the instance is feasible exactly under the stated condition, and feasibility forces the fixed charge to be exactly $f_1 + g_1$. The only remaining variables are x_{11l} and w_{1kl}. Feasibility needs $w_{1kl} \geq q_{kl}$ by (2) and $x_{11l} \geq \sum_k w_{1kl} \geq D_l$

by (3), and since all costs are nonnegative the cheapest choice is $w_{1kl} = q_{kl}$ and $x_{11l} = D_l$, of cost $\sum_k d_{1kl} q_{kl}$ on the second stage and $c_{11l} D_l$ on the first. This choice is itself feasible, since (4) reads $x_{11l} = D_l \le b_{1l}$ and (5) reads $\sum_k w_{1kl} = D_l \le p_{1l}$, both guaranteed by the feasibility condition, while (2) and (3) hold with equality. No feasible routing costs less, since its variables dominate this choice componentwise by the bounds just derived and every cost coefficient is nonnegative. Summing over l and adding $f_1 + g_1$ gives the closed form, evaluated in $O(|K||L|)$ time. □

Finally, the cardinality version sits in XP by brute force over the opening pattern, a bound the algorithms of Section 5 later refine and, under the Exponential Time Hypothesis, match up to the constant in the exponent.

Observation 1 (membership in XP)**.** MP-TSCFLP(B, k) *is decidable in time* $n^{k+O(1)} \cdot \mathrm{poly}(N)$*, with* N *the instance size. When* $n = 0$ *the empty design is the only design, one application of Propositions 1 and 2 to it decides the instance in polynomial time, and the counting below is not needed, so assume* $n \ge 1$*. One may further normalize* $k \le n$*, since no design opens more than* n *facilities. Enumerating every set* $S \subseteq I \cup J$ *of at most* k *facilities then decides the instance. Each* S *is identified with the design* (y, z) *whose indicators it defines, so that* $v(S)$ *denotes* $v(y, z)$*, feasibility of* S *is tested by Proposition 2, and for feasible* S *the total cost* $\sum_{i \in S \cap I} f_i + \sum_{j \in S \cap J} g_j + v(S)$ *is evaluated with* $v(S)$ *from Proposition 1. The answer is* YES *if and only if some such cost is at most* B*, since by the definition of* v *the cheapest solution whose opening pattern is* S *costs exactly this amount, and every solution opening at most* k *facilities has its pattern among the enumerated sets. The number of sets is* $\sum_{r=0}^{k} \binom{n}{r} \le \sum_{r=0}^{k} n^r \le (k+1)\, n^k$*, using* $\binom{n}{r} \le n^r \le n^k$ *for* $r \le k$ *and* $n \ge 1$*. Each set is processed in polynomial time, and the factor* $k + 1 \le n + 1$ *is absorbed into the* $n^{O(1)}$ *term, which gives the stated bound.*

4. Hardness

This section locates the intractability of MP-TSCFLP along the four dimensions of an instance. We first show that shrinking two of the four sets to a single element already forces strong NP-hardness and logarithmic inapproximability, and then that the same covering mechanism can be lodged in the products and, in the numeric layer, in the factories themselves rather than in the customers. The cardinality parameter k then inherits W[2]-hardness together with a tight ETH lower bound, and finally a purely numeric group of instances, in which both demand dimensions are trivial, turns out to be hard as well. The reductions grow from the second-stage set-cover decision already visible in Section 3, and the computational checks behind the constructions are reproducible from the accompanying software archive. Two conventions hold throughout the section. In every construction with a single product the product index is suppressed, so the data of that product are written b_i, p_j, q_k, c_{ij} and d_{jk}. We also impose a nondegeneracy convention on the source instances, requiring the universe U of each Set Cover instance to be nonempty and the item total A of each Partition instance to be positive, for an instance violating either requirement is a yes-instance witnessed by the empty solution. Three recurring terms organize the section. The covering layer, or covering mechanism, is the Set Cover structure a construction embeds, and the discriminating layer is the facility side whose cost matrix tells the members of the covered dimension apart. The numeric layer is the hardness that survives when both demand dimensions are trivial.

The constructions of the section share one architecture. Theorem 1 is the canonical covering reduction, Theorem 2 and Corollary 3 relocate the same mechanism to the products, discriminated by the factories in the first and by the depots in the second, and Corollary 1 and Theorem 4 are consequences of the canonical reduction rather than new constructions. Theorem 6 completes the fourth discrimination pair by a different mechanism, the forced saturation of the factories, and Table 2 summarizes the constructions side by side.

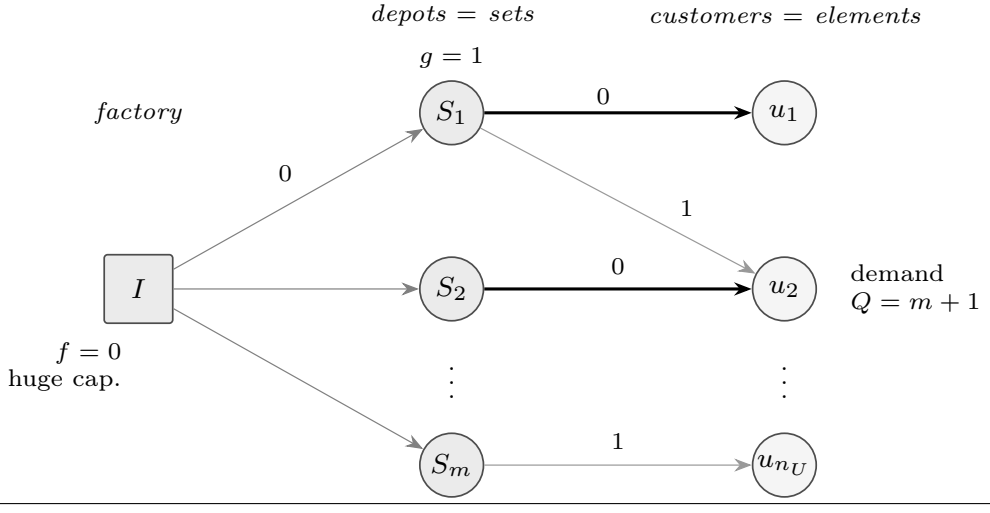
Table 2 The covering constructions of Section 4, one column per construction. Each column names the dimension whose members play the Set Cover elements, the facility side whose openings choose the sets, the cost block in $\{0, 1\}$ that discriminates the covered dimension, the openings forced by feasibility, and the amplifier Q that makes an uncovered element incompatible with the budget, with U the source universe, $n_U = |U|$, m the number of sets and $Q = m + 1$ throughout. Theorem 4 inherits the construction of the first column unchanged, with the cardinality bound $k = t + 1$.

	Thm. 1	Thm. 2	Cor. 3	Thm. 6
covered dimension	customers, $K = U$	products, $L = U$	products, $L = U$	factories, $I = U$
deciding side	depots, $g \equiv 1$	factories, $f \equiv 1$	depots, $g \equiv 1$	depots, $g \equiv 1$
discriminating block	second stage, $d_{ju} = 0$ iff $u \in S_j$	first stage, $c_{i1l} = 0$ iff $l \in S_i$	second stage, $d_{j1l} = 0$ iff $l \in S_j$	first stage, $c_{ij} = 0$ iff $i \in S_j$
forced openings	the single free factory	the single free depot	the single free factory	all n_U free factories, saturated
amplifier	$Q = m + 1$ on each demand q_u	$Q = m + 1$ on each demand q_{1l}	$Q = m + 1$ on each demand q_{1l}	$Q = m + 1$ on each factory load

4.1. Two restrictions already suffice

After the preliminaries the first question is how much of the model can be disabled before the difficulty disappears, and the answer is that almost all of it can. One product and one factory, no factory charge, uniform unit depot charges, and second-stage costs restricted to two values leave a subclass that any casual reading would call trivial. The single factory feeds whichever depots are open, so the only decision left is which depots to install, and choosing them so that every customer is reached through a zero-cost arc is a set-cover decision in disguise. Figure 3 draws the reduced instance. The hypotheses of the next theorem are severe, and the difficulty survives all of them at once.

Figure 3 The Set Cover reduction behind Theorem 1, with a single product and a single free factory of ample capacity. Each depot is a set S_j with opening cost 1, each customer is a universe element with demand $Q = m + 1$, and a heavy stage-2 arc costs 0 when the element lies in the set while a light arc costs 1. Opening a depot is choosing its set into the cover, an uncovered element can be served only at cost at least $Q > B$, and the budget $B = t$ is therefore attainable exactly when a cover of size t exists.



Theorem 1. MP-TSCFLP(B) *is strongly NP-hard even when, simultaneously,* $|L| = 1$ *and* $|I| = 1$*, the charges satisfy* $f \equiv 0$ *and* $g \equiv 1$*, all transport costs lie in* $\{0, 1\}$*, with in fact* $c \equiv 0$ *and* $d \in \{0, 1\}$*, and every number of the instance is bounded by* $|K|\,(|J| + 1)$*. In particular the problem is NP-hard already under a unary encoding of the data.*

We sketch the reduction before giving the formal proof. From an instance of Set Cover with universe U, family $\mathcal{S} = \{S_1, \dots, S_m\}$ and target t we build one depot per set, one customer per element with demand $Q = m+1$, and a single free factory of ample capacity. The second-stage cost d_{ju} is zero when $u \in S_j$ and one otherwise, so a customer whose element is covered by some open depot is served for free, while a customer left uncovered pays one unit on every unit of its demand and therefore at least Q. Since $Q = m + 1$ exceeds both the number of depots and the target t, a solution can stay within budget $B = t$ only if its open depots cover U, and then its cost is exactly the number of open depots. Deciding whether budget t is attainable is thus deciding whether a cover of size t exists. The amplifier Q is what makes an uncovered element incompatible with the budget, and the per-unit accounting is what keeps divisible service from diluting the penalty.

Proof. The plan follows the sketch, with the amplifier making the accounting exact. We reduce from Set Cover, which is NP-complete [13], under the harmless convention $1 \le t \le m$. The remaining values of t are decidable in polynomial time, since an instance with $t = 0$ is a yes-instance exactly when $U = \emptyset$, while an instance with $t > m$ is a yes-instance exactly when $\bigcup_j S_j \supseteq U$. The nondegeneracy convention guarantees in addition that every cover of U is nonempty.

Construction. Put $L = \{1\}$ and $I = \{1\}$ with $f_1 = 0$ and capacity $b_1 = n_U Q$, where $n_U = |U|$ and $Q = m + 1$. Take one depot per set, $J = \{1, \dots, m\}$ with $g_j = 1$ and $p_j = n_U Q$, and one customer per element, $K = U$ with demand $q_u = Q$. Set $c_{1j} = 0$ for every j, set $d_{ju} = 0$ when $u \in S_j$ and $d_{ju} = 1$ otherwise, and ask whether $\mathrm{OPT} \le B = t$. The construction runs in $O(n_U m)$ time and its largest number is $n_U(m + 1) \le |K|(|J| + 1)$.

Feasibility. By Proposition 2 a design that opens the factory and a depot set Z is feasible if and only if $b_1 \ge D_1$ and $\sum_{j \in Z} p_j \ge D_1$, where the single product has total demand $D_1 = \sum_u q_u = n_U Q$. Since $b_1 = n_U Q$ and each $p_j = n_U Q$, (F1) holds whenever the factory is open and (F2) holds whenever $Z \neq \emptyset$. Positive demand, guaranteed by the nondegeneracy convention, forces the factory open, so the feasible designs are exactly those with $Z \neq \emptyset$, and the empty design is infeasible.

Correctness. Let $Z \subseteq J$ be the depots opened by a feasible solution. Because $f_1 = 0$ and $c \equiv 0$, its cost equals $|Z| + \sum_{j \in Z} \sum_{u \notin S_j} w_{ju}$, the transport sum ranging over open depots because a closed depot forwards nothing by (5). By constraint (2) every unit sent to an uncovered customer travels on an arc of cost one, so the cost is at least $|Z| + Q \cdot V_Z$, where V_Z counts the elements no open depot covers. This lower bound holds for arbitrary divisible flows, since the second term is charged one unit per unit of flow and an uncovered customer has no cheaper arc available. It is attained by routing each covered customer through a covering depot and each uncovered one through any open depot, with the first-stage flow x_{1j} set to the outflow of depot j, a choice that costs nothing since $c \equiv 0$. The capacities accommodate this routing, because $b_1 = p_j = n_U Q$ equals the total demand and so (3)–(5) hold. Now, if C covers U with $|C| \le t$, then C is nonempty by the nondegeneracy convention, so opening $Z = C$ is feasible and serves every customer for free at cost $|C| \le t$. Conversely a solution of cost at most t cannot leave an element uncovered, for that would force cost at least $|Z| + Q \ge Q = m + 1 > m \ge t$, so its open depots form a cover, and cost $\ge |Z|$ gives $|Z| \le t$. Neither direction used any restriction beyond the budget, a fact that Theorem 4 reuses when a cardinality bound is added.

All numbers are bounded by $n_U(m + 1)$ and hence polynomial in the input size, so the subclass remains NP-hard when its data are written in unary, which is strong NP-hardness in the sense of Garey and Johnson [14]. A pseudo-polynomial algorithm would run in polynomial time on this subclass, whose numbers are polynomially bounded, and would therefore give $\mathsf{P} = \mathsf{NP}$. □

For coverable instances the value of the reduced instance equals the minimum cover size, so the approximation threshold of Set Cover transfers verbatim. The only care needed is that a multiplicative factor below one is unsatisfiable once the optimum is at least one, which holds here because a depot must open, and this is why the factor is stated as a maximum with one.

Corollary 1. *For every $\varepsilon > 0$, unless $\mathsf{P} = \mathsf{NP}$, no polynomial-time algorithm approximates the optimization version of MP-TSCFLP within a factor $\max\{1, (1-\varepsilon)\ln|K|\}$, even with $|L| = |I| = 1$, $f \equiv 0$, $g \equiv 1$, $c \equiv 0$, $d \in \{0,1\}$ and all numbers polynomially bounded in the instance size. The same conclusion follows under the stronger assumption $\mathsf{NP} \not\subseteq \mathrm{DTIME}(n^{O(\log\log n)})$ from the earlier bound of Feige [11].*

Proof. Enlarging the amplifier forces even approximately optimal solutions to be covering. Rerun the construction with the larger amplifier $Q' = n_U m + m + 1$, so that $q_u = Q'$ and $b_1 = p_j = n_U Q'$, whose only effect is to widen the penalty. The cost identity of Theorem 1, namely that any feasible solution costs at least $|Z| + Q' V_Z$ with equality for the routing described there, is unchanged. The optimum of a coverable instance therefore equals its minimum cover size t^*, attained by covering designs, since any non-covering design costs at least Q' and $Q' > m \geq t^*$ prevents it from being optimal. Write $\rho = \max\{1, (1-\varepsilon)\ln n_U\}$ for the claimed factor, and note $\rho \leq 1 + \ln n_U$ and $t^* \leq m$, so a ρ-approximation returns a solution of cost at most $\rho\, t^* \leq (1 + \ln n_U)\, m$. Since $\ln n_U < n_U$ this is below $Q' = (n_U + 1)m + 1$, so the returned design is covering. The returned solution costs at least $|Z|$ by the identity, where Z denotes its open depots, so $|Z| \leq \rho\, t^*$. As $|K| = |U| = n_U$, this yields a polynomial-time approximation of factor ρ for minimum set cover on the coverable instances. The hardness of Dinur and Steurer [12] concerns exactly that minimization problem, whose instances are coverable by definition, since a cover is required for the objective to exist. On instances with $(1-\varepsilon)\ln n_U < 1$ the universe has constant size at most $e^{1/(1-\varepsilon)}$, every minimal cover has at most n_U sets, and exhaustive search over the polynomially many subfamilies of at most n_U sets solves the problem exactly in polynomial time. On all remaining instances $\rho = (1-\varepsilon)\ln n_U$, so the combined algorithm achieves the factor $\max\{1, (1-\varepsilon)\ln|U|\}$ on every instance of minimum set cover, exactly on the small universes and within $(1-\varepsilon)\ln|U|$ elsewhere. In either regime the combined procedure would approximate minimum set cover within a factor smaller than the threshold of Dinur and Steurer [12] on every instance, and that is impossible unless $\mathsf{P} = \mathsf{NP}$. The variant under Feige's assumption is identical. □

Fixing a set to a single element removes tractability entirely rather than merely obstructing efficiency, which is the parameterized reading of the same reduction.

Corollary 2. MP-TSCFLP(B) *is strongly para-NP-hard with respect to $|L|$, with respect to $|I|$, and with respect to the pair $(|L|, |I|)$. Unless $\mathsf{P} = \mathsf{NP}$, none of these parameters admits an XP algorithm, and a fortiori none is FPT.*

Proof. The claims follow from the fixed slice. The hard subclass of Theorem 1 has $|L| = |I| = 1$, so a fixed slice of each parameter is already NP-hard, and strongly so by the same theorem. A parameterized problem NP-hard at a fixed parameter value is para-NP-hard, and an XP algorithm would decide that slice in polynomial time [15]. □

Before the covering elements migrate to other dimensions of the model, we record what the uniform costs of the construction do and do not imply.

Remark 2. The uniformity in Theorem 1 is deliberate. All of the difficulty is already present with $f \equiv 0$, $g \equiv 1$ and binary transport, so parameters counting distinct cost values, or bounding the ratio of the positive costs, are constant on this hard subclass and help nothing on their own. The instance is, in essence, a non-metric uncapacitated facility location problem with a free source stage attached, and its equivalence with Set Cover traces back to Hochbaum [10]. What the theorem contributes is not the technique but the location of the difficulty in the most restricted subclass the model contains.

4.2. Relocating the covering elements

Theorem 1 kept the customers as the covering elements and let the depots do the discriminating, but nothing forces that assignment. If the products carry the elements and the factories discriminate through their first-stage costs, the same amplifier argument reaches a group of instances that Theorem 1 cannot, namely one where the customers collapse to a single point. The hypotheses stay just as tight, and the difficulty is now carried by the product dimension.

Theorem 2. MP-TSCFLP(B) *is strongly NP-hard even when, simultaneously, $|J| = |K| = 1$, the charges satisfy $f \equiv 1$ and $g_1 = 0$, first-stage costs lie in $\{0,1\}$ while second-stage costs vanish, and every number is bounded by $|I| + 1$. The difficulty is carried by $|L|$, which remains unrestricted.*

The elements have migrated from the customers to the products and the discriminating layer from the depots to the factories, and the hard group moves to $|J| = |K| = 1$ accordingly.

Proof. Again the source is Set Cover [13], with $1 \le t \le m$ and $Q = m + 1$.

Construction. Let $L = U$ and $I = \{1, \dots, m\}$ with $f_i = 1$ and $b_{il} = Q$ for every l. Take one depot with $g_1 = 0$ and $p_{1l} = Q$, one customer with $q_{1l} = Q$, first-stage cost $c_{i1l} = 0$ when $l \in S_i$ and 1 otherwise, second-stage cost $d \equiv 0$, and budget $B = t$. The construction takes $O(n_U m)$ time, since each cost c_{i1l} is a membership test. Full capacity on every product makes feasibility of a design independent of coverage, so coverage is punished only in the cost.

Correctness. By Proposition 2, applied per product with $b_{il} = p_{1l} = Q$ and $D_l = q_{1l} = Q$, a design (Y, z_1) is feasible exactly when $Y \neq \emptyset$ and $z_1 = 1$. Products exist because $L = U \neq \emptyset$ under the nondegeneracy convention, and for each product (F1) reads $Q|Y| \ge Q$ and holds precisely when $Y \neq \emptyset$, while (F2) reads $Qz_1 \ge Q$ and forces $z_1 = 1$, an opening that is free since $g_1 = 0$. A feasible solution with open factories Y and $z_1 = 1$ costs $|Y| + \sum_l \sum_{i \in Y,\, l \notin S_i} x_{i1l}$, where the sum ranges over open factories only, by (4). For each product the depot must receive Q units by (2)–(3). If no open factory contains that product all Q units travel on cost-one arcs, so the cost is at least $|Y| + Q\,V_Y$, where V_Y counts uncovered products. Equality is attained by routing each covered product through a containing factory and each uncovered product through any open factory, within the per-product capacities $b_{il} = Q$, and the second stage forwards the Q units to the customer at no charge. In the forward direction a cover C opens $Y = C$ and $z_1 = 1$ at cost $|C| \le t$. Conversely a solution of cost at most $t < Q$ leaves no product uncovered, since one uncovered product would force cost at least $|Y| + Q \ge 1 + Q > m \ge t$, feasibility having forced $Y \neq \emptyset$, and then cost $\ge |Y|$ gives $|Y| \le t$, a cover. The largest number is $Q = m + 1$, so the hardness is strong [14]. □

The stages of the model are not interchangeable, because the balance constraint (3) orients the flow, so the mirror image with depots as the discriminating layer needs its own short argument rather than a symmetry appeal.

Corollary 3. MP-TSCFLP(B) *is strongly NP-hard even when, simultaneously,* $|I| = |K| = 1$*, the charges satisfy* $f_1 = 0$ *and* $g \equiv 1$*, first-stage costs vanish while second-stage costs lie in* $\{0, 1\}$*, and every number is bounded by* $|J| + 1$*.*

Proof. The construction of Theorem 2 transfers across the two stages, and the proof is the following list of changes. Exchange the two facility layers, so the sets become depots $J = \{1, \dots, m\}$ with $g_j = 1$ and $p_{jl} = Q$, the single free facility becomes a factory with $f_1 = 0$ and $b_{1l} = Q$, and the penalty moves onto the second stage, $c \equiv 0$ and $d_{j1l} = 0$ when $l \in S_j$ and 1 otherwise, with $L = U$, the demands $q_{1l} = Q$, the budget $B = t$ and the construction time unchanged. In the correctness argument the two feasibility tests swap parts, (F1) reading $Qy_1 \ge Q$ and forcing $y_1 = 1$ at no charge, (F2) reading $Q|Z| \ge Q$, that is $Z \neq \emptyset$. The cost of a feasible solution becomes $|Z| + \sum_l \sum_{j \in Z,\, l \notin S_j} w_{j1l}$ with the sum over open depots by (5), and the lower bound $|Z| + Q\,V_Z$, with V_Z the number of uncovered products, holds as before, each product's Q units reaching the customer by (2). Equality is attained by the routing of Theorem 2 with each first-stage flow x_{1jl} set equal to w_{j1l}, at no cost since $c \equiv 0$, within $b_{1l} = Q$ and satisfying (3) and (4). The two directions of the equivalence then read verbatim with Y replaced by Z, one uncovered product forcing cost at least $1 + Q > m \ge t$. Every number of the image is again at most $Q = m + 1$, so the reduction is unary-robust and strong NP-hardness follows [14]. □

Because these slices are strongly hard, the customer and depot parameters already fail at the value one, and unlike the weak slice below the difficulty does not vanish under a unary encoding.

Corollary 4. MP-TSCFLP(B) *is strongly para-NP-hard with respect to* $|K|$*, to* $|J|$*, and to each of the pairs* $(|J|, |K|)$ *and* $(|I|, |K|)$*. No* XP *algorithm exists for any of these parameters unless* P = NP*, and every one of these exclusions holds even under a unary encoding.*

Proof. Two fixed slices supply the claims. Theorem 2 gives a strongly NP-hard slice with $|J| = |K| = 1$ and $|L|$ free, and Corollary 3 one with $|I| = |K| = 1$. The parameterized reading is that of Corollary 2. □

Where no facility layer sees a demand dimension it can distinguish, the covering mechanism becomes inactive and only the numbers remain hard. This happens exactly with one depot and one product, and there the difficulty drops from strong to weak.

Theorem 3. MP-TSCFLP(B) *is NP-hard even when* $|J| = |L| = |K| = 1$, $g_1 = 0$, *all transport costs vanish, and* $f_i = b_i$ *for every factory. The hardness is only weak, since this group of instances is solvable in pseudo-polynomial time* $O(|I| \cdot (D + b_{\max}))$ *with* $D = q_{11}$ *and* $b_{\max} = \max_i b_i$, *and is therefore in* P *under a unary encoding, hence not strongly NP-hard unless* P = NP.

Proof. The reduction is from Partition [13], which asks whether integers $a_1, \dots, a_m$ with even sum A admit a subset summing to $A/2$, and the matching pseudo-polynomial program follows it.

Construction. Take $L = K = \{1\}$ with demand $q = A/2$, one factory per item with $f_i = b_i = a_i$, one depot with $g_1 = 0$ and $p_1 = A/2$, all transport costs zero, and budget $B = A/2$.

Correctness. Every transport coefficient is zero and $g_1 = 0$, so the cost of any feasible solution is $\sum_{i \in S} a_i$ with S the open factories. By Proposition 2 the design is feasible exactly when $z_1 = 1$ and $\sum_{i \in S} a_i \geq A/2$. The first condition is forced by (F2), which reads $p_1 z_1 = (A/2) z_1 \geq A/2$ with $A > 0$ by the nondegeneracy convention, and the opening carries no charge. The second condition is (F1). A partition S^* opens a feasible solution of cost exactly $A/2$, and a feasible solution of cost at most $A/2$ has $\sum_S a_i \geq A/2$ and $\leq A/2$, hence a partition. The instance numbers are the a_i themselves, written in binary, so the reduction is not unary-robust and the hardness is only weak [14].

The matching program. On the general group of the statement the depot capacity p_1 is arbitrary, so we first test $p_1 \geq D$ in $O(1)$ time and declare the instance infeasible when the test fails. When it holds the depot opens at no charge, the cost of opening S is $\sum_{i \in S} f_i = \sum_{i \in S} b_i$, and feasibility is $\sum_{i \in S} b_i \geq D$ by Proposition 2, so the problem is the knapsack-cover of minimizing a subset sum subject to reaching D. Let $T(i, s) \in \{0, 1\}$ record whether some subset of the first i factories has capacity sum exactly s, for $0 \leq s \leq D + b_{\max}$. Sums beyond $D + b_{\max}$ are irrelevant, for if a feasible S has $\sum_{i \in S} b_i \geq D + b_{\max}$ then removing any item of S keeps the sum at least D, so repeated removal produces a feasible subset of no larger cost whose sum lies below $D + b_{\max}$. The recurrence $T(i, s) = T(i-1, s) \vee T(i-1, s - b_i)$, with $T(0, 0) = 1$, with $T(0, s) = 0$ for $s > 0$, and with the convention $T(i-1, s - b_i) = 0$ when $s < b_i$, fills the table in $O(|I| \cdot (D + b_{\max}))$ time. By induction on i the recurrence is sound and complete, every set bit corresponding to a genuine subset sum and every subset sum in range setting its bit. The optimum is the least $s \geq D$ with $T(|I|, s) = 1$, and if no such s exists the instance is infeasible, since the removal argument above would otherwise produce a feasible sum within the range of the table. The table is pseudo-polynomial, so under a unary encoding this group lies in P and the weak hardness is best possible. □

The same weak reduction runs with the two facility layers exchanged, which is what closes the last quadrant of the dichotomy, the one that keeps the depots free.

Corollary 5 (depot mirror of the weak group)**.** MP-TSCFLP(B) *is NP-hard even when* $|I| = |L| = |K| = 1$, $f_1 = 0$, *all transport costs vanish, and* $g_j = p_j$ *for every depot. The hardness is only weak, since this group is solvable in pseudo-polynomial time and hence lies in* P *under a unary encoding.*

Proof. Exchanging the two facility layers in the construction of Theorem 3 yields the reduction, and only the following changes are needed. The items become depots with $g_j = p_j = a_j$, the single free depot becomes a factory with $f_1 = 0$ and $b_1 = A/2$, and the customer, the demand $q = A/2$, the zero transport costs and the budget $B = A/2$ are unchanged. The two feasibility tests exchange roles in the correctness argument, (F1) reading $b_1 y_1 = (A/2) y_1 \geq A/2$ with $A > 0$ by the nondegeneracy convention, forcing the free factory opening, and (F2) supplying the covering test $\sum_{j \in Z} a_j \geq A/2$ over the open depots Z. The equivalence between solutions of cost at most $A/2$ and partitions then reads word for word as in Theorem 3 with Z in place of S. The instance numbers are the a_j in binary, so the reduction is not unary-robust. The dynamic program of Theorem 3, run with $g_j = p_j = a_j$ in place of $f_i = b_i$ after the $O(1)$ test $b_1 \geq D$, whose failure means infeasibility, places the group in P under a unary encoding [14]. □

Together the two weak slices delimit the quadrant of the model in which the covering layer is silent, and the next observation makes that delimitation explicit.

Observation 2. *With $|J| = |L| = 1$ and $|I|, |K|$ free the covering layer is entirely inactive. Through a single depot the factories cannot tell customers apart, and a single product offers no dimension to discriminate either, so the group of instances is at once weakly NP-hard, by Theorem 3, and pseudo-polynomially solvable. With one product and one depot every unit of every q_k crosses the same depot to its customer, so the second-stage cost of any trimmed routing is the fixed constant $\sum_k d_{1k} q_k$, independent of the design. Feasibility further requires $p_1 \geq \sum_k q_k$, a test supplied by (F2) whose failure means the instance is infeasible, and when the total demand is positive the forced opening $z_1 = 1$ adds the constant charge g_1 to every solution. What remains is a factory-side knapsack-cover with opening charges f_i and per-unit costs c_{i1} against demand $\sum_k q_k$, solvable in pseudo-polynomial time by the factory half of the dynamic program behind Theorem 7, of which the subset-sum of Theorem 3 is the special case $f_i = b_i$, $c \equiv 0$. This is the only quadrant we identify with such a mixed profile, and it witnesses the two-layer picture directly. The covering layer needs a facility layer that discriminates a dimension carrying demand, the factories under tight capacities included, and here none does, so the weak numeric hardness alone remains, vanishing under a unary encoding.*

4.3. The parameter k

The genuine parameter of the study is the number of opened facilities $k = \sum_i y_i + \sum_j z_j$ of Definition 2, and the covering reductions already say what to expect of it. Set Cover parameterized by the cover size is the canonical W[2]-complete problem, and the construction of Theorem 1 maps that parameter almost onto k. The only subtlety is that the forced free factory consumes one unit of cardinality without consuming budget.

Theorem 4. MP-TSCFLP(B, k) *parameterized by k is W[2]-hard, even on the subclass of Theorem 1. The same reduction shows that the budget-only version* MP-TSCFLP(B) *parameterized by B is W[2]-hard, and that the variant whose cardinality bound counts only depots, requiring $\sum_j z_j \leq k_z$, is W[2]-hard with parameter k_z, the reduction setting $k_z = t$.*

The budget parameterization is W[2]-hard here because the slice carries unit depot charges, so the budget counts the open depots, and parameterization by a budget value is encoding-sensitive in general.

Proof. We reduce from Set Cover parameterized by t, which is W[2]-complete [15]. The map is that of Theorem 1, computed in polynomial time, with $k = t + 1$ a computable function of the source parameter, so it is an FPT reduction [15, 16]. For the equivalence, a cover C of size at most t yields a feasible solution of cost $|C| \leq t$ and cardinality $1 + |C| \leq t + 1$, the factory, forced open by the positive demand, contributing the extra unit while carrying no charge. Conversely any witness of the target side satisfies cost at most t and therefore, by the converse direction of Theorem 1, opens a cover of size at most t. The cardinality bound only restricts the witnesses and so cannot break this direction.

Slack. In the image, cost at most B forces $\sum_j z_j \leq \sum_j g_j z_j \leq$ cost $\leq B$ because $g \equiv 1$ and the other charges are nonnegative, while $\sum_i y_i = 1$ because (F1) with positive demand forces the single factory open. Every feasible solution of cost at most B therefore has cardinality at most $B + 1$, so the cardinality bound with $k = B + 1$ is vacuous. For the budget-only version the same instance with parameter $B = t$ is already a reduction, for its equivalence is verbatim that of Theorem 1, whose two directions used only the budget. For the depot-only variant set $k_z = t$. A cover C yields a solution with $\sum_j z_j = |C| \leq t$, and the bound k_z only restricts the witnesses, so the converse direction of Theorem 1 is unaffected. □

The exponent of the trivial enumeration is linear in k, and under the Exponential Time Hypothesis [26] that linear exponent cannot be improved, which pairs with the XP membership of Observation 1 to show that XP is the right home for this parameter.

Theorem 5. *Assuming the ETH, no algorithm decides* MP-TSCFLP(B, k) *in time $f(k) \cdot N^{o(k)}$, where N is the instance size and f is computable, even on the subclass of Theorem 1. Since N and $n = |I| + |J|$ are polynomially related on the image, the $n^{k+O(1)}$ enumeration of Observation 1 is optimal up to the constant in the exponent.*

Proof. We compose two maps and track the parameter through the composition. The first is the closed-neighbourhood embedding of Dominating Set into Set Cover, whose universe is $V(G)$ and whose sets are the closed neighbourhoods $N[v]$ for $v \in V(G)$, so that dominating sets of size t correspond exactly to

covers of size t. A one-vertex instance of Dominating Set is decidable in constant time, so we assume $n_G \geq 2$, where n_G is the number of vertices. Composing the embedding with the map of Theorem 4 gives Dominating Set$(t) \to$ MP-TSCFLP$(B{=}t, k{=}t{+}1)$ with an image of size $N \leq n_G^c$ for an absolute constant c. The size bound follows because the embedded Set Cover instance has $n_U = m = n_G$, so the image of the map of Theorem 1 consists of an $n_G \times n_G$ second-stage cost table together with numbers bounded by $n_G(n_G + 1)$. Suppose an algorithm decided the image in time $f(k) \cdot N^{o(k)}$. Substituting $k = t + 1$ and $N \leq n_G^c$ with c constant gives $N^{o(k)} = n_G^{c \cdot o(t+1)} = n_G^{o(t)}$, since a constant multiple of a sublinear function is sublinear and $o(t + 1) = o(t)$, so the composition would decide Dominating Set in time $f'(t) \cdot n_G^{o(t)}$ for a computable f', the polynomial cost of the two reduction steps being absorbed into the product $f'(t) \cdot n_G^{o(t)}$. This contradicts the ETH lower bound of Chen et al. [17], also in Cygan et al. [16, Chapter 14]. Finally, on the image the instance size N and the facility count $n = |I| + |J|$ satisfy $n \leq N \leq n^{c'}$ for a constant c', so an algorithm running in time $f(k) \cdot n^{o(k)}$ would run in time $f(k) \cdot N^{o(k)}$ and is excluded by the bound just proved, which makes the $n^{k+O(1)}$ enumeration of Observation 1 optimal up to the constant in the exponent. □

The quantitative form of the exclusion requires one clarification. Running times of the shape $f(k) \cdot N^{O(k)}$ are not, and could not be, excluded, because the enumeration of Observation 1 achieves exactly such a bound. The excluded class is fixed before the composition. For every computable f and every function h with $h(t)/t \to 0$ no algorithm decides the target in time $f(t)\, N^{h(t)}$, and the composition maps an algorithm with exponent $h(k) = o(k)$ to one with exponent $h'(t) \leq c\, h(t + 1) + c$, which is again $o(t)$. The pair of results thus brackets the parameter from both sides, the exponent linear in k being attainable and, under the ETH, optimal.

Remark 3. Membership in W[2] is left open. A witness is the set of at most k opened facilities, which has the shape W[2] expects, but the acceptance test compares binary sums and, worse, minimizes an internal routing flow by Proposition 1, and neither is known to translate into bounded weft. In particular the natural verifier must evaluate a minimum-cost flow, a computation that is P-complete in general and is not known to be expressible by bounded-weft circuits. What can be stated without risk is that MP-TSCFLP(B, k) lies in W[P], by guessing the $O(k \log n)$ bits of the opening pattern and verifying in polynomial time, so the parameter sits between W[2]-hardness and W[P] ∩ XP.

4.4. The numeric layer

The covering reductions accounted for three discrimination pairs, each pairing a facility layer with a demand dimension, depots against customers, factories against products, and depots against products. The remaining question is the group of instances where both demand dimensions are trivial, $|K| = |L| = 1$, and only the general transport matrix couples the two facility layers. A computational search over small instances and candidate reduction rules, recorded with the accompanying code, surfaced no tractable structure for this group, and the explanation is that the group carries a fourth discrimination pair. Tight capacities turn the factories themselves into carriers of demand, and the first-stage matrix then distinguishes factories that no open depot covers.

Theorem 6. MP-TSCFLP(B) *is strongly NP-hard even when, simultaneously,* $|K| = |L| = 1$*, the charges satisfy* $f \equiv 0$ *and* $g \equiv 1$*, first-stage costs lie in* $\{0, 1\}$ *while second-stage costs vanish, capacities are uniform on each side, and every number is bounded by* $|I|\,(|J| + 1)$*. Consequently this group of instances admits no pseudo-polynomial algorithm unless* $\mathsf{P} = \mathsf{NP}$*.*

The discriminating pair is now the two facility layers, mediated by tight capacities that force every feasible solution to open and saturate all factories, and both demand dimensions stay fixed at one.

Proof. The source problem is once more Set Cover [13], with $1 \leq t \leq m$ and $Q = m + 1$.

Construction. Put $K = L = \{1\}$ with demand $D = n_U Q$, factories $I = U$ with $f_i = 0$ and $b_i = Q$, depots $J = \{1, \ldots, m\}$ with $g_j = 1$ and $p_j = n_U Q$, first-stage cost $c_{ij} = 0$ when $i \in S_j$ and 1 otherwise, second-stage cost $d \equiv 0$, and budget $B = t$. As before the construction takes $O(n_U m)$ time.

Correctness. By Proposition 2 a design (Y, Z) is feasible exactly when (F1) $\sum_{i \in Y} b_i = Q|Y| \geq D = Qn_U$, that is $Y = I$, and (F2) $\sum_{j \in Z} p_j \geq D$, that is $Z \neq \emptyset$, the second equivalence using $D > 0$, guaranteed by the nondegeneracy convention. Fix a feasible solution, write $X_j = \sum_i x_{ij}$ for the flow into depot

j, and write $w_j = w_{j11}$ for the flow that depot j forwards to the customer. Summing (2) over the single customer gives $\sum_j w_j \geq D$, summing (3) over j gives $\sum_j X_j \geq \sum_j w_j$, and summing (4) over i gives $\sum_j X_j \leq \sum_i b_i y_i = Q n_U = D$. The chain $D \leq \sum_j w_j \leq \sum_j X_j \leq D$ collapses to equalities, so $\sum_j w_j = \sum_j X_j = D$ and, since each factory ships $\sum_j x_{ij} \leq b_i = Q$ while the total over the n_U open factories is $Q n_U$, each factory ships exactly Q. Since $\sum_j (X_j - w_j) = 0$ with every $X_j - w_j \geq 0$ by (3), $X_j = w_j$ for all j. A closed depot has $w_j = 0$ by (5), hence $X_j = 0$, so no flow enters a closed depot and the cost sum below runs only over $j \in Z$.

The cost is then $|Z| + \sum_i \sum_{j \in Z,\, i \notin S_j} x_{ij}$, at least $|Z| + Q\, V_Z$ with V_Z the number of factories no open depot covers, since a factory i that no open depot covers ships its full load Q on cost-one arcs. Equality holds by routing each covered factory's load through a covering depot and each uncovered factory's load through any open depot, at cost Q for the latter, then setting $w_j = X_j$ at no charge since $d \equiv 0$, the depot capacities $p_j = n_U Q$ accommodating any inflow. A cover C opens $Y = I$, forced by (F1) and free since $f \equiv 0$, together with $Z = C$ at cost $|C| \leq t$. Conversely the amplifier argument of Theorem 1 again shows that a solution of cost at most t leaves no factory uncovered and opens at most t depots. The largest number is $n_U(m+1)$, polynomial in the input, so the subclass stays NP-hard when the data are written in unary [14]. □

What survives the reduction is exactly the structure of the transport matrix, and killing that structure restores tractability. When the matrix is separable the two facility layers decouple and each becomes a pseudo-polynomial knapsack-cover, so the separable matrices form a pseudo-polynomially solvable subclass, while Theorem 6 shows that strong hardness persists outside the separable class. The classification of the territory between the two, including structured non-separable matrices such as low-rank or Monge matrices, remains open.

Theorem 7 (separable first stage)**.** *On the group $|K| = |L| = 1$, if the first-stage matrix is separable, that is $c_{ij} = \gamma_i + \delta_j$ for integers γ, δ with the costs nonnegative, then the problem is solvable in pseudo-polynomial time $O\big(|I||J| + (|I| + |J|) \cdot D\big)$. Separability is decidable in $O(|I||J|)$ time.*

The proof, given in B.1, reduces the routing cost to a function of the two load marginals, realizes any consistent pair of loads by a northwest-corner plan, and solves each facility layer by a dynamic program over integer loads, integrality of the split optima coming from the vertex structure of the load polytopes. Separability is the quadrangle condition $c_{ij} + c_{11} = c_{i1} + c_{1j}$, checked in one pass over the matrix.

The separable subclass rounds out the numeric layer, and its parameterized reading closes the account of the group with both demand dimensions trivial.

Corollary 6. MP-TSCFLP(B) *is strongly para-NP-hard with respect to the pair $(|K|, |L|)$, and unless $\mathsf{P} = \mathsf{NP}$ no XP algorithm exists for it, even under a unary encoding. Under a unary encoding the witness slices of the weak groups become polynomial, by Theorem 3, Corollary 5, Observation 2 and Theorem 7, while the four covering reductions of Theorems 1, 2, 6 and Corollary 3 are unary-robust, so the covering layer survives unchanged.*

Proof. The claims assemble the slices already proved. The slice $(|K|, |L|) = (1, 1)$ is strongly NP-hard by Theorem 6, and Corollary 2 again supplies the parameterized reading, now robust under a unary encoding because the instance numbers of Theorem 6 are polynomial. For the remaining claims of the statement, the four positive results match their slices as follows. The dynamic program of Theorem 3 covers the group $|J| = |L| = |K| = 1$, its layer-exchanged form in Corollary 5 covers the mirror group $|I| = |L| = |K| = 1$, the knapsack-cover of Observation 2 covers the quadrant $|J| = |L| = 1$, and Theorem 7 covers the separable part of the group $|K| = |L| = 1$. Each of the four runs in pseudo-polynomial time and hence in polynomial time once the data are written in unary. The images of the four covering reductions, in Theorems 1, 2, 6 and Corollary 3, carry numbers bounded by $|K|(|J| + 1)$, $|I| + 1$, $|I|(|J| + 1)$ and $|J| + 1$ respectively, so all four remain valid polynomial-time reductions when their images are encoded in unary. □

Taken together, the reductions of this section show that every way of bounding a subset $P \subseteq \{|I|, |J|, |K|, |L|\}$ that fails to retain both facility layers, that is with $\{|I|, |J|\} \not\subseteq P$, leaves an NP-hard slice, so unless $\mathsf{P} = \mathsf{NP}$ a tractable parameterization of this kind must retain $|I|$ and $|J|$ together. Section 5

takes up exactly that surviving combination, where the enumeration of Observation 1 sharpens into fixed-parameter tractability. The exponent of the $n^{k+O(1)}$ enumeration itself cannot be lowered under the ETH, by Theorem 5.

5. Parameterized algorithms and kernels

The hardness map of Section 4 is dense, yet it leaves positive windows, and this section fills them. We give an XP algorithm in the number of opened facilities, the branch-and-bound refinement we implement, the fixed-parameter algorithm that pins down the size-parameter dichotomy, the routing certificates that bridge the enumeration to the exact formulation, and the two sides of the kernelization question. A short remark on width parameters closes the account, and Table 3 then collects the whole landscape.

5.1. An XP algorithm in the number of opened facilities

Once a design is fixed the cost is determined by the routing oracle of Proposition 1, so the only combinatorial freedom left is which facilities to open. Bounding that freedom by k leaves at most $e\,n^k$ candidate supports, and each is priced by one oracle call. The next theorem promotes the brute-force membership of Observation 1 to a running time whose exponent is linear in k, and the branch-and-bound that follows stays within the same $n^{k+O(1)}$ regime while pruning aggressively in practice.

The *support* of a design is the set of facilities it opens. We identify each set $S \subseteq I \cup J$ with the design that opens exactly the facilities of S, and under this identification we write $v(S)$ for its routing value, $f(S) = \sum_{i \in S \cap I} f_i$ and $g(S) = \sum_{j \in S \cap J} g_j$ for its two fixed charges, and $\mathrm{cost}(S) = f(S) + g(S) + v(S)$ for its total cost. On an infeasible support the convention $v(S) = +\infty$ of Section 3 gives $\mathrm{cost}(S) = +\infty$.

Theorem 8. *Write T_{orac} for the time of one routing-oracle call and normalize $k \le n$. Then* MP-TSCFLP(B, k) *is decidable, and the least cost* OPT_k *over feasible designs opening at most k facilities is computable, in time $O\big(n^k \cdot (T_{\mathrm{orac}} + n|L|)\big)$ for $k \ge 1$, and in time $O(T_{\mathrm{orac}} + n|L|)$ for $k = 0$. In particular the problem lies in XP in the parameter k.*

Proof. A feasible design opening at most k facilities has a support $S \subseteq I \cup J$ with $|S| \le k$, its cost is the fixed charge of S plus the routing value $v(S)$, and by Proposition 1 the routing value is attained, so each feasible support realizes exactly its cost and OPT_k is the minimum of $\mathrm{cost}(S)$ over feasible supports of size at most k. When no such support is feasible we put $\mathrm{OPT}_k = +\infty$, and the correct answer is then NO. Enumerate every such S, test feasibility by Proposition 2 in $O(n|L|)$ time, and price the feasible ones by one oracle call. The number of supports is $\sum_{t=0}^{k} \binom{n}{t} \le \sum_{t=0}^{k} \frac{n^t}{t!} \le n^k \sum_{t \ge 0} \frac{1}{t!} = e\,n^k$ for $k \le n$ and $k \ge 1$. The middle inequality uses $n^t \le n^k$ for $t \le k$, valid since $n \ge 1$, and extending the sum adds nonnegative terms. Each support costs one $O(n|L|)$ feasibility test plus at most one oracle call, and T_{orac} is left as a stated quantity rather than bounded below, since an implementation may share preprocessed data across calls, giving $O\big(n^k(T_{\mathrm{orac}} + n|L|)\big)$ in total. For $k = 0$ only the empty design is priced. That design is feasible exactly when every $D_l = 0$, since for it the left-hand sums of (F1) and (F2) are empty, so one feasibility test and at most one oracle call suffice, within $O(T_{\mathrm{orac}} + n|L|)$. Comparing the minimum against B decides MP-TSCFLP(B, k). □

5.2. A practical branch-and-bound refinement

Theorem 8 settles the classification, and the rest is implementation. The enumeration it counts is wasteful whenever a partial design already cannot be completed within the budget, and the branch-and-bound of Algorithm 1, the code the computational study runs, closes such subtrees early. Its principal device is a covering count. Because both stages are complete, feasibility of a product needs only enough open capacity on each side, and a sorted prefix determines the fewest facilities that supply a demand from a nonnegative capacity vector.

Lemma 1 (covering count)**.** *Let $a_1, \dots, a_m \ge 0$ and $D \ge 0$, and let $a_{(1)} \ge \cdots \ge a_{(m)}$ be the decreasing order. Some S with $|S| \le s$ has $\sum_{i \in S} a_i \ge D$ if and only if $\sum_{r \le s} a_{(r)} \ge D$. When $\sum_{r \le m} a_{(r)} < D$ no such s exists and we set $k^*(a; D) = +\infty$. Otherwise the least such s, written $k^*(a; D)$, is computable by one sort and one prefix scan in $O(m \log m)$ time, the same scan detecting the infinite case.*

Proof. Both directions follow from majorization by the sorted prefix. If $\sum_{r\le s} a_{(r)} \ge D$ the set of the s largest entries witnesses the forward direction. Conversely, order any S decreasingly. Its rth element is at most $a_{(r)}$, because the r largest elements of S are r distinct entries of a, each at least that element, so a has at least r entries at least as large. Summing gives $\sum_{i\in S} a_i \le \sum_{r\le|S|} a_{(r)}$. Hence if some S with $|S| \le s$ has $\sum_{i\in S} a_i \ge D$ then $D \le \sum_{i\in S} a_i \le \sum_{r\le|S|} a_{(r)} \le \sum_{r\le s} a_{(r)}$, the last step by $a \ge 0$. The prefix sums are nondecreasing in s, so the least s with $\sum_{r\le s} a_{(r)} \ge D$ is found by one sort and one scan. □

Algorithm 1 searches over partial designs, and we name the quantities its nodes carry before giving the listing. A node is a pair (O, C) in which O is the set of facilities forced open and C the set forced closed, so that $F = (I \cup J) \setminus (O \cup C)$ collects the free facilities and $r = k - |O|$ is the remaining cardinality. For a product l let

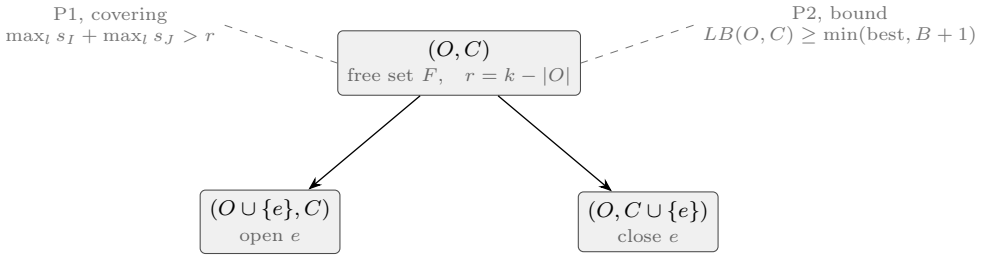
$$s_I(l; O, C) = k^*\Big((b_{il})_{i\in F\cap I};\, \max\big(0,\, D_l - \textstyle\sum_{i\in O\cap I} b_{il}\big)\Big),$$

the covering count of Lemma 1 over the free factory capacities against the demand left after crediting the already-open factories, and let

$$s_J(l; O, C) = k^*\Big((p_{jl})_{j\in F\cap J};\, \max\big(0,\, D_l - \textstyle\sum_{j\in O\cap J} p_{jl}\big)\Big)$$

be the analogous count for (F2) over the free depot capacities. The quantities of line 2 are the root specializations $k_I^*(l) = k^*\big((b_{il})_{i\in I}; D_l\big) = s_I(l; \emptyset, \emptyset)$ and $k_J^*(l) = k^*\big((p_{jl})_{j\in J}; D_l\big) = s_J(l; \emptyset, \emptyset)$. Let $v^\uparrow = v(O \cup F)$ be the routing value of the all-open completion, with the convention $v = +\infty$ of Section 3, and let $LB(O, C) = f(O) + g(O) + v^\uparrow$. Finally, a facility of O is *unused* by a flow when the flow carries zero on every arc incident to that facility in every product's network. Three prunes act on these quantities, and after the listing we show each is sound and then that the search returns OPT_k. Figure 4 places the three prunes on a single branching step.

Figure 4 One branching step of Algorithm 1. A node carries the forced-open set O, the forced-closed set C, the free set F and the remaining cardinality r, and branching on a free facility e produces the two children along the solid arrows. The dashed leaders attach P1 and P2 to the popped node they test, while P3 fires only at a leaf, so the three prunes act exactly on the named quantities (Lemma 1, Proposition 6).



P3, dominance, only at a leaf, where $F = \varnothing$ and the oracle flow leaves some facility of O unused

Algorithm 1 Branch-and-bound for MP-TSCFLP(B, k) (Theorem 8).

1: $k \leftarrow \min(k, n)$ ▷ normalization, no design opens more
2: **for each** product l sort $(b_{il})_i$ and $(p_{jl})_j$ and compute $k_I^*(l), k_J^*(l)$ ▷ Lemma 1
3: **if** $\max_l k_I^*(l) + \max_l k_J^*(l) > k$ **then return** NO with $\mathrm{OPT}_k = +\infty$ ▷ a priori bound k^*
4: best $\leftarrow +\infty$; push the root $(O, C) = (\emptyset, \emptyset)$
5: **while** the stack is nonempty **do**
6: pop (O, C); let F be the free facilities and $r \leftarrow k - |O|$; **if** $r < 0$ **then** continue
7: **if** $\max_l s_I(l; O, C) + \max_l s_J(l; O, C) > r$ **then** continue ▷ prune P1, Lemma 1
8: evaluate the all-open completion by the oracle to get $v^\uparrow$ and $LB(O, C)$
9: **if** $v^\uparrow = +\infty$ **or** $LB(O, C) \geq \min(\text{best}, B + 1)$ **then** continue ▷ prune P2
10: **if** $F = \emptyset$ **then** ▷ leaf, the design is $S = O$
11: **if** the oracle solution returned at the leaf leaves some facility of O unused **then** continue ▷ prune P3
12: best $\leftarrow \min(\text{best}, LB(O, C))$
13: **else** pick $e \in F$ of least index and push $(O \cup \{e\}, C)$ and $(O, C \cup \{e\})$
14: **if** best $< +\infty$ **and** best $\leq B$ **then return** YES **else return** NO, either way reporting best ▷ run with $B = \infty$ to read OPT_k off either branch

P1, covering. Every completion of (O, C) opens O together with some free facilities. For such a completion S and a product l, (F1) reads $\sum_{i \in S \cap I} b_{il} \geq D_l$, which states that the free factories of S supply at least the residual demand $\max(0, D_l - \sum_{i \in O \cap I} b_{il})$, so Lemma 1, applied to $(b_{il})_{i \in F \cap I}$ against that residual demand, shows that S adds at least $s_I(l; O, C)$ free factories. One open set must satisfy (F1) for all products at once, so it adds at least $\max_l s_I(l; O, C)$ free factories, and likewise at least $\max_l s_J(l; O, C)$ free depots. Factories and depots are disjoint, so the two increments add, and when their sum exceeds r no completion has cardinality at most k, so P1 discards (O, C) soundly. An infinite count falls under the same reading, since $s_I(l; O, C) = +\infty$ means that even opening every free factory leaves (F1) violated for l, so no completion is feasible and the prune fires for every finite r.

P2, bound. Any completion opens a set S with $O \subseteq S \subseteq O \cup F$. Its fixed charge is at least $f(O) + g(O)$ by nonnegativity of the charges, and its routing value is at least $v(O \cup F) = v^\uparrow$ because $S \leq O \cup F$ componentwise and v is monotone (Proposition 3). Hence $LB(O, C)$ is a lower bound on the cost of *every* completion, even one opening more than k facilities, which is all the bound needs. P2 discards (O, C) when $LB(O, C) \geq \min(\text{best}, B + 1)$, and either threshold justifies the discard. If $LB(O, C) \geq \text{best}$ then no completion improves the incumbent. If $LB(O, C) \geq B + 1$ then no completion meets the budget, because the data are nonnegative integers and the routing values are integers by Proposition 1, so every cost is an integer and a cost at most B is a cost strictly below $B + 1$. When $v^\uparrow = +\infty$ the same monotonicity gives $v(S) \geq v(O \cup F) = +\infty$ for every completion S, so every completion is infeasible under the convention of Section 3 and discarding is again sound. At a leaf, where $F = \emptyset$, one has $LB(O, C) = \mathrm{cost}(O)$.

P3, dominance. When the oracle solution returned at the leaf leaves an open facility of O unused, closing that facility gives a design of strictly smaller cardinality. The oracle solution returned at the leaf carries nothing on any arc of the closed facility, so it remains a feasible routing of the smaller design, whence the routing value does not increase, and the fixed charge drops by the nonnegative charge of that facility. The smaller design is therefore feasible at no greater cost, so O is dominated and P3 discards it.

With the three prunes justified, the remaining question is whether the surviving search still reaches an optimum.

Proposition 6 (correctness)**.** *Algorithm 1 decides* MP-TSCFLP(B, k)*, and when run with $B = +\infty$ it returns* best $= \mathrm{OPT}_k$*.*

The full argument appears in B.2. Its three steps justify the early return of line 3 through Lemma 1, bound best below by OPT_k because only feasible designs within the cardinality budget ever reach a leaf, and bound it above by tracking a minimum-cardinality optimum down the tree, checking that no prune ever discards the node carrying it.

The prunes only discard nodes, and the tree they act on is itself small. Branching always selects the free facility of least index, so at every node the decided set $O \cup C$ is a prefix of the facility order and the node is determined by the prefix length m together with $O \subseteq \{1, \dots, m\}$. Children are pushed only from nodes with $|O| \le k$, so every node ever created has $|O| \le k+1$ and the tree holds at most $\sum_{m=0}^{n} \sum_{t=0}^{k+1} \binom{m}{t} = O(n^{k+2})$ nodes, since $\sum_{t=0}^{k+1} \binom{m}{t} \le \sum_{t=0}^{k+1} n^t \le 2n^{k+1}$ for $n \ge 2$ by the geometric series and the cases $n \le 1$ are trivial. Each node performs the sorts and prefix scans of P1 and at most one oracle call, so the whole search runs in $O\big(n^{k+2}(T_{\mathrm{orac}} + n|L| \log n)\big)$ time, a polynomial factor above the enumeration bound of Theorem 8 and within the same $n^{k+O(1)}$ regime, which is all the classification uses. On the seeded validation suite recorded in the accompanying code, the reference implementation visited roughly ninety per cent fewer nodes than the plain enumeration and returned the brute-force optimum on every instance. The linear exponent is essentially the best one can hope for, since under the Exponential Time Hypothesis no algorithm decides the problem in time $f(k) \cdot N^{o(k)}$ (Theorem 5), so the enumeration of supports is tight up to the constant in the exponent.

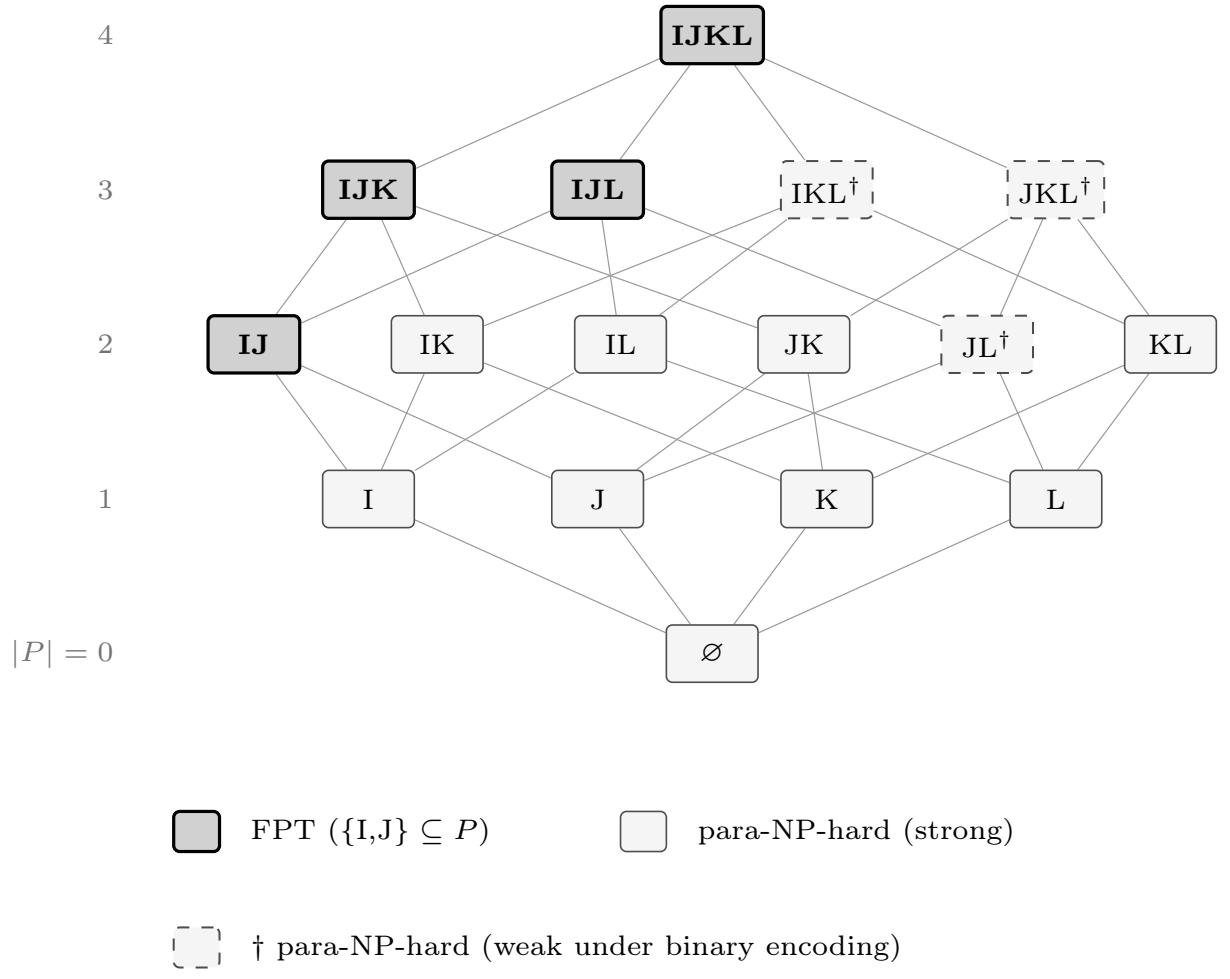
One relation among the prunes has consequences that Section 6.2 meets again on the machine, so we record it here.

Observation 3 (the covering prune in plain optimization)**.** *Run with $k = n$, prune P1 discards no node beyond the infeasibility branch of P2, since at any node $r = n - |O| = |F| + |C| \ge |F|$. If the all-open completion $O \cup F$ is feasible, then for every product the free facilities of each side supply that side's residual demand, so Lemma 1 gives $\max_l s_I(l; O, C) \le |F \cap I|$ and $\max_l s_J(l; O, C) \le |F \cap J|$, the two maxima add to at most $|F| \le r$, and P1 does not fire. When P1 fires, therefore, the all-open completion is infeasible, so $v^{\uparrow} = +\infty$ and the infeasibility branch of P2 discards the same node at the price of one oracle call. Under a binding cardinality $k < n$ the subsumption fails, since a node may admit a feasible all-open completion while no completion within the remaining budget r exists, a situation P1 can detect and the infeasibility branch of P2 cannot.*

5.3. Fixed-parameter tractability and the size-parameter dichotomy

The parameter k counts openings, whereas the four dimensions $|I|, |J|, |K|, |L|$ measure the instance itself, and only the two facility sides carry tractability. Enumerating designs rather than supports gives a fixed-parameter algorithm in $|I| + |J|$, and coupling it with the hardness slices of Section 4 pins the exact border that Figure 5 maps over the sixteen subsets.

Figure 5 The size-parameter dichotomy of Theorem 10 over the sixteen subsets $P \subseteq \{|I|, |J|, |K|, |L|\}$, arranged by cardinality as a Hasse diagram whose edges are the cover relations. The problem is fixed-parameter tractable exactly on the four upward-closed sets that contain both facility layers I and J, shaded and shown in bold (Theorem 9), and para-NP-hard on the remaining twelve. Hardness is strong except at the three sets marked with a dagger, whose witnessing slices are only weakly NP-hard under a binary encoding (Theorem 3, Corollary 5).



Theorem 9 (fixed-parameter tractability)**.** MP-TSCFLP(B)*, its optimization form and its cardinality version are solvable in time* $2^{|I|+|J|} \cdot \mathrm{poly}(N)$*, where* N *is the input size. Hence the problem is* FPT *in the combined parameter* $(|I|, |J|)$*, and a fortiori under any parameter set containing both.*

Proof. Enumerate all $2^{|I|+|J|}$ designs, test each by Proposition 2, price the feasible ones by Proposition 1, filter by $\sum y + \sum z \le k$ when the cardinality bound is present, and return the least cost, compared against B for the decision versions, reporting infeasibility through the value $+\infty$ when no design passes the test. Correctness is immediate from the attainment in Proposition 1, each step per design runs in time polynomial in N, and the function $2^{|I|+|J|}$ reads only the two facility counts, which is the definition of FPT in the combined parameter. The a fortiori clause follows because $2^{|I|+|J|}$ is also a computable function of any parameter set that contains both facility counts. □

The synthesis theorem states the border cleanly. For a parameter set P drawn from the four dimensions the problem is fixed-parameter tractable exactly when P contains both facility sides, and otherwise it is hard already at a fixed slice.

Theorem 10 (dichotomy)**.** *Let $P \subseteq \{|I|, |J|, |K|, |L|\}$ parameterize* MP-TSCFLP(B)*. If $\{|I|, |J|\} \subseteq P$ the problem is FPT, unconditionally. If $\{|I|, |J|\} \not\subseteq P$ it is para-NP-hard, unconditionally, because some assignment of fixed values to the parameters of P gives an NP-hard slice, and then no XP algorithm exists unless $\mathsf{P} = \mathsf{NP}$. Assuming $\mathsf{P} \neq \mathsf{NP}$, therefore, the problem is FPT if and only if $\{|I|, |J|\} \subseteq P$. The same statement transfers verbatim to* MP-TSCFLP(B, k) *by appending $k := n$.*

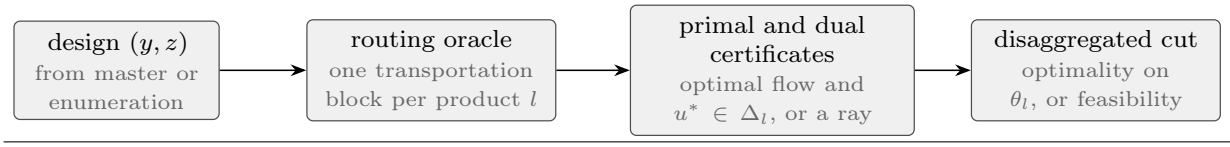
Proof. The positive half is Theorem 9. For the negative half it suffices to exhibit, for each of the twelve subsets P with $\{|I|, |J|\} \not\subseteq P$, an NP-hard slice that fixes every parameter of P to the constant one, since a problem NP-hard at a fixed parameter value is para-NP-hard and an XP algorithm would decide that slice in polynomial time. Six slices from Section 4 cover all twelve. The slice of Theorem 1 fixes $|I| = |L| = 1$ and witnesses $\emptyset$, $\{|I|\}$, $\{|L|\}$ and $\{|I|, |L|\}$. The slice of Theorem 2 fixes $|J| = |K| = 1$ and witnesses $\{|J|\}$, $\{|K|\}$ and $\{|J|, |K|\}$, while its mirror, Corollary 3, fixes $|I| = |K| = 1$ and witnesses $\{|I|, |K|\}$. The slice of Theorem 6 fixes $|K| = |L| = 1$ and witnesses $\{|K|, |L|\}$. The weak slice of Theorem 3 fixes $|J| = |K| = |L| = 1$ and witnesses $\{|J|, |L|\}$ and the triple $\{|J|, |K|, |L|\}$, and its depot mirror, Corollary 5, fixes $|I| = |K| = |L| = 1$ and witnesses the remaining triple $\{|I|, |K|, |L|\}$. Each of the twelve subsets with $\{|I|, |J|\} \not\subseteq P$ is contained in one of the six fixed sets $\{|I|, |L|\}$, $\{|J|, |K|\}$, $\{|I|, |K|\}$, $\{|K|, |L|\}$, $\{|J|, |K|, |L|\}$, $\{|I|, |K|, |L|\}$, and fixing a superset of P to constants fixes P, so the case analysis is complete. Appending $k := n$ preserves the answer, because no design opens more than n facilities, so the appended bound is vacuous and the two problems coincide instance by instance, and it leaves the fixed values of P untouched. □

Two qualifications attach to the negative half. Para-NP-hardness asks only for NP-hardness of a slice under the standard binary encoding, and weak hardness already is NP-hardness, so the weakly hard slices count towards the dichotomy exactly as the strongly hard ones do. The encoding matters only when we ask whether the border survives unary numbers. There the positive half is untouched, and the negative half survives for the nine parameter sets whose witnessing slices are strongly NP-hard, while for $\{|J|, |L|\}$ and the two triples $\{|I|, |K|, |L|\}$ and $\{|J|, |K|, |L|\}$ the witnessing slices turn polynomial under unary data, through the pseudo-polynomial programs of Theorem 3 and Corollary 5, and no other hard slice is known, so their unary status stays open.

5.4. Certificates and a bridge to the exact formulation

The dichotomy settles what the parameters buy, and this subsection turns the same oracle into certificates a mixed-integer solver can consume. Each oracle call solves one transportation problem per product, and its linear-programming dual both certifies the routing value and supplies a cut that a Benders master can reuse. The fact that carries the subsection is that the dual polyhedron depends only on the data, never on the design, so a single dual point yields an inequality valid for every design whose block is feasible. Figure 6 traces the pipeline the proposition formalizes.

Figure 6 From a design to a cut, the pipeline of Proposition 7. Each oracle call of Proposition 1 prices the design one product at a time, returns an optimal flow together with a dual point $u^* \in \Delta_l$ certifying the block value, and that dual point yields the disaggregated Benders optimality cut, valid for every design with a feasible block because Δ_l never depends on (y, z), while an infeasible block yields a recession ray and a feasibility cut instead.



Proposition 7 (routing certificate and Benders cuts)**.** *Fix a product l and let Δ_l be the dual polyhedron of its transportation block, which is independent of (y, z). If the block is feasible then some $u^* \in \Delta_l$ attains the dual value $\mu_l(y, z)$, and the pair of an optimal flow and u^* certifies $\mu_l(y, z)$ in time $O(|I||J| + |J||K|)$. Further, writing θ_l for a variable through which a master problem represents the block value, for every*

$u = (\alpha, \beta, \sigma, \pi) \in \Delta_l$ *the inequality*

$$\theta_l \;\geq\; \sum_k q_{kl}\,\alpha_k - \sum_i b_{il}\,\sigma_i\,y_i - \sum_j p_{jl}\,\pi_j\,z_j$$

is valid for every design whose block l is feasible, and it holds with equality at the design that generated u^*. Two explicit recession rays of Δ_l certify infeasibility, and their objectives are positive exactly when (F1), respectively (F2), of Proposition 2 fails.

The proof, given in B.3, writes the block dual explicitly. No dual constraint mentions the design, which yields the independence of Δ_l, weak duality yields the displayed inequality, strong duality supplies the attaining point u^*, and the two recession rays are exhibited coordinate by coordinate and matched to the failures of (F1) and (F2).

These are precisely the per-product optimality cuts of a disaggregated Benders decomposition, and the recession rays are its feasibility cuts. The certificates the enumeration consumes are the objects the Benders decomposition emits, so each oracle call in Algorithm 1 can return, beside its value, the very cut that a branch-and-Benders-cut master would add at that design. The proposition certifies the routing value and the validity of the cuts, not the global optimum, which still needs the master's own bounds.

5.5. Kernelization

Preprocessing is the last algorithmic question, and it divides cleanly. Customers with identical cost columns aggregate exactly, products do not, a bound on costs and total demands yields a polynomial kernel outright, a bound on costs and budget still yields a polynomial compression through the weight-reduction technique of Frank and Tardos [18], and yet no polynomial kernel exists in the winning parameter unless the polynomial hierarchy collapses. The first reduction is safe and cheap.

Proposition 8 (customer aggregation)**.** *If customers $k \neq k'$ have identical cost columns, that is $d_{jkl} = d_{jk'l}$ for all j, l, then merging k' into k and adding their demands preserves feasibility, routing value and cost of every design, hence the optimal value, the set of optimal designs and the answers of both decision versions. Feasible complete solutions map in both directions under the explicit merge and split maps of the proof, which never raise the cost and preserve the optimal routing value of every design, and consequently one may assume $|K| \leq \tau$, where τ is the number of distinct second-stage cost columns, computable in polynomial time.*

We prove this in B.4 by exhibiting the two maps directly. The merge adds the flow rows of the two customers and preserves every constraint and every cost term. The split trims any surplus of the merged row and expands it by the northwest-corner procedure, which preserves feasibility at no cost increase, so the optimal routing values agree design by design, and a companion reduction bounds the capacities without touching any design.

Remark 4 (aggregation up to additive shifts). The exact-equality hypothesis can be relaxed at the price of a budget shift. Suppose the columns of k and k' differ by a product-wise constant, $d_{jk'l} = d_{jkl} + \alpha_l$ for all j, l. In a routing trimmed as in the proof of Proposition 8, customer k' receives exactly $q_{k'l}$ units of product l, each paying α_l more than under the column of k at the same depot. Replacing the column of k' by that of k therefore shifts the routing value of every feasible design by exactly $-\sum_l \alpha_l q_{k'l}$, and the shift reaches the optima because they are attained at trimmed routings, trimming never raising the cost in either instance. The merge of Proposition 8 then applies to the now identical columns once the budget is replaced by $B - \sum_l \alpha_l q_{k'l}$, the answer being NO outright when that quantity is negative, since costs are nonnegative. We keep τ as defined, the refined count of columns distinct up to product-wise shifts not being needed elsewhere.

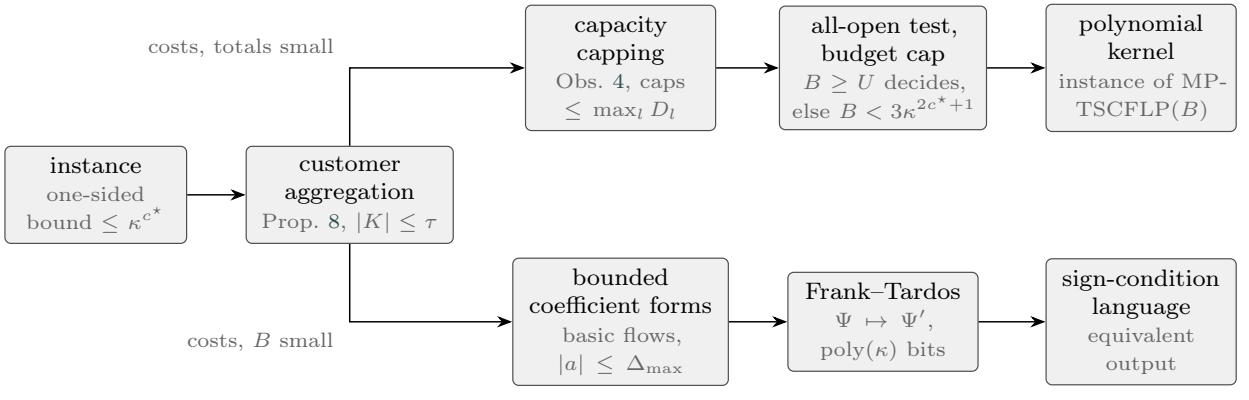
Observation 4 (capacity capping)**.** *Replacing every capacity b_{il} by $\min(b_{il}, D_l)$ and every p_{jl} by $\min(p_{jl}, D_l)$ preserves feasibility, routing value and cost of every design, hence the optimum and both decision answers.*

Appendix B.5 proves this by trimming any routing until every product moves exactly D_l across each stage, at no cost increase, after which no capacity beyond D_l can carry flow and the cap changes nothing.

The symmetric move on products is unsound. In analogy with Proposition 8, merging two products with identical cost columns would add their demands and add their capacity columns entrywise while keeping the

shared cost columns, and that operation lets the instance transfer capacity between the merged products. Take two factories with charges $f = (0, 10)$, one free depot of ample capacity, one customer, all transport costs zero, and two products of unit demand with capacities $b_{\cdot l} = (2, 0)$ and $b_{\cdot l'} = (0, 2)$. In the original instance (F1) for l' forces the second factory open at charge 10, while every other charge and every transport cost vanishes, so the optimum 10 is both forced and attained. The merged product has demand 2 against capacities $(2, 2)$ and the first factory alone serves it at cost 0, so the merge can lower the value. A variant shows the merge can also repair infeasibility. In the same setting give the two products the capacity columns $b_{\cdot l} = (2, 0)$ and $b_{\cdot l'} = (0, 0)$, so that the original instance is infeasible, (F1) failing for l' at every design, while the merged product has demand 2 against the column $(2, 0)$ and the first factory serves it. The natural rule that adds demands and capacity columns is therefore unsound, and the paper advances no product-side aggregation. A one-sided bound on the data still buys genuine preprocessing, a kernel on one side and a compression on the other, along the two routes that Figure 7 lays out.

Figure 7 The two preprocessing routes of Proposition 9. Aggregation (Prop. 8) bounds the customers by τ in both routes. When costs and total demands are bounded, capping (Obs. 4) bounds every capacity, the all-open cost U either decides the instance or caps the budget, and the output is a polynomial kernel, an instance of the problem itself. When costs and budget are bounded instead, total unimodularity of the oracle networks of Proposition 1 turns acceptance into sign conditions on integer forms with coefficients of magnitude at most $\Delta_{\max}$, and the Frank–Tardos step replaces the numeric vector Ψ by an equivalent Ψ' of $\mathrm{poly}(\kappa)$ bits, onto a language that is not itself a location problem.



Proposition 9 (kernel and compression under one-sided bounds)**.** *Fix $c^\star$ and let $\kappa = |I| + |J| + |L| + \tau$. The subclass of* MP-TSCFLP(B) *whose opening and transport costs f, g, c, d and whose total demands $D_1, \ldots, D_{|L|}$ are integers at most $\kappa^{c^\star}$, with capacities and budget unrestricted, admits a polynomial kernel in κ, an equivalent instance of* MP-TSCFLP(B) *itself of* $\mathrm{poly}(\kappa)$ *bits. The subclass with* $\max(f, g, c, d, B) \le \kappa^{c^\star}$ *and capacities and demands unrestricted admits a polynomial compression in κ into a fixed sign-condition language, which is not itself a location problem. The first subclass is nontrivial. The strongly hard slice with $|K| = |L| = 1$ of Theorem 6 has costs in $\{0, 1\}$ and a single customer whose demand, the only product total, is $|I|\,(|J|+1) \le \kappa^2$, so the slice lies in the subclass for every $c^\star \ge 2$.*

The proof, given in B.6, is elementary for the first subclass. Aggregation and capping leave every datum except the budget within $\kappa^{c^\star}$, the all-open design decides feasibility outright, and its cost, itself polynomially bounded in κ, caps the budget, so the output is an ordinary instance of the problem. No such cap exists in the second subclass, whose large numbers are the capacities and demands, and there acceptance becomes a Boolean combination of sign conditions on integer linear forms with coefficients that total unimodularity bounds by a fixed power of κ, which the Frank–Tardos theorem compresses to $\mathrm{poly}(\kappa)$ bits preserving every sign at once.

The matching negative result completes the account. Two cross-compositions of Set Cover show that, unless NP $\subseteq$ coNP/poly, the parameter of the dichotomy admits no polynomial kernel, and the same construction reaches the augmented parameters and the unary encoding.

Theorem 11 (no polynomial kernel)**.** *Unless* $\mathsf{NP} \subseteq \mathrm{coNP/poly}$, MP-TSCFLP$(B)$ *parameterized by* $|I|+|J|$ *admits no polynomial compression, and in particular no polynomial kernel, already on the subclass* $|I| = |L| = 1$ *and even with unary data. The exclusion transfers to the parameters* $|I|+|J|+|L|$ *and* $|I|+|J|+|K|$*, to the cardinality version* MP-TSCFLP(B, k)*, and to the subclass* $|K| = 1$*.*

Proof. We give an OR-cross-composition of Set Cover [13] into MP-TSCFLP(B) parameterized by $|I|+|J|$, which by the framework of Bodlaender et al. [27] rules out a polynomial compression unless $\mathsf{NP} \subseteq \mathrm{coNP/poly}$. The framework asks for a polynomial equivalence relation, and we take the relation R that declares two well-formed instances equivalent when they agree on the universe size $|U|$, the family size m and the target t, with $|U| \geq 1$ and $1 \leq t \leq m$. Four further classes are reserved, one for the malformed inputs, one for the well-formed instances with empty universe, one for those with nonempty universe and $t = 0$, and one for those with nonempty universe and $t > m$. Each instance falling in one of these four classes is decided individually in polynomial time, a malformed input being rejected by the syntactic check that recognizes it, an empty universe being covered by the empty subfamily, a target of zero over a nonempty universe admitting no cover, and a target above m admitting one exactly when the whole family covers the universe. On inputs from those classes the composition therefore answers the OR of those individual answers, emitting a fixed yes- or no-instance realizing it. Deciding R compares three numbers, and the triple $(|U|, m, t)$ takes at most cubically many values in the total encoding length of the input list, so R is a polynomial equivalence relation in the sense of the framework.

Take now T Set Cover instances $X_1, \ldots, X_T$ from one nontrivial class, sharing $|U| \geq 1$, m and t with $1 \leq t \leq m$, and assume $T = 2^s$ with $s = \lceil \log T \rceil$ by duplicating instances, duplicating once more when $T = 1$ so that $s \geq 1$. Identify the copies with the strings of $\{0,1\}^s$ and write c_a for the ath bit of the copy index c. The budget is $B = s(t+1) + t$, the amplifier is $Q = B + 1$, and the customers comprise $T|U|$ copy customers and s guard customers, one guard per index bit, each of demand Q, so a customer forced onto arcs of unit cost pays at least $Q > B$ even under divisible service, exactly the accounting of Theorem 1. The composed instance has a single product and a single factory of opening cost zero, capacity $(T|U| + s)\,Q$ equal to the total demand, and zero first-stage costs, so the whole construction lives in the depot layer and the second-stage matrix, and $|I| = |L| = 1$ throughout.

Depots. Use m set-depots $D_1, \ldots, D_m$ of opening cost 1, shared across all copies, and s pairs of selector depots (P_a^0, P_a^1), one pair per index bit, each of opening cost $t+1$. Every depot has capacity equal to the total demand. Sharing the set-depots is sound, and no instance is merged, because the copy-specific set membership is carried not by the depots but by the cost matrix to the copy-specific customers introduced next. Opening P_a^b reads bit a of the selected index as b. A guard customer per pair reaches either selector of its own pair at cost 0 and every other depot at cost 1, so an unguarded pair costs at least Q and at least one selector per pair must open. The opening charges cap the number of open selectors, since $s+1$ open selectors already cost $(s+1)(t+1) = s(t+1) + t + 1 = B + 1 > B$, so jointly with the guards a within-budget design opens exactly one selector per pair, fixing a single index $\iota \in \{0,1\}^s$. Because all routing costs are nonnegative, the selector charges leave for the set-depot openings a residual budget of exactly $B - s(t+1) = t$.

Customers. For every copy c and element $u \in U$ create a customer (c, u) with demand Q. The cost from set-depot D_j to (c, u) is 0 when u belongs to the jth set of copy c and 1 otherwise, so the whole of copy c's instance lives in these costs. The cost from selector P_a^b to (c, u) is 0 when $c_a \neq b$, and 1 otherwise.

Correctness. Fix the selected index ι. A copy $c \neq \iota$ differs from ι in some bit a, so the open selector $P_a^{\iota_a}$ serves all of copy c for free, and the non-selected copies cost nothing beyond the selectors already open. The copy ι agrees with the pattern on every bit, so every selector arc into its customers costs one, and each (ι, u), carrying demand Q, must be served through a set-depot whose set in copy ι contains u, since any other service costs at least $Q > B$. Each open set-depot costs 1 against the residual budget t, so at most t set-depots are open. Every element u therefore has an open set-depot whose set in copy ι contains it, and those sets form a cover of X_ι of size at most t. Conversely, if some X_ι has a cover of size at most t, open the factory, the selector pattern of ι and that cover. Every depot has capacity equal to the total demand, so with the free factory and these depots open (F1) and (F2) hold and the design is feasible by Proposition 2. Completeness of the two stages then lets every customer draw its demand through a zero-cost depot. The guards use the open selector of their own pair, the customers of a copy $c \neq \iota$ use the open selector $P_a^{\iota_a}$ at a bit a where c disagrees with ι, as in the forward direction, and the customers of copy ι use the depots of the

cover, so the routing cost is zero and the total cost is the fixed charge $s(t+1)+|C| \le s(t+1)+t = B$. A within-budget design therefore exists if and only if the OR of the T instances is a yes-instance.

Parameter and robustness. The depots number $m+2s$ and the factories one, so $|I|+|J| = O(m+\log T)$, polynomially bounded in the largest input and in $\log T$ exactly as the framework requires. The construction writes $O((T|U|+s)(m+2s))$ cost entries and numbers of $O(\log(T|U|m))$ bits, so it runs in time polynomial in $\sum_c |X_c|$, which is the framework's time bound. The largest numbers are the capacities, equal to the total demand $(T|U|+s)\,Q$. Here $Q = B+1 = s(t+1)+t+1 \le (s+1)(m+1) = O(m\log T)$ since $t \le m$, and $s \le T \le T|U|$ since $|U| \ge 1$, so the total demand is $O(T\,|U|\,m\log T)$, hence polynomial in the output size, so the composition is robust to a unary encoding, and the single product places it in the subclass $|I| = |L| = 1$ while inflating $|K|$.

Transfer to $|K| = 1$. The dual construction keeps one depot and one customer and moves the composition to the factory and product sides, mirroring the primal exactly as Corollary 3 mirrors Theorem 2. Introduce a product (c,u) of demand Q for every copy c and element u. Introduce m shared set-factories $F_1,\dots,F_m$ of opening cost 1, whose first-stage cost to product (c,u) is 0 when u lies in the ith set of copy c and 1 otherwise, together with s pairs of selector factories of opening cost $t+1$, whose cost to (c,u) is 0 when $c_a \ne b$ for the pair's bit a and read value b, and 1 otherwise. Add a guard product per pair, of demand Q, free through either selector factory of its pair and costing 1 elsewhere. The budget is again $B = s(t+1)+t$ with amplifier $Q = B+1$. The single depot and the single customer are free, the customer demands Q units of every product, and all second-stage costs vanish. Every factory and the depot hold, in each product separately, capacity equal to the total demand, so feasibility reads through (F1) and (F2) as before, and since the second stage carries zero cost the entire penalty sits on the first stage. The paragraphs on depots and correctness of the primal now apply verbatim under the substitution of set-factory for set-depot, selector factory for selector depot, product (c,u) for customer (c,u), guard product for guard customer, and first-stage cost for second-stage cost, with (F1), (F2) and the budget arithmetic unchanged. In the transferred construction the single depot receives every product through the first stage and forwards it to the single customer, so constraint (3) reads, for each product, first-stage inflow at least second-stage outflow, and the routing that serves each product through its designated factory satisfies it with equality. The orientation of the balance constraint is therefore respected and the budget arithmetic is unchanged. A within-budget design therefore selects one index ι and covers copy ι's products with at most t set-factories, and conversely a cover of size at most t yields a design of cost at most B. This keeps $|K| = 1$ while inflating $|L|$, with $|I|+|J| = (m+2s)+1 = O(m+\log T)$ and all numbers polynomial in the output, so it excludes a polynomial kernel on the subclass $|K| = 1$ too.

Finally, the augmented parameters match the two compositions as follows. Adding $|L|$ leaves $|I|+|J|+|L|$ polynomially bounded in the primal composition, where $|L| = 1$, and adding $|K|$ leaves $|I|+|J|+|K|$ polynomially bounded in the dual composition, where $|K| = 1$, so each augmented parameter inherits the exclusion from its composition. Appending the cardinality bound $k = 1+s+t$ is likewise redundant. The total demand is positive, so (F1) forces the single factory of the primal open and (F2) forces the single depot of the dual open, and every within-budget design opens that forced facility, exactly one selector per pair and at most t set-facilities, hence at most $1+s+t = k$ facilities in all. The witness of the converse direction opens $1+s+|C| \le 1+s+t = k$ of them, so the bound thins the set of witnesses and breaks neither direction. □

Fixed-parameter tractability in $|I|+|J|$ still yields an exponential kernel, by the standard equivalence between fixed-parameter tractability and kernelization [e.g. 16], since on instances larger than a threshold depending on the parameter the FPT algorithm runs in polynomial time and a trivial equivalent instance can be output, while below the threshold the instance is its own kernel. What Theorem 11 rules out is only the polynomial one, and whether a polynomial kernel exists for the full parameter $|I|+|J|+|K|+|L|$ with general numbers, where the yes-certificate comparison becomes genuinely bilinear, is left open.

5.6. Width parameters

A reader versed in algorithmic meta-theorems might hope to route the tractable side through graph width, and this last remark closes that door for the representation we analyse. The natural incidence graph carries no useful width, and over that representation the obstruction is arithmetic rather than structural.

Observation 5 (width is uninformative)**.** *The incidence structure comes first. For every instance the layered incidence graph on $I \cup J \cup K$ is the complete bipartite graph $K_{J,\,I\cup K}$, since $K_{I,J} \cup K_{J,K} = K_{J,\,I\cup K}$. Its*

clique-width is at most 2*, witnessed by the expression that introduces every vertex of* J *with label* 1*, introduces every vertex of* $I \cup K$ *with label* 2*, and performs a single join between the two labels.*

Its treewidth [28] equals $\min(|J|, |I| + |K|)$ *once both sides are nonempty, and is therefore unbounded over the class. Indeed, for* $K_{m',n'}$ *with* $1 \leq m' \leq n'$*, the bags formed by the smaller side together with one vertex of the larger side, one bag per such vertex and arranged along a path, give a decomposition of width* m'*. The matching lower bound follows by contracting a matching of size* $m' - 1$ *between the two sides. The contracted vertices become pairwise adjacent, and the remaining vertex of the smaller side and any remaining vertex of the larger side are adjacent to all of them and to each other, so* $K_{m',n'}$ *has a* $K_{m'+1}$ *minor, and since* $K_{m'+1}$ *has treewidth* m' *and treewidth never increases under minors [28], the treewidth is exactly* $m' = \min(m', n')$*.*

The logical consequence closes the observation. The treewidth-one star slice with $|J| = 1$ *is already strongly NP-hard, by Theorem 2 (which fixes* $|J| = |K| = 1$*). Were the problem expressible in the* MSO_1 *optimization logic over the unweighted incidence graph used here, the clique-width meta-theorem [29] together with the bound of at most two would force a polynomial algorithm on the whole class, contradicting strong NP-hardness unless* $P = NP$*, so that route is closed. The capacity and demand constraints compare sums of numbers, which that logic over that graph cannot express, and over the representation used here the obstruction is therefore arithmetic rather than structural. This persists even when every capacity, demand and cost lies in* $\{0, 1, m + 1\}$ *and the budget is at most* m*, where* m *is the size of the set family in the construction of Theorem 2. Richer relational structures, weighted or counting logics, or representations carrying products and constraints as objects are not addressed by this observation, and their width status simply remains open.*

The results of Sections 3 to 5 assemble into a single map, and Table 3 records it. Figure 8 shows which result feeds which before the table lists them row by row. Each row names a parameter or a structural restriction, states the encoding under which the entry holds, gives the status, and traces it to the theorem that establishes it. The rows descend from the polynomial base through the covering layer, the size-parameter dichotomy, the parameter k and the numeric layer to the kernelization and width entries. The table and the figure carry the detail row by row.

Figure 8 The results of the paper as a diagram, each box naming its numbered statements. A solid arrow points from a result to one whose proof uses it, and no other relation is drawn. The structural layer feeds every branch, the hardness constructions carry the whole negative side, from the W[2] and ETH bounds (Theorems 4 and 5) through the kernel exclusion (Theorem 11) to the negative half of the dichotomy (Theorem 10), and the oracle feeds the algorithms, the cuts and the kernel and compression routes (Prop. 9).

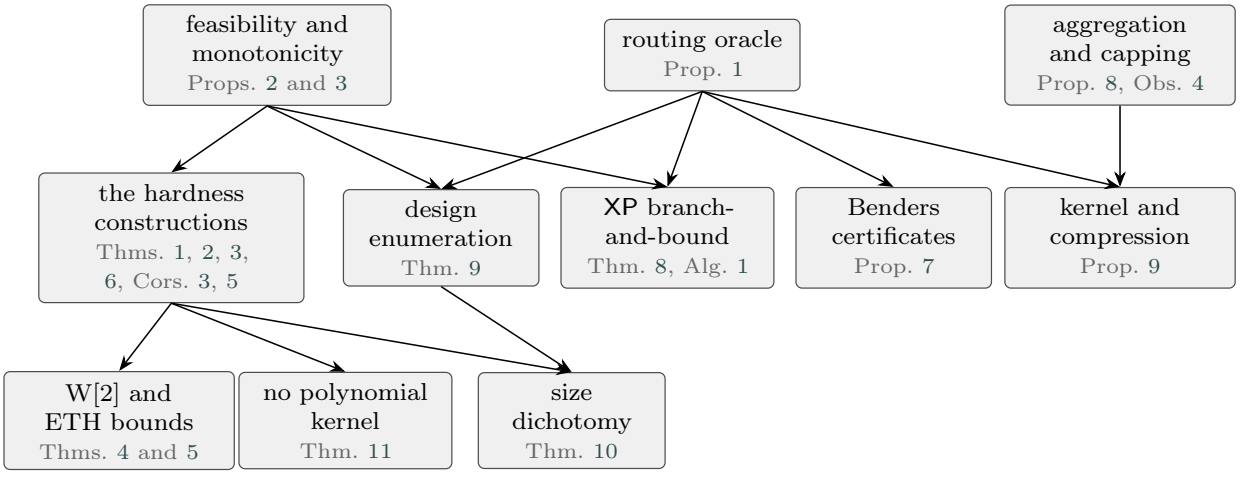


Table 3: The parameterized complexity landscape of MP-TSCFLP. Each row is a parameterization, a restriction, or a preprocessing guarantee, and hard rows either name a witness slice or cite the witnessing construction in their source cell. Sources refer to results of Sections 3 to 5.

Parameter / restriction	Encoding	Status	Source
Polynomial base			
$f \equiv 0$ and $g \equiv 0$	both	P	Prop. 4
$\lvert I\rvert = \lvert J\rvert = 1$	both	P	Prop. 5
Covering layer			
classical, no parameter, witness slice $\lvert I\rvert = \lvert L\rvert = 1$	both, unary-robust	strongly NP-hard	Thm. 1
approximation, witness $\lvert I\rvert = \lvert L\rvert = 1$	both, unary-robust	no $\max\{1, (1-\varepsilon)\ln\lvert K\rvert\}$ factor unless P = NP	Cor. 1
$\lvert K\rvert = 1$, products carry the universe, witness slices $\lvert J\rvert = 1$ and $\lvert I\rvert = 1$	both, unary-robust	strongly NP-hard	Thm. 2, Cor. 3
Size-parameter dichotomy (under P ≠ NP, FPT iff $\{\lvert I\rvert, \lvert J\rvert\} \subseteq P$)			
$P \supseteq \{\lvert I\rvert, \lvert J\rvert\}$ (4 sets)	both	FPT, $2^{\lvert I\rvert+\lvert J\rvert}\,\mathrm{poly}(N)$	Thm. 9
$\emptyset$, $\{\lvert I\rvert\}$, $\{\lvert J\rvert\}$, $\{\lvert K\rvert\}$, $\{\lvert L\rvert\}$, $\{\lvert I\rvert, \lvert K\rvert\}$, $\{\lvert I\rvert, \lvert L\rvert\}$, $\{\lvert J\rvert, \lvert K\rvert\}$, $\{\lvert K\rvert, \lvert L\rvert\}$	both	para-NP-hard, strong slice	Thm. 10, Cor. 2, Cor. 4, Cor. 6
$\{\lvert J\rvert, \lvert L\rvert\}$, $\{\lvert I\rvert, \lvert K\rvert, \lvert L\rvert\}$, $\{\lvert J\rvert, \lvert K\rvert, \lvert L\rvert\}$	binary	para-NP-hard, weak slice	Thm. 10, Thm. 3, Cor. 5
$\{\lvert J\rvert, \lvert L\rvert\}$, $\{\lvert I\rvert, \lvert K\rvert, \lvert L\rvert\}$, $\{\lvert J\rvert, \lvert K\rvert, \lvert L\rvert\}$	unary	open	Cor. 6
The parameter k			
k, lower bound, already at $\lvert I\rvert = \lvert L\rvert = 1$, also budget B alone and depot count k_z	both	W[2]-hard	Thm. 4
k, membership	both	in W[P] ∩ XP	Rem. 3, Thm. 8
k, algorithm	both	XP, $O(n^k(T_{\mathrm{orac}} + n\lvert L\rvert))$	Thm. 8
k, fine bound	both	no $f(k)N^{o(k)}$ under ETH	Thm. 5
a priori bound k^*	both	valid lower bound on the opening count, exact per side and product	Lem. 1
Numeric layer			
$\lvert K\rvert = \lvert L\rvert = 1$, general transport	both, unary-robust	strongly NP-hard	Thm. 6
$\lvert K\rvert = \lvert L\rvert = 1$, separable $c_{ij} = \gamma_i + \delta_j$	binary	pseudo-polynomial	Thm. 7
$\lvert J\rvert = \lvert L\rvert = 1$ ($\lvert I\rvert, \lvert K\rvert$ free), hard already at $\lvert K\rvert = 1$	binary	weakly NP-hard, pseudo-polynomial	Thm. 3, Obs. 2, Thm. 7
$\lvert I\rvert = \lvert K\rvert = \lvert L\rvert = 1$, depot mirror	binary	weakly NP-hard, pseudo-polynomial	Cor. 5
Kernelization and width			
customer aggregation, identical columns	both	exact, $\lvert K\rvert \le \tau$	Prop. 8
capping $b_{il}, p_{jl} \le D_l$	both	exact	Obs. 4
kernel in $\lvert I\rvert + \lvert J\rvert$	both	exponential (standard equivalence)	Thm. 9
polynomial kernel in $\lvert I\rvert + \lvert J\rvert$ ($+\lvert K\rvert$, $+\lvert L\rvert$, and the cardinality version), already at $\lvert I\rvert = \lvert L\rvert = 1$ and at $\lvert K\rvert = 1$	both, unary-robust	none unless NP ⊆ coNP/poly	Thm. 11
kernel in $\kappa = \lvert I\rvert + \lvert J\rvert + \lvert L\rvert + \tau$, costs and total demands bounded by $\kappa^{c^\star}$ (capacities and B free)	binary	polynomial kernel	Prop. 9
compression in κ, costs and B bounded by $\kappa^{c^\star}$ (capacities and demands free)	binary	polynomial compression	Prop. 9

continued on the next page

Table 3 (continued)

Parameter / restriction	Encoding	Status	Source
polynomial kernel in $\|I\| + \|J\| + \|K\| + \|L\|$, general numbers	binary	open	Sec. 5.5
$\mathrm{cwd}(G_{\mathrm{inc}})$	—	≤ 2, uninformative	Obs. 5
$\mathrm{tw}(G_{\mathrm{inc}})$	—	unbounded, slice tw $= 1$ strongly hard	Obs. 5, Thm. 2

6. Computational study

The experiments pursue three questions. (Q1) asks whether the enumeration algorithm of Theorem 8 is practical in its predicted regime and where that regime ends. (Q2) asks how it compares with the standard mixed-integer approach. (Q3) asks how the covering bound k^*, available before anything is solved, relates to the gaps a mixed-integer solver reaches under a fixed limit, and at which stage of the solving process the association is anchored. The remainder of the section is organized so that the reader can locate each answer directly. Section 6.1 describes the codes, the instance families and the machines. Section 6.2 answers Q1 on the family whose covering bound is controlled by construction, Section 6.3 addresses Q2 against the integer programming baseline, and Section 6.4 addresses Q3 by confronting k^* with the gaps of a solver that makes no use of it. The code and the instances used throughout the section are available in the repository at `https://github.com/pehgonzalez/mp-tscflp-parameterized`.

6.1. Setup

The study compares three exact codes. The first (Xp) is a C++20 implementation of Algorithm 1, with the covering pruning, the admissible cost bound and the dominance rule of Section 5, and a deadline-aware enumerator. The baseline (Mip) is the compact model (1)–(6) solved by Gurobi. A third code (Bd) solves the same model by a branch-and-Benders-cut in which the per-product transportation subproblems are separated by the disaggregated cuts of Proposition 7. The two baselines are C++20 codes as well, built with CMake against Gurobi's C++ interface, and they share the instance parser and the exact routing oracle with the enumeration code. All three read the same instance files. The Xp implementation was cross-validated against an independent brute-force enumeration of all designs with an exact per-product min-cost-flow oracle, matching it on 180 small seeded instances and on 1,381 exact comparisons of optimal value, feasibility and covering bound across a full sweep of the cardinality, with no disagreement. The verification battery in the accompanying code spans design enumerations, closed-form comparisons and LP cross-checks, and every one of its roughly nine hundred thousand runs passed. More than 560,000 of the checks are exhaustive design enumerations validating the numeric-layer constructions alone, roughly ten thousand are equivalence verifications, two hundred are independent LP cross-checks, 1,381 are cross-solver comparisons, and the remainder are per-construction parameter sweeps.

The study uses two instance families, the first with a main grid and a boundary extension, the second the benchmark. The first is a family with controlled covering bound, built by a deterministic generator seeded by the parameter tuple, in which, on each side, one facility fewer than the per-side target receives capacities that jointly fall one unit short of the binding product's demand, a choice that pins the per-side covering count to the target. The main grid has target per-side count in $\{2, 3, 4, 5, 6\}$, so $k^* \in \{4, 6, 8, 10, 12\}$, factory and depot counts in $\{20, 40\}$, customers in $\{50, 100\}$, products in $\{5, 10\}$ and five seeds, for 400 instances. A boundary extension on smaller graphs, with per-side target in $\{2, 4, 6, 8\}$ so $k^* \in \{4, 8, 12, 16\}$ and $n = |I| + |J|$ over $\{20, 24, 28, 30, 40\}$, probes where the regime ends, adding 60 instances for 460 runs in all. A second boundary sample, built for the budget question of Q1, reruns the same boundary grid with ten seeds per cell under a tenfold budget. Seeds one to three coincide byte for byte with the 60 s sample and the remaining seven per cell are fresh, for 200 instances, 140 of them new, with their own manifest. The pin is guaranteed by the construction and not only checked afterwards. On a side with target $t \geq 2$ and binding demand D, the $t - 1$ designated facilities carry capacities of $\lfloor (D-1)/(t-1) \rfloor$ or one more that sum to exactly $D - 1$, every other facility of the side carries at most $\lfloor (D-1)/(t-1) \rfloor$, and every capacity is at least one. The largest $t - 1$ capacities therefore sum to exactly $D - 1$ while the largest t sum to at least D, and Lemma 1 puts the covering count of the binding product at exactly t. Every other product receives capacities of at least its total demand divided by t, rounded up, so its count is at most t, the per-side maximum stays at the target, and

k^* equals the sum of the two targets on every instance and every seed. The generator additionally records the recomputed bound alongside each instance, and on all 660 instances the recomputed k^* equals the target exactly, an implementation check of the construction just stated. The second family is the classical benchmark of Mauri et al. [4], its 100 instances split into four size groups $|I|$-$|J|$-$|K|$ of 50-100-200 and 100-200-400 with $|L| \in \{5, 10\}$, for which no optimal value was proven in Mauri et al. [4] nor, to our knowledge, since. On these instances the customer-aggregation rule of Proposition 8 never applies, since in every one of the 100 instances no two customers share their full cost column and hence $\tau = |K|$. Capacity capping leaves every benchmark capacity unchanged, since no capacity exceeds its product's total demand, customer aggregation never applies as just noted, and the covering bound is thus the only signal the paper's toolkit produces on these instances. One by-product of parsing them is used repeatedly below and is recorded here. Every datum is integral, so the optimal value is integral by Proposition 1, and a solver that reports optimality with an absolute gap below one therefore determines the exact optimum, subject to the floating-point tolerances under which the solver certifies its bound, a caveat Section 6.3 makes explicit for the three closed runs. The terminal gap of a run is the relative optimality gap the solver reports when it stops, the difference between incumbent and bound divided by the incumbent, quoted in per cent throughout.

The enumeration runs were executed under a fixed 60 s limit. The Xp binary is single-threaded C++20, compiled with g++ 13.3.0 at optimization level O2, and ran on a Linux virtual machine with an Intel Xeon processor at 2.8 GHz. The integer programming runs use Gurobi 13.0.2 on an Intel Core i9-14900HX with 24 cores under Windows, with the controlled family limited to 600 s and the benchmark to 3,600 s. Gurobi ran with default tolerances, automatic thread count and a fixed seed recorded with every run, and on each run the harness recomputed the incumbent's cost independently from its flows, with agreement on all 900 runs, the 400 Mip runs, the 400 Bd runs and the 100 benchmark runs. The enumeration and the integer programming codes ran on different machines, so times are compared only within a code and never across codes, and the solved-versus-censored contrasts that the section draws are insensitive to the hardware difference. The asymmetry of the limits favours the baseline by design. The enumeration limit maps where that algorithm stops being fast, the larger Gurobi budget gives the baseline its best chance of producing reference optima, and a censored run enters a speed comparison only through the lower bound its limit certifies. The time of a censored run is recorded at its limit. Every completed enumeration run is deterministic, and each instance was run once. The software archive with the codes, the instance generator and manifests, the raw result files and one verification script per hardness construction accompanies the submission as supplementary material, in the repository whose address opens the section.

6.2. The regime of the enumeration algorithm (Q1)

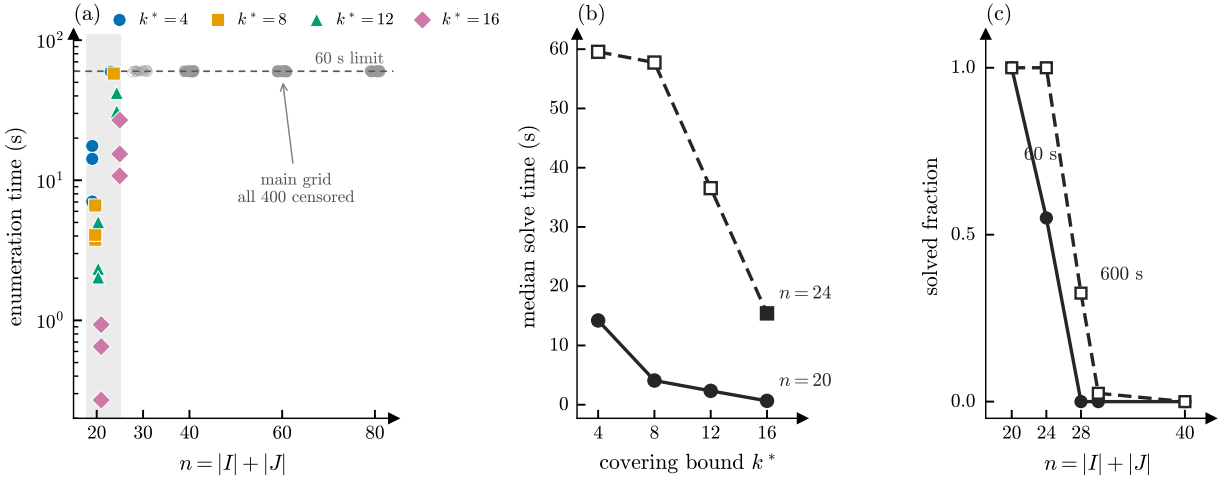
The controlled family was built to probe the boundary that the support-enumeration bound of Theorem 8 motivates, and the outcome is unambiguous. The main grid, whose graphs all have $n = |I| + |J| \geq 40$, lies entirely outside the 60 s regime, so all 400 instances are censored, which is the finding itself and not a failure of protocol. The two node medians we quote cover different populations and are not two readings of one quantity. Over the censored runs at $n \geq 40$ the search processes a median of roughly 2,500 nodes before the limit stops it, whereas over the searches that complete at $n = 24$ the median is roughly 23,000 nodes, so the second figure counts a finished enumeration and the first counts what fits in a minute. The cost of each node grows with $|I|$, $|J|$, $|K|$ and $|L|$ through the oracle, which is why the censored searches at $n \geq 40$ reach so few nodes. The raw per-run records in the archive carry node, pruning and oracle counters. Two boundary samples locate the frontier, and Table 4 reports them together. The first holds three seeded runs per cell under the 60 s limit of the main protocol, and the second reruns the same grid with ten seeds per cell under the tenfold budget of 600 s, on the same virtual machine. The two samples share only three of the ten seeds, so reading the budget effect across them would still confound the budget with the sampling, and the budget contrast is therefore read inside the ten-seed sample alone. The code is deterministic and single-threaded and every record stores its solve time, so a run of that sample counts as solved under 60 s exactly when its recorded time is at most 60 s, and the reclassification is stable because the nearest recorded solve time lies more than seven seconds from the threshold. Medians are taken over solved runs only, so the times in partially solved cells are biased downward by that conditioning, and three runs fix a cell median only coarsely, so the medians of the three-seed rows are read as orders of magnitude. On the same ten seeds per cell, the 60 s budget solves every instance at $n = 20$, twenty-two of forty at $n = 24$ and none from $n = 28$ upward,

while 600 s solves both $n = 20$ and $n = 24$ in full, thirteen of forty at $n = 28$, one of forty at $n = 30$ and none at $n = 40$. The border of the regime is therefore a property of the implementation and the budget together, a tenfold budget displacing the partially solved band by one grid step on identical instances, from $n = 24$ to $n = 28$, consistent with the exponential sensitivity that the support-enumeration bound of Theorem 8 motivates. The same table shows the covering-bound effect at fixed size, with the median time at $n = 20$ falling from 14.2 s at $k^* = 4$ to 0.7 s at $k^* = 16$ under 60 s. Figure 9 shows the three readings graphically, the boundary in panel (a), the covering-bound effect in panel (b), and the budget displacement in panel (c).

Table 4 The boundary extension of Q1 as a function of the budget. Each cell reports solved over attempted instances at the given $n = |I| + |J|$, budget and covering-bound target k^*, with the median time in seconds of the solved runs in parentheses. The 60 s rows use the three-seed boundary extension. The ten-seed rows both draw on the 600 s budget sample, the middle row counting a run as solved under 60 s exactly when its recorded time is at most 60 s, which the deterministic single-threaded code permits, so the budget contrast of the last two rows compares the same instances and only the budget varies. The tenfold budget displaces the partially solved band from $n = 24$ to $n = 28$, where the solved count again rises with the bound, and $n = 40$ stays fully censored. Starred cells have fewer than three solved runs, so their medians rest on one or two observations.

n	budget (s)	$k^* = 4$	$k^* = 8$	$k^* = 12$	$k^* = 16$
20	60	3/3 (14.2)	3/3 (4.1)	3/3 (2.3)	3/3 (0.7)
	60 (ten-seed)	10/10 (15.5)	10/10 (7.4)	10/10 (3.7)	10/10 (0.9)
	600	10/10 (15.5)	10/10 (7.4)	10/10 (3.7)	10/10 (0.9)
24	60	1/3 (59.6)*	1/3 (57.7)*	2/3 (36.5)*	3/3 (15.4)
	60 (ten-seed)	1/10 (48.8)*	2/10 (49.9)*	9/10 (30.3)	10/10 (17.3)
	600	10/10 (90.6)	10/10 (108.2)	10/10 (31.3)	10/10 (17.3)
28	60	0/3	0/3	0/3	0/3
	60 (ten-seed)	0/10	0/10	0/10	0/10
	600	1/10 (469.2)*	2/10 (439.5)*	4/10 (339.4)	6/10 (194.3)
30	60	0/3	0/3	0/3	0/3
	60 (ten-seed)	0/10	0/10	0/10	0/10
	600	0/10	0/10	0/10	1/10 (447.0)*
40	60	0/3	0/3	0/3	0/3
	60 (ten-seed)	0/10	0/10	0/10	0/10
	600	0/10	0/10	0/10	0/10

Figure 9 The enumeration algorithm on the controlled family. Panels (a) and (b) draw the 460 runs under the 60 s limit. Panel (a) plots time against $n = |I| + |J|$. Solved runs are filled and coloured by the covering bound k^*, and they occur only in the shaded band up to $n \approx 24$. Every run beyond it is censored, shown as grey open circles on the limit line, including the whole 400-instance main grid, which lies at $n \geq 40$. Panel (b) gives the median solve time against k^* at $n = 20$ and $n = 24$, where instances with a larger bound solve faster, with medians taken over solved runs only. Open markers in panel (b) are medians over fewer than three solved runs. Panel (c) reads both budgets inside the ten-seed budget sample, a run counting as solved at 60 s exactly when its recorded time is at most 60 s, so the two curves compare the same instances and the tenfold budget displaces the censoring wall from $n \approx 24$ to $n \approx 28$. Lower is better in panels (a) and (b), and the walls of panel (c) are the observed borders of the regime motivated by Theorem 8.



Within the solvable range the covering bound acts in a direction that requires explanation. Under the 60 s budget, time and node counts decrease as k^* grows, and the solved count at $n = 24$ in Table 4 rises with the bound in both samples, from one in three to three in three over the three-seed rows, and from one in ten at $k^* = 4$ through two in ten and nine in ten to ten in ten at $k^* = 16$ on the reclassified ten-seed row. The tenfold budget replicates the pattern one grid step further out, the solved count at $n = 28$ rising from one in ten to six in ten across the same sweep. The covering pruning of Lemma 1 might seem the natural mechanism, but Observation 3 rules that reading out in advance, for these runs perform plain optimization, and there every node the covering rule discards is discarded by the infeasibility branch of P2 as well, so the rule can change which counter certifies a closure and what certifying it costs, never the tree. An ablation confirms the account on the machine. The enumeration carries a switch that disables the covering rule and leaves every other component of the search untouched, and we ran the 24 instances of the two solvable sizes twice under the 60 s protocol, once with the rule and once without it. On the 18 pairs whose searches terminate under both settings the two runs agree node for node, and in every one of them the counter of the covering rule migrates without loss into the counter of the feasibility test. The remaining pruning counters and the optimal values are identical as well, exactly as the observation prescribes. What the rule removes is oracle work, since it decides a node by a prefix scan over capacities where the test solves a minimum-cost flow, and over the 18 terminating pairs the share of nodes decided the cheap way climbs with the bound, from roughly one node in seven hundred at $k^* = 4$ to one node in ten at $k^* = 16$. At these sizes the saving stays inside the run-to-run noise, the 18 terminating pairs costing 242 s with the rule against 245 s without it, and the slower of the two settings splits nine against nine over the individual instances. The six remaining pairs are censored on at least one side, where a node count measures throughput under the limit rather than a completed tree, and their counts differ accordingly.

The explanation of the effect therefore does not run through the covering rule. The capacities that define k^* keep shaping the search through the feasibility test and through the routing oracle, tight capacities closing

subtrees early in whichever form certifies the closure, so k^* is best read as an index of that tightness and not as a lever acting through the rule that computes it. The mirror pattern expected for the cardinality sweep, in which the exponent n^k grows with the target count, is left for a dedicated run.

6.3. Comparison with the mixed-integer approach (Q2)

Q2 places the enumeration and the integer programming codes side by side, each under its own protocol, and the observed outcome is an empirical result consistent with the worst-case limitation of support enumeration rather than a consequence of it. Outside the small-n regime the weight-independent enumeration bound gives XP no way to prune below n^k, whereas the compact model carries the linear structure a modern solver exploits, so MIP is expected to close quickly the very controlled-family instances on which XP exhausts its 60 s budget. The campaign realizes that expectation, and Table 5 summarizes it. MIP closes all 400 controlled instances with a median solving time of 0.55 s, among them every instance the enumeration had left censored at 60 s.

Table 5 The three exact codes on the 400-instance main grid of the controlled family, whose graphs all have $n \geq 40$. Solved means proven optimal within the per-code limit. The median time is over solved runs only, in seconds. MIP closes every instance the enumeration leaves censored, the empirical face of the regime border of Q1. The enumeration ran on a different machine from the two integer programming codes, which shared one machine, so times compare only within a code and between MIP and BD, and the cross-code reading is the solved-versus-censored contrast.

code	limit (s)	solved	median time (s)
XP	60	0/400	–
MIP	600	400/400	0.55
BD	600	8/400	395.57

The value of Theorem 8 is thus classificatory rather than competitive, and the observed dominance is no defeat of the algorithm but an empirical face consistent with the same hardness the theory maps. What the exact codes earn is not speed but proof, since by the integrality remark of Section 6.1 a solver that closes an instance settles a value the benchmark of Mauri et al. [4] had left to heuristics. On the benchmark itself the 3,600 s budget proves optimality on three of the 100 instances, all in the smaller size group with five products, values the original study left unproven. The three instances are PSC1-C1-50-5, PSC4-C1-50-5 and PSC5-C1-50-5, closed at optimal values 1,923,801, 1,909,659 and 1,895,151 in 2,001, 642 and 883 seconds. Each value lies strictly below the best solution reported for the same instance by Mauri et al. [4], respectively 1,923,813, 1,910,011 and 1,896,699, so the campaign closes the three instances and improves their best-known solutions at the same time. On these three runs the stored bound equals the incumbent exactly and the logged relative gap is zero at the eight decimal places the harness records, so the absolute gap lies below one as the exactness argument of Section 6.1 requires, subject to the solver's floating-point bound certificate, with the incumbent costs confirmed by the independent recomputation noted above. The branch-and-Benders-cut BD enters here not as a faster solver, for it closes 8 of the 400 controlled instances within the same budget, but as a device for cross-validation and for the structural reading of Proposition 7, and the codes are compared only on optimal values they jointly certify.

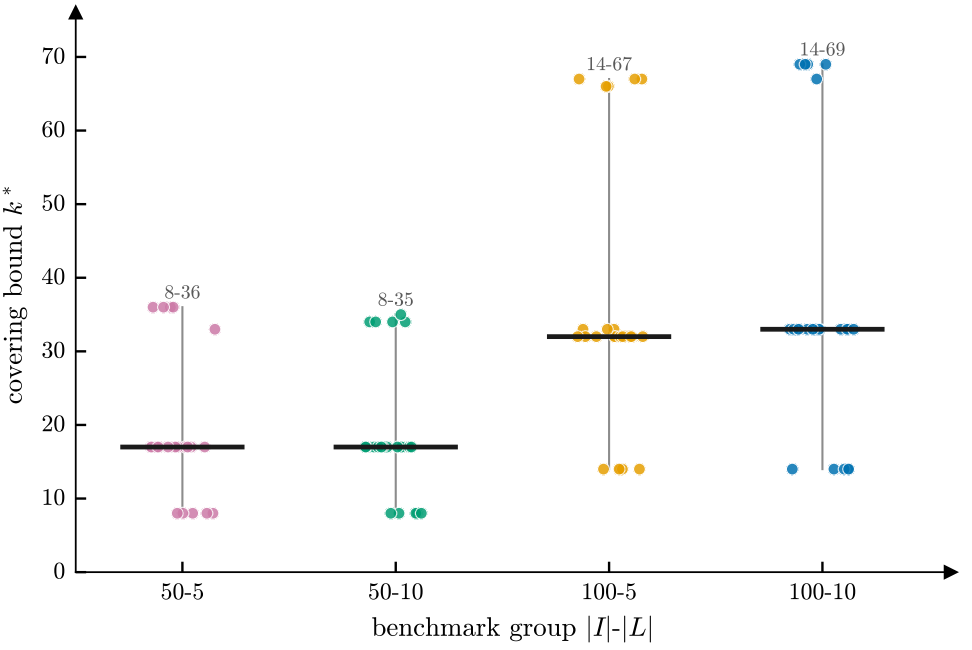
6.4. The covering bound and the root relaxation (Q3)

The last question confronts the covering bound with a solver that makes no use of it, asking whether k^* is associated with the gaps the solver reports under a fixed limit and at which stage of the solving process the association is anchored. We therefore confront the bound with the integer solver on the benchmark that the original study left without a single proven optimum. The bound is available before anything is solved, at the cost of sorting each capacity vector once, and it varies widely across the benchmark, as Figure 10 shows. Table 6 reports the campaign by size group, the range of the bound, the median terminal gap, the proven optima and the within-group correlations with their Holm-adjusted p-values. Within every size group the bound spreads over a factor of four or more, and this spread at fixed size removes the most evident confounder, instance size, while other instance features still vary within a group, so the bound may in part proxy properties the design does not separate.

Table 6 The benchmark campaign by size group, 3,600 s per instance. Columns give the group $|I|$-$|L|$, the instance count, the minimum, median and maximum of the covering bound k^*, the median terminal gap in percent, the number of proven optima, the within-group Spearman correlation between k^* and the terminal gap, and its Holm-adjusted two-sided permutation p-value. Correlations carry percentile bootstrap 95% confidence intervals in brackets. Each interval describes one group alone, and the within-group column is the sharper instrument, the pooled value mixing the group effect with size. The pooled correlation over all 100 instances is -0.28, Holm-adjusted $p = 0.013$.

group	#	k^* min/med/max	gap (%)	opt.	ρ [95% CI]	p
50-5	25	8/17/36	0.4	3	-0.59 [-0.81, -0.22]	0.006
50-10	25	8/17/35	0.4	0	-0.39 [-0.69, $+0.09$]	0.102
100-5	25	14/32/67	1.1	0	-0.70 [-0.86, -0.45]	0.001
100-10	25	14/33/69	4.2	0	-0.40 [-0.75, $+0.12$]	0.102

Figure 10 The covering bound k^* across the four benchmark groups. Each point is one instance and the bar marks the group median. The bound spreads by a factor of more than four inside every group, varying across instances of equal size, so its variation is not a proxy for instance size.



The completed campaign lets the association between k^* and the terminal gaps Mip reports on the 100 benchmark instances be read directly, with a within-group reading that separates the association of the bound from the association of size. Pooled over the benchmark, the covering bound and the terminal gap are rank-correlated with Spearman $\rho = -0.28$. Ninety-seven of the hundred runs end at the limit, so solving time carries no usable signal on this benchmark and the analysis reads the terminal gap alone. The terminal gap blends primal and dual progress and depends on the objective scale and on the solver tolerances, so it is an imperfect proxy of computational effort. Within the four size groups the association strengthens, the four coefficients being negative throughout and ranging from -0.39 to -0.70 with descriptive mean -0.52, as the ρ column of Table 6 shows. Significance was assessed by a two-sided permutation test on the Spearman statistic. The test is Monte Carlo with 20,000 permutations under the fixed seed 20260718, every bootstrap interval of the section uses 10,000 resamples under the fixed seed 20260719, the p-value uses the add-one convention so it cannot vanish, ties including zero gaps are handled by mid-ranks, and every reported p-value is Holm-adjusted over the five gap correlations. The pooled correlation and the two five-product groups

survive the correction, with adjusted p-values of 0.013, 0.006 and 0.001, while the two ten-product groups do not, and the p column of Table 6 reports the adjusted value of every group. The direction is negative in all four groups, established where the test passes and inconclusive where it does not. The bootstrap intervals of Table 6 tell the same story, excluding zero exactly in the groups that survive the correction. Figure 11 plots the hundred instances behind these coefficients, and the negative drift is visible inside each group. The sign is the substantive finding. A naive reading of the bound would predict the opposite, a small k^* meaning few facilities to open and hence an easy instance, but on this benchmark larger bounds go with smaller terminal gaps, the same inverse direction the enumeration exhibits in Q1, where tight capacities let the search certify infeasibility soonest.

The stored solver logs let the association be examined conditionally on the root relaxation with no further runs, the extraction being pinned to a script and its coefficients collected in Table 7. The reading rests first on how the campaign is built. Ninety-seven of the hundred runs stop at the time limit, and a run that stops there reports the distance between its incumbent and a dual bound that has moved little from the root value, so on this benchmark the terminal gap is close to a deterministic image of the root relaxation gap. The measured coefficients bear that out, the two quantities rank-correlating at $+0.97$ pooled and between $+0.89$ and $+1.00$ within groups. Whatever the covering bound is associated with, it is thus associated with a quantity settled before the branch-and-bound search begins, and the conditional coefficients below are a descriptive check consistent with that reading, not a causal identification.

Table 7 Conditional association coefficients on the 100 benchmark runs, by size group and pooled. Rows give the Spearman correlation of the covering bound with the terminal gap, of the bound with the root relaxation gap, of the two gaps with each other, the partial rank correlation of the bound with the terminal gap at fixed root gap, and the correlation of the bound with the explored node count. The partial coefficient is the Pearson correlation of the residuals of the two rank-on-rank regressions on the root-gap ranks, and the pooled column conditions on the root gap only, not on the group. Brackets hold percentile bootstrap 95% confidence intervals over 10,000 resamples of the instances (seed 20260719), not adjusted for multiplicity and drawn without stratification, so the pooled interval retains the group-composition component that the within-group permutation removes. The last row reports the Holm-adjusted two-sided p-value of the partial coefficient from a Freedman–Lane residual permutation test with 20,000 permutations (seed 20260718), permutations running within each size group in the pooled column. In the two hundred-factory groups the two gaps correlate at 0.99 or above, so barely one per cent of the rank variance of the terminal gap survives conditioning and the partial coefficient in those columns rests on very little, which its wide interval records.

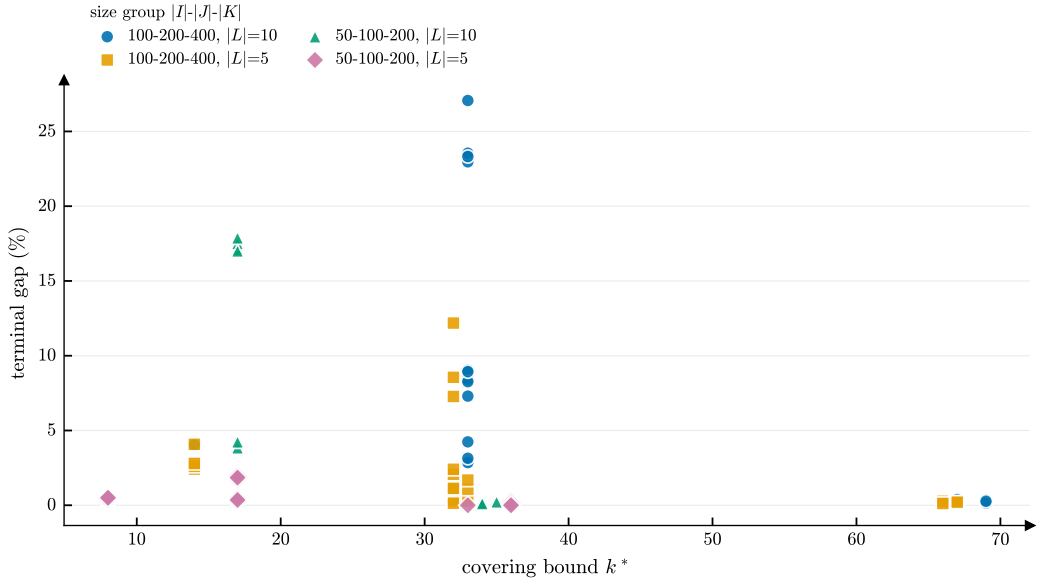
	50-5	50-10	100-5	100-10	pooled
$\rho(k^*,\ \text{gap})$	-0.59	-0.39	-0.70	-0.40	-0.28
	$[-0.81, -0.22]$	$[-0.69, +0.09]$	$[-0.86, -0.45]$	$[-0.75, +0.12]$	$[-0.47, -0.06]$
$\rho(k^*,\ \text{root gap})$	-0.65	-0.61	-0.73	-0.42	-0.36
	$[-0.84, -0.32]$	$[-0.83, -0.25]$	$[-0.87, -0.48]$	$[-0.75, +0.10]$	$[-0.55, -0.15]$
$\rho(\text{gap},\ \text{root gap})$	$+0.89$	$+0.91$	$+0.99$	$+1.00$	$+0.97$
	$[+0.68, +0.98]$	$[+0.73, +0.98]$	$[+0.96, +1.00]$	$[+0.97, +1.00]$	$[+0.94, +0.98]$
$\rho(k^*,\ \text{gap} \mid \text{root gap})$	-0.04	$+0.52$	$+0.30$	$+0.20$	$+0.29$
	$[-0.35, +0.19]$	$[+0.28, +0.71]$	$[-0.34, +0.67]$	$[-0.04, +0.41]$	$[+0.12, +0.42]$
$\rho(k^*,\ \text{nodes})$	$+0.41$	$+0.22$	$+0.83$	$+0.79$	$+0.02$
	$[-0.10, +0.75]$	$[-0.38, +0.66]$	$[+0.60, +0.93]$	$[+0.53, +0.92]$	$[-0.21, +0.22]$
adjusted p (partial)	1.000	0.055	0.624	1.000	1.000

The bound behaves accordingly, correlating negatively with the root gap in every group, from -0.42 to -0.73 with -0.36 pooled, tight capacities going with tight relaxations. Once the root gap is held fixed the negative association with the terminal gap does not survive, and in no group and not in the pool does the partial coefficient stay distinguishably negative. The partial coefficient is the Pearson correlation of the residuals of the two rank-on-rank regressions on the root-gap ranks, and its p-value comes from a Freedman–Lane permutation of those residuals, run within each size group in the pooled column so that pooled inference does not draw on group composition. The partial is indistinguishable from zero in three

groups and positive at +0.52 in the fourth, whose Holm-adjusted p of 0.055 sits just above the threshold, so no partial coefficient of either sign survives the correction. The pooled partial is positive at +0.29 descriptively, but the two residual series rise together across the four groups, the hundred-factory groups carrying the larger residuals of both, and once permutations respect the groups the pooled value is not distinguishable from zero. The intervals of the table are percentile bootstrap intervals over instances and are not adjusted for multiplicity, and each instance was run once under automatic thread count and a wall-clock limit, so they quantify sampling variability across instances and not solver noise. The two hundred-factory groups deserve little weight on their own here, since their two gaps correlate at 0.99 or above and barely one per cent of the rank variance of the terminal gap survives conditioning, an instability their wide intervals record. The design of the campaign and the conditional coefficients therefore point the same way, and the second is a check on the first rather than the evidence for it.

The bound also correlates with the explored node count, strongly in the two hundred-factory groups, weakly in the two fifty-factory groups and not at all pooled. That direction opposes the one Q1 records, where larger bounds bought smaller searches, and the two counts are taken under different conditions. In Q1 every counted node belongs to a search that ran to completion, so the count measures the work the algorithm needed. Here almost every search is cut at an hour, so the count measures how many nodes the solver managed to process inside a fixed budget. A tight relaxation leaves the solver with small subproblems and a bound it seldom revises, and it processes many of them, so under censoring a positive correlation between the bound and the node count is a statement about throughput and carries no claim that those instances are harder. On this benchmark the covering bound annotates relaxation tightness rather than search difficulty, its direction stable across groups and its strength moderate within the five-product groups and weaker pooled. We make no causal claim beyond the controlled comparison.

Figure 11 Terminal gap of MIP under the 3,600 s limit against the covering bound k^*, one marker per benchmark instance, coloured by size group. Within every group larger bounds accompany smaller gaps, the association Table 6 quantifies. The within-group reading is what controls for instance size.



7. Conclusion

We gave a parameterized account of MP-TSCFLP, and the picture that emerges has two layers, a covering layer, structural and strong, that fires wherever a facility side tells the members of a dimension apart through a $\{0,1\}$ cost matrix, tight capacities supplying the fourth such pair, and a numeric layer whose hardness outside the fourth pair is weak and dissolves under a unary encoding. Table 3 and Figure 8 record the resulting

landscape entry by entry, from the size-parameter dichotomy through the bracketing of the parameter k to the kernelization and width results, with witnesses on both sides of every border of the binary-encoding map.

Five questions remain, and we regard them as the natural continuation of this work. The first asks for a greedy $\ln|K|$ upper bound matching the inapproximability of the covering layer, the constructive half that would close the approximability of that layer from above and below, with the non-metric uncapacitated median problem [10] as the precedent. A parameterized companion attaches to the same layer. The reduction behind Theorem 1, run with the enlarged amplifier of Corollary 1, maps the minimum cover size onto the optimum value while shifting the parameter by one, so a constant-factor FPT approximation in k for MP-TSCFLP would hand back one for Set Cover parameterized by the cover size, which is itself W[2]-hard to approximate within any constant [23], and we conjecture that no such approximation exists. The second concerns membership of the parameter k in W[2], since our sandwich leaves the exact class as the one open classification of the project, and the barrier of binary arithmetic and flow minimization inside the verifier against weft two is of interest beyond this model. The third would settle the unary status of $\{|J|, |L|\}$ and of the two triples $\{|I|, |K|, |L|\}$ and $\{|J|, |K|, |L|\}$, the only gaps of the sixteen-by-two dichotomy table, where the witnessing slices go polynomial under unary data and no other hard slice is yet known. The fourth is whether a polynomial kernel exists in the full parameter $|I| + |J| + |K| + |L|$ with general numbers, the exact point where the yes-certificate comparison turns bilinear and both weight families would have to be reduced at once, a genuine methodological frontier of weighted kernelization. The fifth asks whether transport costs structured beyond separability, uniform per side, metric or of low rank, fall on the tractable or the hard side, refining the frontier over the cost classes that appear in real data. Every hardness construction in the paper uses transport costs without metric structure, so the map may bind differently on distance-based data. Progress on any one of the five would redraw a border of the landscape assembled here.

For practice the reading is brief and concrete. The a priori bound k^* is computed by a sort and a prefix scan of the capacities per product and side, and by nothing costlier, lower-bounds the facilities any feasible design must open, and is the threshold on which our branch-and-bound prunes when the cardinality bound binds. The completed campaign tests it against the terminal gaps of a mixed-integer solver on the full benchmark and finds a direction a naive reading would not guess, for larger bounds accompany smaller terminal gaps, in an association that is negative in every size group and survives multiplicity correction pooled and in the five-product groups. The log analysis places that association at the root relaxation rather than at the search, since almost every run ends at the time limit with its bound close to the root value, so on this benchmark the bound is best read as an annotation of relaxation tightness and not as a forecast of search effort.

Acknowledgments

This research was financed in part by the Conselho Nacional de Desenvolvimento Científico e Tecnológico - CNPq (304043/2025-7), Fundação Carlos Chagas Filho de Amparo à Pesquisa do Estado do Rio de Janeiro - FAPERJ (E-26/204.211/2025, E-26/202.365/2025) and Federal University of Rio de Janeiro - UFRJ (23079.227622/2023-70).

References

[1] A. Klose, A. Drexl, Facility location models for distribution system design, European Journal of Operational Research 162 (1) (2005) 4–29.

[2] C. Ortiz-Astorquiza, I. Contreras, G. Laporte, Multi-level facility location problems, European Journal of Operational Research 267 (3) (2018) 791–805.

[3] H. Pirkul, V. Jayaraman, A multi-commodity, multi-plant, capacitated facility location problem: formulation and efficient heuristic solution, Computers & Operations Research 25 (10) (1998) 869–878.

[4] G. R. Mauri, F. L. Biajoli, R. L. Rabello, A. A. Chaves, G. M. Ribeiro, L. A. N. Lorena, Hybrid metaheuristics to solve a multiproduct two-stage capacitated facility location problem, International Transactions in Operational Research 28 (2021) 3069–3093, doi:10.1111/itor.12930.

[5] I. Morais, G. Souto, G. M. Ribeiro, I. Mendonça, P. H. González, A hybrid BRKGA approach for the multiproduct two stage capacitated facility location problem, in: 2022 IEEE Congress on Evolutionary Computation (CEC), IEEE, doi:10.1109/CEC55065.2022.9870321, 2022.

[6] M. R. Fellows, H. Fernau, Facility location problems: A parameterized view, Discrete Applied Mathematics 159 (2011) 1118–1130.

[7] I. Litvinchev, E. L. Ozuna Espinosa, Lagrangian bounds and a heuristic for the two-stage capacitated facility location problem, International Journal of Energy Optimization and Engineering 1 (1) (2012) 59–71.
[8] D. R. M. Fernandes, C. T. M. Rocha, D. J. Aloise, G. M. Ribeiro, E. M. Santos, A. Silva, A simple and effective genetic algorithm for the two-stage capacitated facility location problem, Computers & Industrial Engineering 75 (2014) 200–208.
[9] M. Fischetti, I. Ljubić, M. Sinnl, Redesigning Benders decomposition for large-scale facility location, Management Science 63 (7) (2017) 2146–2162.
[10] D. S. Hochbaum, Heuristics for the fixed cost median problem, Mathematical Programming 22 (1) (1982) 148–162.
[11] U. Feige, A threshold of $\ln n$ for approximating set cover, Journal of the ACM 45 (4) (1998) 634–652.
[12] I. Dinur, D. Steurer, Analytical approach to parallel repetition, in: Proceedings of the 46th Annual ACM Symposium on Theory of Computing (STOC), ACM, 624–633, 2014.
[13] R. M. Karp, Reducibility among combinatorial problems, in: Complexity of Computer Computations, Plenum Press, 85–103, 1972.
[14] M. R. Garey, D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman, 1979.
[15] R. G. Downey, M. R. Fellows, Fundamentals of Parameterized Complexity, Springer, 2013.
[16] M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, S. Saurabh, Parameterized Algorithms, Springer, 2015.
[17] J. Chen, X. Huang, I. A. Kanj, G. Xia, Strong computational lower bounds via parameterized complexity, Journal of Computer and System Sciences 72 (8) (2006) 1346–1367.
[18] A. Frank, É. Tardos, An application of simultaneous Diophantine approximation in combinatorial optimization, Combinatorica 7 (1) (1987) 49–65.
[19] V. Cohen-Addad, A. Gupta, A. Kumar, E. Lee, J. Li, Tight FPT approximations for k-median and k-means, in: 46th International Colloquium on Automata, Languages, and Programming (ICALP), vol. 132 of *LIPIcs*, 42:1–42:14, 2019.
[20] V. Cohen-Addad, J. Li, On the fixed-parameter tractability of capacitated clustering, in: 46th International Colloquium on Automata, Languages, and Programming (ICALP), vol. 132 of *LIPIcs*, 41:1–41:14, 2019.
[21] A. E. Feldmann, A. Mukherjee, E. J. van Leeuwen, The parameterized complexity of the survivable network design problem, Journal of Computer and System Sciences 148 (2025) 103604.
[22] F. V. Fomin, V. Ramamoorthi, On the parameterized complexity of the expected coverage problem, Theory of Computing Systems 66 (2) (2022) 432–453.
[23] B. Lin, X. Ren, Y. Sun, X. Wang, Constant approximating parameterized k-SetCover is W[2]-hard, in: Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, doi:10.1137/1.9781611977554.ch126, 2023.
[24] M. Dantas da Silva, F. Protti, U. S. Souza, Revisiting the complexity of and/or graph solution, Journal of Computer and System Sciences 79 (7) (2013) 1156–1163.
[25] J. B. Orlin, A faster strongly polynomial minimum cost flow algorithm, Operations Research 41 (2) (1993) 338–350.
[26] R. Impagliazzo, R. Paturi, F. Zane, Which problems have strongly exponential complexity?, Journal of Computer and System Sciences 63 (4) (2001) 512–530.
[27] H. L. Bodlaender, B. M. P. Jansen, S. Kratsch, Kernelization lower bounds by cross-composition, SIAM Journal on Discrete Mathematics 28 (1) (2014) 277–305.
[28] N. Robertson, P. D. Seymour, Graph minors. II. Algorithmic aspects of tree-width, Journal of Algorithms 7 (3) (1986) 309–322.
[29] B. Courcelle, J. A. Makowsky, U. Rotics, Linear time solvable optimization problems on graphs of bounded clique-width, Theory of Computing Systems 33 (2) (2000) 125–150.
[30] A. Schrijver, Theory of Linear and Integer Programming, John Wiley & Sons, Chichester, 1986.
[31] M. Etscheid, S. Kratsch, M. Mnich, H. Röglin, Polynomial kernels for weighted problems, Journal of Computer and System Sciences 84 (2017) 1–10.

A. Proofs of the structural propositions

This appendix collects the full proofs of the three structural propositions of Section 3, which rest on classical network-flow and transportation techniques. The body keeps their statements, the layered-network construction they build on, and the facts that the later sections consume, and the proofs appear here in statement order.

Proposition 1 (routing oracle). *Fix $(y, z) \in \{0,1\}^{|I|} \times \{0,1\}^{|J|}$. The residual routing problem separates over products, and for each product l its optimal cost equals the minimum cost μ_l of an S–T flow of value D_l in the layered network $N_l(y, z)$ of Section 3, with $\mu_l = +\infty$ when no such flow exists. The finite μ_l are attained by integral flows and are nonnegative integers, and $v(y, z) = \sum_l \mu_l$. Hence $v(y, z)$ is computable in strongly polynomial time by $|L|$ minimum-cost flow computations [25].*

Proof of Proposition 1. We argue through the correspondence between routings and flows. The routing splits into independent per-product blocks, each block optimum is identified with the minimum cost of a value-D_l

flow in the network $N_l(y,z)$ constructed in Section 3, and the claims of the statement all follow from that identification.

Separation. Each of (2)–(5) constrains a single product l, and the objective is a sum of per-product terms, so the feasible set of routings is the product of the per-product feasible sets and the objective is separable. The minimum is thus the sum over l of the per-product minima, and the design is feasible if and only if every product admits a routing. In particular, if some product admits no routing, then its per-product minimum is $+\infty$, the correspondence established below yields $\mu_l = +\infty$ for that product, and both $v(y,z)$ and $\sum_l \mu_l$ equal $+\infty$, so the claimed identity holds under the stated convention. The correspondence itself needs no finiteness assumption, so we now fix a product l and establish it in general.

Routings and flows coincide. We show the block optimum equals μ_l. Take a feasible routing (x,w) of block l. First lower the flows into each over-served customer until $\sum_j w_{jkl} = q_{kl}$, then lower the flows into each depot until $\sum_i x_{ijl} = \sum_k w_{jkl}$. Which coordinates are lowered is immaterial, and each stopping rule halts exactly at equality, so (2) and (3) hold with equality afterwards. Feasibility of the routing persists throughout the lowering. Each lowering decreases only the tracked left-hand side, (2) for w and (3) for x, which stops exactly at its target, together with quantities whose decrease merely slackens (3), (4) and (5). No variable becomes negative, since each lowering drives a nonnegative total down to a nonnegative target. Each lowering removes flow from arcs of nonnegative cost, so the cost does not rise. By (4) at $y_i = 0$ and (5) at $z_j = 0$, together with nonnegativity, every x_{ijl} with $y_i = 0$ and every w_{jkl} with $z_j = 0$ vanishes, so restricting attention to the open facilities discards no flow and no cost. Setting $f(S \to i) = \sum_j x_{ijl}$, $f(i \to j_{\mathrm{in}}) = x_{ijl}$, the split-arc flow $= \sum_i x_{ijl}$, $f(j_{\mathrm{out}} \to k) = w_{jkl}$ and $f(k \to T) = q_{kl}$ then conserves flow at every node and respects every capacity. The result is an S–T flow whose value, the total inflow $\sum_k f(k \to T)$ at the sink, equals $\sum_k q_{kl} = D_l$, and its cost is that of the lowered routing, hence at most that of (x,w).

Conversely, given an S–T flow f of value D_l, set $x_{ijl} = f(i \to j_{\mathrm{in}})$ and $w_{jkl} = f(j_{\mathrm{out}} \to k)$ at the open facilities, and set $x_{ijl} = 0$ whenever $y_i = 0$ and $w_{jkl} = 0$ whenever $z_j = 0$. At the closed facilities (4) and (5) read $0 \le 0$ and hold. Value D_l saturates every arc $k \to T$. Indeed the value equals $\sum_k f(k \to T) \le \sum_k q_{kl} = D_l$, so the two sums are equal, and since each term obeys $f(k \to T) \le q_{kl}$ every term is at capacity. Conservation at k then gives $\sum_j w_{jkl} = q_{kl}$, which is (2), conservation at the split node gives $\sum_i x_{ijl} = \sum_k w_{jkl}$, which is (3), and the split- and source-arc capacities give (5) and (4) at the open facilities. The routing so obtained is therefore feasible, and its cost equals that of the flow because the costs agree arc by arc. Assembling the two directions, every feasible routing of block l yields a value-D_l flow of no greater cost, so μ_l is at most the block optimum, and every value-D_l flow yields a feasible routing of equal cost, so the block optimum is at most μ_l. The two inequalities together give the equality, and a value-D_l flow exists if and only if the block admits a routing, so $\mu_l = +\infty$ exactly when the block is infeasible.

Integrality and complexity. The node–arc incidence matrix of a flow network is totally unimodular [30], and all finite arc capacities b_{il}, p_{jl}, q_{kl} are integers. Every arc of $N_l(y,z)$ belongs to one layer, and each layer is an S–T cut that every path crosses exactly once and only forwards, so the arc flows within a layer sum to the flow value. Since the flows are nonnegative, every arc in a flow of value D_l therefore carries at most D_l, and the polytope of value-D_l flows is bounded. A finite μ_l is thus attained at a vertex of that polytope, since a linear function on a nonempty bounded polytope attains its minimum at a vertex [30]. The vertex is integral by the Hoffman–Kruskal theorem [30], and the nonnegative integer arc costs make μ_l a nonnegative integer. Each N_l has $O(n + |K|)$ nodes and $O(|I||J| + |J||K| + n + |K|)$ arcs, and a minimum-cost flow of prescribed value is computed in strongly polynomial time [25]. Summing over the $|L|$ products gives $v(y,z) = \sum_l \mu_l$ within the stated bound. □

Proposition 2 (feasibility). *A design (y,z) is feasible if and only if, for every product l, $\sum_{i \in I} b_{il}\, y_i \ge D_l$ (F1) and $\sum_{j \in J} p_{jl}\, z_j \ge D_l$ (F2). The test runs in $O((n + |K|)\,|L|)$ time, of which $O(|K|\,|L|)$ computes the totals D_l.*

Proof of Proposition 2. Necessity will come from aggregating the four constraint groups, and sufficiency from exhibiting a flow of value D_l in the layered network $N_l(y,z)$. For necessity fix a feasible routing and a product l. Summing (2) over k gives $\sum_{j,k} w_{jkl} \ge D_l$, and summing (5) over j gives $\sum_{j,k} w_{jkl} \le \sum_j p_{jl} z_j$, so (F2) holds. Summing (3) over j gives $\sum_{i,j} x_{ijl} \ge \sum_{j,k} w_{jkl} \ge D_l$, and summing (4) over i gives $\sum_{i,j} x_{ijl} \le \sum_i b_{il} y_i$, so (F1) holds.

For sufficiency we route each product l independently and exhibit a flow of value D_l in $N_l(y,z)$. By the flow-to-routing direction of the correspondence established for Proposition 1, any flow of value exactly D_l in $N_l(y,z)$ reads back as a feasible routing of block l, so such flows for all products assemble into a feasible routing for the design. The finite-capacity arcs of $N_l(y,z)$ fall into three layers, the source arcs $S\to i$ with total capacity $\sum_i b_{il}y_i$, the depot split arcs with total $\sum_j p_{jl}z_j$, and the demand arcs $k\to T$ with total D_l. Every S–T path crosses each layer exactly once, so each layer is an S–T cut. Since the value of any flow is at most the capacity of any S–T cut [30], the maximum flow is at most each of the three totals. We claim it attains the least total, and since (F1) and (F2) make that least total equal to D_l, a flow of value D_l then exists.

Let F be the least of the three totals. Because F does not exceed any layer total, one can choose factory loads $a_i \le b_{il}y_i$, depot loads $e_j \le p_{jl}z_j$ and customer intakes $r_k \le q_{kl}$ each summing to F, for instance by scanning the terms of each layer in a fixed order and truncating the running sum once it reaches F. Both bipartite stages are complete and uncapacitated, so for any two nonnegative load vectors with equal totals there is a transfer between them that meets those loads as its row and column sums. One such transfer, the northwest-corner procedure of transportation theory, maintains a current supplier and a current receiver, assigns them the minimum of the remaining supply and the remaining demand, and advances past whichever of the two is exhausted. Every step reduces the residual supply total and the residual demand total by the same amount and exhausts a supplier or a receiver, so the procedure terminates, and since the two totals stay equal it ends with every supply and every demand met exactly, which is the required row and column sums. Route a to e across the first stage and e to r across the second by such transfers, and put a_i on the source arc $S\to i$ and r_k on the demand arc $k\to T$. Flow is conserved at each factory node because the first transfer has row sums a, and at each customer node because the second transfer has column sums r. Each depot's inflow and outflow both equal $e_j \le p_{jl}z_j$, so flow is conserved at the split arc and its capacity is respected. The source and demand arcs carry $a_i \le b_{il}y_i$ and $r_k \le q_{kl}$, within their capacities, and the result is an S–T flow of value $\sum_k r_k = F$. Hence the maximum flow equals F, and under (F1) and (F2) the least of the three totals is D_l, so a flow of value D_l exists and the design is feasible. Finally we bound the running time. One pass over the demand matrix computes every total D_l in $O(|K|\,|L|)$ time, after which the test evaluates $2|L|$ sums of at most n terms each, for $O((n+|K|)\,|L|)$ in total. □

Proposition 3 (monotonicity). *If $(y,z)\le(y',z')$ componentwise then feasibility of (y,z) implies feasibility of (y',z'), and $v(y',z')\le v(y,z)$ always, with the convention $v=+\infty$ on infeasible designs. In particular the all-open design attains the least routing value among all feasible designs.*

Proof of Proposition 3. Since the data satisfy $b_{il}\ge 0$ and $p_{jl}\ge 0$, the left-hand sides $\sum_i b_{il}y_i$ and $\sum_j p_{jl}z_j$ of (F1) and (F2) are nondecreasing in (y,z). If (y,z) is feasible, the necessity direction of Proposition 2 gives (F1) and (F2) at (y,z), hence at (y',z'), and the sufficiency direction then gives feasibility of (y',z'). If (y,z) is infeasible then $v(y,z)=+\infty$ and the inequality is trivial, so assume (y,z) feasible, in which case (y',z') is feasible too. Since $(y,z)\le(y',z')$ componentwise, every node and arc of $N_l(y,z)$ appears in $N_l(y',z')$ with the same capacity and cost, so $N_l(y,z)$ is a subnetwork of $N_l(y',z')$ and the passage from one to the other only adds the nodes and arcs of newly opened facilities. Every S–T flow of value D_l in $N_l(y,z)$ is therefore still one in $N_l(y',z')$, carrying zero on the new arcs, at the same cost. The feasible set of value-D_l flows only grows and its minimum cost cannot increase, so $\mu_l(y',z')\le\mu_l(y,z)$ for every l, where $\mu_l(\cdot,\cdot)$ denotes the block optimum of Proposition 1 viewed as a function of the design. Since $v=\sum_l\mu_l$ by that proposition, summing over the products gives $v(y',z')\le v(y,z)$. For the final claim, every design satisfies $(y,z)\le(\mathbf{1},\mathbf{1})$ componentwise because the variables are binary. The feasibility part then makes the all-open design feasible whenever some feasible design exists, and the value part gives $v(\mathbf{1},\mathbf{1})\le v(y,z)$ for every feasible (y,z), so the all-open design attains the least routing value among feasible designs, a claim vacuous when none exists. □

B. Technical verifications

This appendix collects the proofs whose methods are standard, invariant tracking for the correctness of the branch-and-bound, linear-programming duality for the certificates, exchange arguments for the two preprocessing reductions, dynamic programming for the separable subclass, and the weight-reduction technique

of Frank and Tardos [18] for the compression. The statements remain in the body, each with a paragraph naming the route of its proof, and the proofs appear here grouped by result, one subsection each, so that a reference from the body lands directly on the argument it names.

B.1. The separable first stage

Theorem 7 (separable first stage). *On the group $|K| = |L| = 1$, if the first-stage matrix is separable, that is $c_{ij} = \gamma_i + \delta_j$ for integers γ, δ with the costs nonnegative, then the problem is solvable in pseudo-polynomial time $O\big(|I||J| + (|I| + |J|) \cdot D\big)$. Separability is decidable in $O(|I||J|)$ time.*

Proof of Theorem 7. The argument reduces the routing cost to a function of its marginals, and then each facility layer to a knapsack-cover program. With $|K| = |L| = 1$ there is one customer and one product. Write $D = q_{11}$, w_j for the flow depot j forwards to the customer, and $d_j = d_{j11}$ for its unit second-stage cost. Fix a feasible design (Y, Z). In an optimal solution we may assume that (2) and (3) are tight, since lowering flow on the nonnegative-cost arcs cannot raise the cost. The lowering argument is the one in the proof of Proposition 1 in A. Under this assumption the depot marginal $e_j = \sum_i x_{ij} = w_j$ is both the inflow and the outflow of depot j, the factory marginal $a_i = \sum_j x_{ij}$ is the load of factory i, and $\sum_i a_i = \sum_j e_j = D$, with $0 \le a_i \le b_i$ ($a_i = 0$ for $i \notin Y$) and $0 \le e_j \le p_j$ ($e_j = 0$ for $j \notin Z$). The total routing cost is

$$\sum_{ij} c_{ij} x_{ij} + \sum_j d_j w_j = \sum_{ij} (\gamma_i + \delta_j) x_{ij} + \sum_j d_j e_j = \sum_i \gamma_i a_i + \sum_j (\delta_j + d_j)\, e_j,$$

using $\sum_i x_{ij} = e_j$ and $\sum_j x_{ij} = a_i$. The second-stage cost is thus a per-unit depot cost d_j folded into the depot term. The cost depends on the transport plan only through the marginals, and on the complete uncapacitated bipartite stage any nonnegative a and e with $\sum_i a_i = \sum_j e_j$ are realized by some plan, for instance the northwest-corner procedure used in the proof of Proposition 2. A load $a_i > 0$ forces $y_i = 1$ by (4) and a load $e_j > 0$ forces $z_j = 1$ by (5), while an open facility with zero load may be closed at no extra cost since $f, g \ge 0$, so we may identify the open facilities with the supports of the loads. Conversely any nonnegative loads within the capacities and summing to D on each side define a feasible solution that opens exactly the supports. Adding the fixed charges, the optimum therefore splits into two independent problems, one per layer, namely

$$\min_a \Big(\sum_{i : a_i > 0} f_i + \sum_i \gamma_i a_i \Big) \quad \text{and} \quad \min_e \Big(\sum_{j : e_j > 0} g_j + \sum_j (\delta_j + d_j) e_j \Big),$$

each over nonnegative loads bounded by the capacities and summing to D. The two split problems range over real loads while the dynamic program below ranges over integer ones. For a fixed support the fixed charges are constant and each problem becomes a linear program over a box intersected with a single equality, a polytope whose vertices are integral for integer capacities and integer D, so the real and the integer optima coincide and the restriction to integer loads loses nothing. Solve the factory problem by the dynamic program

$$V(i, s) = \min \Big\{ V(i-1, s),\ \min_{1 \le a \le b_i} V(i-1, s-a) + f_i + \gamma_i a \Big\},$$

with $V(0, 0) = 0$, with $V(0, s) = +\infty$ for $s > 0$, with the convention $V(i-1, s-a) = +\infty$ when $s < a$, and with s ranging over $0, \dots, D$. The first branch closes factory i and the second opens it with load a. The optimum of the factory problem is $V(|I|, D)$, the value $+\infty$ signalling that no factory loads reach D within the capacities, in which case the instance is infeasible. Substituting $t = s - a$ linearizes the inner minimum, since

$$\min_{1 \le a \le b_i} V(i-1, s-a) + f_i + \gamma_i a = f_i + \gamma_i s + \min_{\max(0, s-b_i) \le t \le s-1} \big(V(i-1, t) - \gamma_i t \big).$$

As s runs from 0 to D the window of admissible t slides monotonically to the right, so a monotone double-ended queue over the keys $V(i-1, t) - \gamma_i t$ returns each window minimum in $O(1)$ amortized time, every index entering and leaving the queue once per row. The device works whatever the sign of γ_i, and the infinite entries are skipped since they can never attain a finite minimum. Each row therefore costs $O(D)$ and the factory

table $O(|I| \cdot D)$ time. The depot problem is the same recurrence with g_j, $\delta_j + d_j$ and p_j, in $O(|J| \cdot D)$ time, its value $+\infty$ likewise signalling infeasibility. Their sum is the instance optimum, within the stated bound. The matrix is separable exactly when $c_{ij} + c_{11} = c_{i1} + c_{1j}$ for all i, j, the condition being necessary by substituting $c_{ij} = \gamma_i + \delta_j$, and sufficient because then $\gamma_i = c_{i1} - c_{11}$ and $\delta_j = c_{1j}$ satisfy $\gamma_i + \delta_j = c_{i1} + c_{1j} - c_{11} = c_{ij}$, and it is checked in $O(|I||J|)$ time. The $O(|I||J|)$ term of the stated running time reads the matrix, checks separability and recovers γ and δ. □

B.2. Correctness of the branch-and-bound

Proposition 6 (correctness). *Algorithm 1 decides* MP-TSCFLP(B, k)*, and when run with* $B = +\infty$ *it returns* best $= \mathrm{OPT}_k$.

Proof of Proposition 6. The proof runs in three steps, justifying the early return of line 3, bounding best from below by soundness, and bounding it from above by tracking one distinguished optimum through the search.

The early return. Line 3 is the root instance of P1. If some feasible design O opens at most k facilities, then for every product l the factories of O satisfy (F1), so Lemma 1 gives $k_I^*(l) \le |O \cap I|$, and likewise $k_J^*(l) \le |O \cap J|$, whence $\max_l k_I^*(l) + \max_l k_J^*(l) \le |O| \le k$ and the return does not fire. When it fires, therefore, no feasible design opens at most k facilities, so $\mathrm{OPT}_k = +\infty$ and the answer NO is correct.

Soundness. best is assigned only at a leaf, to $LB(O, C) = \mathrm{cost}(O)$. Such a leaf survived P2, so $v^{\uparrow} = v(O \cup \emptyset) = v(O) < +\infty$ and O is feasible, and it survived the cardinality test of line 6, so $r = k - |O| \ge 0$ and $|O| \le k$. Hence best $\ge \mathrm{OPT}_k$ throughout, where $\mathrm{OPT}_k = +\infty$ if no feasible design opens at most k facilities.

Completeness under $\mathrm{OPT}_k \le B$. Suppose $\mathrm{OPT}_k \le B$ and fix O^*, a feasible design with $|O^*| \le k$ and $\mathrm{cost}(O^*) = \mathrm{OPT}_k$, of least cardinality among such. Call a stack node (O, C) *live* if $O \subseteq O^*$ and $O^* \cap C = \emptyset$, in which case $O^* \subseteq O \cup F$, since O^* avoids C and every facility lies in $O \cup C \cup F$. A step of the search is one iteration of the while loop, and we show by induction over steps that either best $= \mathrm{OPT}_k$ or the stack holds a live node. Before the first step the stack holds the root, which is live. Once best $= \mathrm{OPT}_k$ holds it holds forever, since best never increases and soundness bounds it below by OPT_k. When a step pops a node that is not live while a live node is on the stack, the live node stays there untouched and the invariant survives, so assume the popped node (O, C) is live and best $> \mathrm{OPT}_k$.

We first check that no test removes the live node. It has $|O| \le |O^*| \le k$, so $r \ge 0$ and line 6 keeps it. Beyond O the design O^* opens only free facilities, and for every product l the added factories $(O^* \setminus O) \cap I$ supply the residual demand of (F1), so Lemma 1 gives $s_I(l; O, C) \le |(O^* \setminus O) \cap I|$ and likewise $s_J(l; O, C) \le |(O^* \setminus O) \cap J|$. Hence

$$\max_l s_I(l; O, C) + \max_l s_J(l; O, C) \;\le\; |O^* \setminus O| \;=\; |O^*| - |O| \;\le\; k - |O| \;=\; r,$$

and P1 spares the node. Monotonicity along $O^* \subseteq O \cup F$ gives $v^{\uparrow} = v(O \cup F) \le v(O^*) < +\infty$ (Proposition 3), so the infeasibility branch of P2 does not fire, and O^* is a completion of (O, C), so $LB(O, C) \le \mathrm{cost}(O^*) = \mathrm{OPT}_k <$ best, which with $\mathrm{OPT}_k \le B$ gives $LB(O, C) < \min(\text{best}, B + 1)$ and the bound branch of P2 spares it as well.

If the live node is internal it is expanded on a free facility e, and the child that records the status of e in O^*, namely $(O \cup \{e\}, C)$ when $e \in O^*$ and $(O, C \cup \{e\})$ otherwise, is live, so the invariant survives. If the live node is a leaf then $F = \emptyset$, so $O^* \subseteq O \cup F = O$, and $O \subseteq O^*$ forces $O = O^*$. The oracle solution returned at the leaf is an optimal flow of O^* and leaves no open facility unused, since closing an unused facility would give a feasible design opening at most k facilities at cost at most OPT_k, hence an optimum of smaller cardinality than O^*, against its choice. P3 therefore does not fire, and the leaf sets best to $LB(O^*, C) = \mathrm{cost}(O^*) = \mathrm{OPT}_k$, after which the invariant persists in its first form. Each child carries one fewer free facility than its parent, so the search tree has depth at most n and finitely many nodes, and the loop terminates. At termination the stack is empty, so the invariant leaves only best $= \mathrm{OPT}_k$.

Conclusion. If $\mathrm{OPT}_k \le B$ then completeness gives best $= \mathrm{OPT}_k \le B$, a finite value since the budget B of the decision problem is a nonnegative integer, so both tests of line 14 pass and the algorithm returns YES. If $\mathrm{OPT}_k > B$ then best $\ge \mathrm{OPT}_k > B$ by soundness, one of the two tests fails whether best is finite or not,

and the algorithm returns NO, and the answer NO of line 3, when that return fires instead, is correct by the early-return argument. Either way the algorithm decides MP-TSCFLP(B, k). Running with $B = +\infty$ makes $\mathrm{OPT}_k \le B$ hold whenever OPT_k is finite, so completeness gives best $= \mathrm{OPT}_k$ in the finite case. When $\mathrm{OPT}_k = +\infty$ soundness keeps best unassigned, so it retains its initial value $+\infty = \mathrm{OPT}_k$, the finiteness test of line 14 fails and the label is NO, and when the early return fires instead line 3 reports the same infinite optimum. On either branch of line 14 the reported best equals OPT_k, so the run with $B = +\infty$ reads the optimum off every exit. □

B.3. Routing certificates and Benders cuts

Proposition 7 (routing certificate and Benders cuts). *Fix a product l and let Δ_l be the dual polyhedron of its transportation block, which is independent of (y, z). If the block is feasible then some $u^* \in \Delta_l$ attains the dual value $\mu_l(y, z)$, and the pair of an optimal flow and u^* certifies $\mu_l(y, z)$ in time $O(|I||J| + |J||K|)$. Further, writing θ_l for a variable through which a master problem represents the block value, for every $u = (\alpha, \beta, \sigma, \pi) \in \Delta_l$ the inequality*

$$\theta_l \;\ge\; \sum_k q_{kl}\,\alpha_k - \sum_i b_{il}\,\sigma_i\,y_i - \sum_j p_{jl}\,\pi_j\,z_j$$

is valid for every design whose block l is feasible, and it holds with equality at the design that generated u^*. Two explicit recession rays of Δ_l certify infeasibility, and their objectives are positive exactly when (F1), respectively (F2), of Proposition 2 fails.

Proof of Proposition 7. The block dual, once written explicitly, yields the certificate, the cuts and the rays. Write the block primal as the minimization of $\sum_{ij} c_{ijl}x_{ijl} + \sum_{jk} d_{jkl}w_{jkl}$ over nonnegative flows subject to the demand rows (2), the balance rows (3), oriented as $\sum_i x_{ijl} - \sum_k w_{jkl} \ge 0$, the factory rows (4) with right-hand side $b_{il}y_i$, and the depot rows (5) with right-hand side $p_{jl}z_j$. The trimming step in the proof of Proposition 1, given in A, lowers flows at no cost increase until (3) is tight, so replacing (3) by the equality $\sum_i x_{ijl} = \sum_k w_{jkl}$ changes neither the feasibility nor the optimal value of the block, and we dualize it in that equality form, which is what licenses a free multiplier on the balance rows. Assign dual multipliers $\alpha_k \ge 0$ to the demand rows, a free β_j to the balance equalities, and $\sigma_i \ge 0$ and $\pi_j \ge 0$ to the two capacity groups. The dual constraints, one per primal variable, read $\beta_j - \sigma_i \le c_{ijl}$ for x_{ijl} and $\alpha_k - \beta_j - \pi_j \le d_{jkl}$ for w_{jkl}, and none of them mentions the design, so the dual feasible set Δ_l is indeed independent of (y, z). The dual objective is $\sum_k q_{kl}\alpha_k - \sum_i b_{il}y_i\sigma_i - \sum_j p_{jl}z_j\pi_j$, so weak duality gives, for every $u \in \Delta_l$, the displayed lower bound on the block value θ_l, valid across all designs whose block is feasible precisely because Δ_l carries no design. When the block is feasible its primal is bounded below by 0 through the nonnegative costs, so the finite optimum makes linear-programming strong duality supply a $u^* \in \Delta_l$ attaining $\mu_l(y, z)$, giving equality there. Checking a certificate pair is verifying primal and dual feasibility and equal objectives, which certifies optimality by weak duality, in one pass over the $O(|I||J| + |J||K|)$ coefficients. When the block is infeasible the primal is empty, and Δ_l contains the point $u = 0$ by nonnegativity of the costs c and d, so the dual is feasible and therefore unbounded along a recession ray of Δ_l, whose cone is $\beta_j - \sigma_i \le 0$, $\alpha_k - \beta_j - \pi_j \le 0$ with $\alpha, \sigma, \pi \ge 0$. Two rays realize the feasibility cuts. The ray $\alpha \equiv 1$, $\beta \equiv 1$, $\sigma \equiv 1$, $\pi \equiv 0$ lies in the cone, since its constraint values are $\beta_j - \sigma_i = 1 - 1 = 0 \le 0$ and $\alpha_k - \beta_j - \pi_j = 1 - 1 - 0 = 0 \le 0$, and it has objective $\sum_k q_{kl} - \sum_i b_{il}y_i = D_l - \sum_i b_{il}y_i$, positive exactly when (F1) fails. The ray $\alpha \equiv 1$, $\beta \equiv 0$, $\sigma \equiv 0$, $\pi \equiv 1$ lies in the cone through $\beta_j - \sigma_i = 0 \le 0$ and $\alpha_k - \beta_j - \pi_j = 1 - 0 - 1 = 0 \le 0$, and has objective $D_l - \sum_j p_{jl}z_j$, positive exactly when (F2) fails. For the concluding equivalence note that block l is feasible exactly when (F1) and (F2) hold for l, since both the aggregation argument for necessity and the flow construction for sufficiency in the proof of Proposition 2 treat each product on its own. A design's block is therefore infeasible if and only if (F1) or (F2) fails for l, which happens if and only if one of these two rays has positive objective, recovering the aggregate test of Proposition 2. □

B.4. Customer aggregation

Proposition 8 (customer aggregation). *If customers $k \ne k'$ have identical cost columns, that is $d_{jkl} = d_{jk'l}$ for all j, l, then merging k' into k and adding their demands preserves feasibility, routing value and cost of every design, hence the optimal value, the set of optimal designs and the answers of both decision versions.*

Feasible complete solutions map in both directions under the explicit merge and split maps of the proof, which never raise the cost and preserve the optimal routing value of every design, and consequently one may assume $|K| \le \tau$, where τ is the number of distinct second-stage cost columns, computable in polynomial time.

Proof of Proposition 8. Fix a design and a product l. The first stage and the fixed charges are untouched, so only the second-stage cost of serving k and k' against the merged customer $\hat{k}$ with demand $q_{kl} + q_{k'l}$ is at issue, and the two customers share the cost column $d_{j\bullet l}$. Given a routing serving both, the merge map sets $w_{j\hat{k}l} = w_{jkl} + w_{jk'l}$. This serves the merged demand, leaves every depot total unchanged, so (3) and (5) still hold, and costs $\sum_j d_{j\bullet l}(w_{jkl} + w_{jk'l})$, exactly the original cost. Conversely, given a routing serving $\hat{k}$, trim any surplus so that $\sum_j w_{j\hat{k}l} = q_{kl} + q_{k'l}$, which only slackens (3) and (5) and, by nonnegativity of d, does not raise the cost, then apply the split map, a northwest-corner expansion of each $w_{j\hat{k}l}$ into $w_{jkl} + w_{jk'l}$ with $\sum_j w_{jkl} = q_{kl}$ and $\sum_j w_{jk'l} = q_{k'l}$, which is possible because only the totals are constrained. The cost is again unchanged. The two instances therefore have the same feasible designs with the same routing values and costs, hence the same optimal value, the same optimal designs and the same decision answers. The merge map preserves the cost exactly, and the split map, run after the trim, never raises it, so the two maps carry feasible complete solutions across in both directions and preserve the optimal routing value of every design, although the trim may strictly lower the cost of a routing that carries surplus, so the correspondence is not a cost-preserving bijection on all complete solutions. Iterating over equal columns leaves at most one customer per distinct column, so $|K| \le \tau$. □

B.5. Capacity capping

Observation 4 (capacity capping). *Replacing every capacity b_{il} by $\min(b_{il}, D_l)$ and every p_{jl} by $\min(p_{jl}, D_l)$ preserves feasibility, routing value and cost of every design, hence the optimum and both decision answers.*

Proof of Observation 4. Fix a design of the original instance. Any routing of it can be trimmed, as in the proof of Proposition 1 in A, until each product's total flow across each stage equals D_l, at no increase in cost. A trimmed routing sends at most D_l units of product l through any single factory or depot, so it respects the capped capacities and is a routing of the capped instance at the same cost. Conversely every routing of the capped instance is one of the original, because capping only lowers capacities, so the two instances give every design the same routing value. Feasibility is also unchanged, since $\sum_i \min(b_{il}, D_l)\, y_i \ge D_l$ holds if and only if $\sum_i b_{il}\, y_i \ge D_l$, either because some open factory alone reaches D_l or because no cap binds, and the depot side is identical. The two instances therefore share feasible designs, routing values and costs, hence the optimum and both decision answers. □

B.6. Kernel and compression under one-sided bounds

Proposition 9 (kernel and compression under one-sided bounds). *Fix $c^\star$ and let $\kappa = |I|+|J|+|L|+\tau$. The subclass of* MP-TSCFLP(B) *whose opening and transport costs f, g, c, d and whose total demands $D_1, \dots, D_{|L|}$ are integers at most $\kappa^{c^\star}$, with capacities and budget unrestricted, admits a polynomial kernel in κ, an equivalent instance of* MP-TSCFLP(B) *itself of* poly(κ) *bits. The subclass with* $\max(f, g, c, d, B) \le \kappa^{c^\star}$ *and capacities and demands unrestricted admits a polynomial compression in κ into a fixed sign-condition language, which is not itself a location problem. The first subclass is nontrivial. The strongly hard slice with $|K| = |L| = 1$ of Theorem 6 has costs in $\{0, 1\}$ and a single customer whose demand, the only product total, is $|I|\,(|J| + 1) \le \kappa^2$, so the slice lies in the subclass for every $c^\star \ge 2$.*

Proof of Proposition 9. Consider the first subclass, in which the opening and transport costs and the totals D_l are integers at most $\kappa^{c^\star}$ while the capacities and the budget carry no bound. Proposition 8 leaves $|K| \le \tau$ customers, each merged demand entry is a sum of original entries of the same product and hence at most $D_l \le \kappa^{c^\star}$, and the totals D_l themselves survive the merge unchanged. The capping of Observation 4 then bounds every capacity by $\max_l D_l \le \kappa^{c^\star}$. After these two reductions at most κ facilities, $\tau \le \kappa$ customers and $|L| \le \kappa$ products remain and every datum except the budget is an integer at most $\kappa^{c^\star}$, so the instance without its budget occupies poly(κ) bits and only B stands in the way of a kernel.

The budget is capped against the all-open design. If that design is infeasible then no design is feasible, since every design lies componentwise below it and feasibility is monotone by Proposition 3, and the kernel outputs a fixed no-instance, a single facility pair of zero capacities facing one unit of demand. Otherwise one

oracle call prices the all-open design in polynomial time (Proposition 1) at $U = f(I) + g(J) + v(\mathbf{1}, \mathbf{1})$. The fixed charges total at most $\kappa \cdot \kappa^{c^\star}$, at most $\sum_l D_l \le \kappa^{c^\star+1}$ units of flow move in all, and each unit traverses one first-stage and one second-stage arc of costs at most $\kappa^{c^\star}$ each, so $U \le \kappa^{c^\star+1} + 2\kappa^{2c^\star+1} \le 3\kappa^{2c^\star+1}$. If $B \ge U$ the all-open design already witnesses acceptance and the kernel outputs a fixed yes-instance, one facility pair of zero costs and zero demand with budget zero. Otherwise $B < U \le 3\kappa^{2c^\star+1}$, the budget fits in $O(\log \kappa)$ bits for fixed $c^\star$, and the reduced instance, an ordinary instance of MP-TSCFLP(B) equivalent to the original one, occupies $\mathrm{poly}(\kappa)$ bits in total, which is the kernel claimed.

In the second subclass the costs and the budget are at most $\kappa^{c^\star}$ and the large numbers are the capacities and demands, so no polynomially bounded cost of a witness design is available to cap them and the argument changes tool. The compression treats the extended vector $\Psi = (b, p, q, 1)$ at once, whose final coordinate absorbs the additive constants below, and customer aggregation brings the dimension of Ψ down to $\mathrm{poly}(\kappa)$. For any fixed design S the routing value is the sum over products l of a minimum-cost flow in the network $N_l(S)$ of Proposition 1, and each per-product minimum is attained at a basic (vertex) flow. The node–arc incidence matrix of each $N_l(S)$ is totally unimodular [30], so every subdeterminant lies in $\{-1, 0, 1\}$, a nonsingular basis matrix M has determinant ± 1, and by Cramer's rule each entry of M^{-1}, a cofactor divided by that determinant, lies in $\{-1, 0, 1\}$ as well. Every basic-flow value is therefore a $\{-1, 0, 1\}$-combination of right-hand sides drawn from Ψ, so the nonnegativity of a basic flow and the comparison of its routing cost against the budget B minus the fixed charge of the design are sign conditions on integer linear forms in Ψ. The coefficients of these forms on the capacity–demand entries are signed sums of the small cost entries, of magnitude $\mathrm{poly}(\kappa)$, the coefficient on the final entry is an integer of magnitude at most B plus the total fixed charge, again $\mathrm{poly}(\kappa)$, and the tests (F1) and (F2) contribute further forms with coefficients in $\{-1, 0, 1\}$. Acceptance is thus a Boolean combination of sign conditions on integer linear forms in Ψ of dimension $\nu = \mathrm{poly}(\kappa)$ with coefficients bounded by some $\Delta_{\max} = \mathrm{poly}(\kappa)$. The Frank–Tardos theorem [18, 31] produces, for any prescribed bound N_0, in time polynomial in the encoding length of Ψ and in $\log N_0$, an integer vector Ψ' of $\mathrm{poly}(\nu, \log N_0)$ bits such that $\mathrm{sign}(a \cdot \Psi') = \mathrm{sign}(a \cdot \Psi)$ for every integer vector a whose ℓ_1-norm is strictly below N_0. We invoke it with $N_0 = \nu\, \Delta_{\max} + 1$. Every acceptance form has $\|a\|_\infty \le \Delta_{\max}$ and hence $\|a\|_1 \le \nu\, \Delta_{\max} = N_0 - 1$, so every form is strictly covered, and $\log N_0 = O(\log \kappa)$ for fixed $c^\star$, so Ψ' has $\mathrm{poly}(\kappa)$ bits and replacing Ψ by Ψ' preserves the answer. The output pairs the unchanged small data with the compressed vector, the Boolean combination of sign conditions still decides acceptance, and the map is a polynomial compression onto the fixed language of these sign conditions rather than a well-formed instance of MP-TSCFLP(B), because entries of Ψ' may be negative. For each fixed $c^\star$ that target language is a fixed problem, and the compression framework places no requirement that it lie in NP. With both sides free the acceptance condition compares products of a large cost with a large capacity, the forms become bilinear, and the tool no longer applies. □